

CSE thesis XGBoost P3

January 24, 2026

```
[1]: """
Reusable XGBoost Pipeline for Microfinance Impact Analysis
-----

Matches your Random Forest pipeline structure:
- Universal target detection (CI, BF1, BF_1, BF 1, bf-1, etc.)
- Median imputation + StandardScaler in a Pipeline
- 10-fold Stratified CV with multiple metrics
- Train/Test/Validation split option
- Comprehensive evaluation + plots + saving outputs (metrics, preds, model,
  ↳feature importances)

NOTE:
- This is for binary classification (0/1 target).
"""

# =====
# 0) INSTALLS
# =====
# If you're in Colab, uncomment:
# !pip -q install xgboost openpyxl joblib

# =====
# 0) IMPORTS
# =====
import os, re, warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, StratifiedKFold,
  ↳cross_validate
from sklearn.metrics import (
    classification_report, confusion_matrix, roc_auc_score, roc_curve,
    precision_recall_curve, average_precision_score,
```

```

    accuracy_score, precision_score, recall_score, f1_score
)
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
import joblib

from xgboost import XGBClassifier

RANDOM_STATE = 42

# =====
# 1) DATA LOADING
# =====
def load_excel_from_colab():
    """Upload Excel in Google Colab."""
    try:
        from google.colab import files
        print(" Please select the Excel file with your encoded dataset...")
        uploaded = files.upload()
        path = list(uploaded.keys())[0]
        df = pd.read_excel(path)
        print(f" Loaded dataset: {df.shape} (rows, cols)")
        return df
    except Exception as e:
        raise RuntimeError("Colab upload failed.") from e

def load_excel_locally():
    """Upload Excel when running locally."""
    try:
        import tkinter as tk
        from tkinter import filedialog
        print(" Please select the Excel file...")
        root = tk.Tk()
        root.withdraw()
        file_path = filedialog.askopenfilename(
            title="Select Excel file",
            filetypes=[("Excel files", "*.xlsx *.xls")]
        )
        if not file_path:
            raise ValueError("No file selected")
        df = pd.read_excel(file_path)
        print(f" Loaded dataset: {df.shape} (rows, cols)")
        return df
    except Exception as e:

```

```

print(f" File dialog failed: {e}")
file_path = input(" Enter full path to your Excel file: ")
df = pd.read_excel(file_path)
print(f" Loaded dataset: {df.shape} (rows, cols)")
return df

# =====
# 2) UNIVERSAL TARGET SELECTION
# =====
def detect_target_column(df: pd.DataFrame, target: str) -> str:
    """
    Universal target detector:
    1) If `target` matches an existing column exactly -> use it.
    2) If target looks like BF<number>, try common BF aliases (BF 1, BF_1,
    ↪bf-1, etc.).
    3) Else try case-insensitive exact match.
    4) Else ask user to type exact column name.
    """
    if target is None or str(target).strip() == "":
        raise ValueError("target cannot be empty. Example: target='CI' or
    ↪target='BF1'")

    target = str(target).strip()

    # (1) Exact match
    if target in df.columns:
        print(f" Found target column (exact): {target}")
        return target

    # (2) BF alias support if user typed BF-style
    m = re.match(r"^s*BF\s*(\d+)\s*$", target, flags=re.I)
    if m:
        k = m.group(1)
        aliases = [
            f"BF{k}", f"bf{k}",
            f"BF {k}", f"bf {k}",
            f"BF_{k}", f"bf_{k}",
            f"BF-{k}", f"bf-{k}",
        ]
        for a in aliases:
            if a in df.columns:
                print(f" Found target column (BF alias): {a}")
                return a

    # regex scan for bf variants
    pat = re.compile(rf"\bbf\s*[_-]?s*{k}\b", flags=re.I)

```

```

        for col in df.columns:
            if pat.search(str(col)):
                print(f" Found target column (BF regex): {col}")
                return col

    # (3) Case-insensitive exact match
    lower_map = {str(c).strip().lower(): c for c in df.columns}
    if target.lower() in lower_map:
        real_col = lower_map[target.lower()]
        print(f" Found target column (case-insensitive): {real_col}")
        return real_col

    # (4) Ask user
    print(f" Target column '{target}' not found automatically.")
    print(" Available columns:")
    print(df.columns.tolist())
    user_col = input(" Enter the exact target column name: ").strip()
    if user_col in df.columns:
        print(f" Using specified target column: {user_col}")
        return user_col
    raise ValueError(f"Column '{user_col}' not found in dataset")

def preprocess_target_binary(y_series: pd.Series, target_label: str):
    """
    Convert target responses into binary classification (expects 0/1 after
    ↪ cleaning).
    """
    y_numeric = pd.to_numeric(y_series, errors="coerce")
    valid_mask = y_numeric.notna() & np.isfinite(y_numeric)
    y_clean = y_numeric[valid_mask]

    if len(y_clean) == 0:
        raise ValueError(f" No valid values for {target_label} after cleaning")

    print(f"\n TARGET ANALYSIS: {target_label}")
    print("-" * 40)
    print("Value distribution:")
    print(y_clean.value_counts().sort_index())

    unique_vals = sorted(y_clean.unique())
    print(f"Unique values: {unique_vals}")

    y_binary = y_clean.astype(int)

    class_balance = y_binary.value_counts(normalize=True).round(3)
    print(f" Final class balance: {class_balance.to_dict()}")

```

```

    return y_binary, valid_mask

# =====
# 3) FEATURE ENGINEERING
# =====
def build_feature_matrix(df_clean: pd.DataFrame, target_col: str):
    """Prepare features for ML (numeric-only)."""
    print("\n FEATURE ENGINEERING")
    print("-" * 40)

    X = df_clean.drop(columns=[target_col], errors="ignore")
    numeric_cols = X.select_dtypes(include=[np.number]).columns.tolist()
    print(f" Found {len(numeric_cols)} numeric features")

    if len(numeric_cols) == 0:
        raise ValueError(" No numeric features found for modeling")

    print(" Handling missing values with median imputation...")
    imputer = SimpleImputer(strategy="median")
    X_imputed = imputer.fit_transform(X[numeric_cols])
    X_imputed = pd.DataFrame(X_imputed, columns=numeric_cols, index=X.index)

    non_constant_cols = [c for c in X_imputed.columns if X_imputed[c].nunique()
↳ 1]
    dropped = len(numeric_cols) - len(non_constant_cols)
    if dropped > 0:
        print(f" Removed {dropped} constant columns")

    X_final = X_imputed[non_constant_cols]
    print(f" Final feature matrix: {X_final.shape} (rows, cols)")
    return X_final, non_constant_cols

# =====
# 4) CROSS-VALIDATION
# =====
def _compute_scale_pos_weight(y):
    # For XGBoost: scale_pos_weight = (#neg / #pos)
    pos = int((y == 1).sum())
    neg = int((y == 0).sum())
    if pos == 0:
        return 1.0
    return neg / pos

```

```

def perform_cross_validation(X, y, cv_folds=10):
    print(f"\n PERFORMING {cv_folds}-FOLD CROSS-VALIDATION")
    print("=" * 50)

    preprocessor = Pipeline([
        ("impute", SimpleImputer(strategy="median")),
        ("scale", StandardScaler())
    ])

    spw = _compute_scale_pos_weight(y)

    xgb = XGBClassifier(
        n_estimators=600,
        learning_rate=0.05,
        max_depth=4,
        min_child_weight=3,
        subsample=0.85,
        colsample_bytree=0.85,
        gamma=0.0,
        reg_lambda=1.0,
        reg_alpha=0.0,
        objective="binary:logistic",
        eval_metric="logloss",
        random_state=RANDOM_STATE,
        n_jobs=-1,
        scale_pos_weight=spw,
        tree_method="hist" # fast + stable in CPU
    )

    pipe = Pipeline([("pre", preprocessor), ("xgb", xgb)])

    scoring = {
        "accuracy": "accuracy",
        "precision": "precision",
        "recall": "recall",
        "f1": "f1",
        "roc_auc": "roc_auc",
        "average_precision": "average_precision"
    }

    cv_results = cross_validate(
        pipe, X, y,
        cv=StratifiedKFold(n_splits=cv_folds, shuffle=True,
        random_state=RANDOM_STATE),
        scoring=scoring,
        return_train_score=True,
        n_jobs=-1

```

```

)

print(" CROSS-VALIDATION RESULTS (Mean ± Std):")
print("-" * 40)

metrics_summary = {}
for metric in scoring.keys():
    test_scores = cv_results[f"test_{metric}"]
    train_scores = cv_results[f"train_{metric}"]

    test_mean = float(np.mean(test_scores))
    test_std = float(np.std(test_scores))
    train_mean = float(np.mean(train_scores))

    metrics_summary[metric] = {
        "test_mean": test_mean,
        "test_std": test_std,
        "train_mean": train_mean
    }

    print(f"{metric.upper():<15}: {test_mean:.4f} ± {test_std:.4f} (train:␣
↪{train_mean:.4f})")

    overfit_gap = metrics_summary["accuracy"]["train_mean"] -␣
↪metrics_summary["accuracy"]["test_mean"]
    print(f"\n Overfitting Analysis:")
    print(f" Train-Test Accuracy Gap: {overfit_gap:.4f}")
    print(" Potential overfitting detected" if overfit_gap > 0.05 else " ␣
↪ Model generalizes well")

    return metrics_summary, cv_results

# =====
# 5) TRAIN/TEST/VALIDATION SPLITS + TRAINING
# =====
def create_validation_set(X, y, validation_size=0.2):
    print(f"\n CREATING VALIDATION SET ({validation_size*100:.0f}% of data)")
    print("=" * 50)

    X_train_test, X_val, y_train_test, y_val = train_test_split(
        X, y, test_size=validation_size, stratify=y, random_state=RANDOM_STATE
    )

    X_train, X_test, y_train, y_test = train_test_split(
        X_train_test, y_train_test, test_size=0.25, stratify=y_train_test,␣
↪random_state=RANDOM_STATE

```

```

)

print(f" Training set: {X_train.shape[0]} samples")
print(f" Test set: {X_test.shape[0]} samples")
print(f" Validation set: {X_val.shape[0]} samples")

return X_train, X_test, X_val, y_train, y_test, y_val

def train_with_comprehensive_validation(X, y, use_validation_set=True):
    print("\n MODEL TRAINING: XGBoost")
    print("-" * 40)

    preprocessor = Pipeline([
        ("impute", SimpleImputer(strategy="median")),
        ("scale", StandardScaler())
    ])

    spw = _compute_scale_pos_weight(y)

    xgb = XGBClassifier(
        n_estimators=600,
        learning_rate=0.05,
        max_depth=4,
        min_child_weight=3,
        subsample=0.85,
        colsample_bytree=0.85,
        gamma=0.0,
        reg_lambda=1.0,
        reg_alpha=0.0,
        objective="binary:logistic",
        eval_metric="logloss",
        random_state=RANDOM_STATE,
        n_jobs=-1,
        scale_pos_weight=spw,
        tree_method="hist"
    )

    pipe = Pipeline([("pre", preprocessor), ("xgb", xgb)])

    if use_validation_set:
        X_train, X_test, X_val, y_train, y_test, y_val = \
        ↪ create_validation_set(X, y)

        # Early stopping uses a validation set, but sklearn.Pipeline
        ↪ complicates passing eval_set cleanly.
        # So we keep training stable via regularization + conservative params.

```

```

    print(" Training model with regularization...")
    pipe.fit(X_train, y_train)

    return pipe, X_train, X_test, X_val, y_train, y_test, y_val

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=RANDOM_STATE
)
print(f" Training set: {X_train.shape[0]} samples")
print(f" Test set: {X_test.shape[0]} samples")
print(" Training model with regularization...")
pipe.fit(X_train, y_train)
return pipe, X_train, X_test, None, y_train, y_test, None

# =====
# 6) EVALUATION
# =====
def evaluate_model_comprehensive(model, X_test, y_test, dataset_name="Test"):
    y_pred = model.predict(X_test)
    y_proba = model.predict_proba(X_test)[:, 1]

    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred, zero_division=0)
    rec = recall_score(y_test, y_pred, zero_division=0)
    f1 = f1_score(y_test, y_pred, zero_division=0)
    roc_auc = roc_auc_score(y_test, y_proba)
    pr_auc = average_precision_score(y_test, y_proba)

    print("\n" + "=" * 60)
    print(f" {dataset_name.upper()} SET COMPREHENSIVE EVALUATION")
    print("=" * 60)
    print(f" Accuracy:  {acc:.4f}")
    print(f" Precision: {prec:.4f}")
    print(f" Recall:    {rec:.4f}")
    print(f" F1-Score:   {f1:.4f}")
    print(f" ROC-AUC:    {roc_auc:.4f}")
    print(f" PR-AUC:     {pr_auc:.4f}")

    print(f"\n {dataset_name} Classification Report:")
    print(classification_report(y_test, y_pred, digits=3))

    cm = confusion_matrix(y_test, y_pred)
    print(f" {dataset_name} Confusion Matrix:")
    print(" [[TN FP]\n [FN TP]]")
    print(cm)

```

```

return y_pred, y_proba, {
    "accuracy": acc, "precision": prec, "recall": rec, "f1": f1,
    "roc_auc": roc_auc, "pr_auc": pr_auc
}

# =====
# 7) VISUALIZATIONS
# =====
def create_validation_comparison_plot(cv_metrics, output_dir, prefix):
    metrics_names = list(cv_metrics.keys())
    test_means = [cv_metrics[m]["test_mean"] for m in metrics_names]
    test_stds = [cv_metrics[m]["test_std"] for m in metrics_names]

    plt.figure(figsize=(12, 6))
    x_pos = np.arange(len(metrics_names))
    bars = plt.bar(x_pos, test_means, yerr=test_stds, capsize=5, alpha=0.7)

    for i, b in enumerate(bars):
        b.set_color(f"C{i}")

    plt.xlabel("Performance Metrics")
    plt.ylabel("Score")
    plt.title(f"10-Fold Cross-Validation Performance ({prefix.upper()})")
    plt.xticks(x_pos, [m.upper() for m in metrics_names], rotation=0)
    plt.ylim(0, 1)
    plt.grid(True, alpha=0.3, axis="y")

    for i, (mean, std) in enumerate(zip(test_means, test_stds)):
        plt.text(i, mean + 0.02, f"{mean:.3f}±{std:.3f}", ha="center",
        va="bottom", fontsize=10)

    plt.tight_layout()
    plt.savefig(f"{output_dir}/{prefix}_cross_validation_results.png", dpi=200,
    bbox_inches="tight")
    plt.show()

def create_comprehensive_visualizations(y_test, y_proba, model, feature_names,
    output_dir, prefix, cv_metrics=None):
    os.makedirs(output_dir, exist_ok=True)

    xgb_model = model.named_steps["xgb"]
    imp_df = pd.DataFrame({
        "feature": feature_names,
        "importance": xgb_model.feature_importances_
    }).sort_values("importance", ascending=False)

```

```

imp_df.to_csv(f"{output_dir}/{prefix}_feature_importances.csv", index=False)

# Feature importance plot
plt.figure(figsize=(12, 8))
top_features = imp_df.head(15).iloc[::-1]
plt.barh(range(len(top_features)), top_features["importance"], alpha=0.7)
plt.yticks(range(len(top_features)), top_features["feature"])
plt.xlabel("Feature Importance Score")
plt.title(f"Top 15 Features for {prefix.upper()} Prediction (XGBoost)")
plt.grid(True, alpha=0.3, axis="x")
plt.tight_layout()
plt.savefig(f"{output_dir}/{prefix}_feature_importances.png", dpi=200,
↳ bbox_inches="tight")
plt.show()

# ROC + PR curves
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))

fpr, tpr, _ = roc_curve(y_test, y_proba)
roc_auc = roc_auc_score(y_test, y_proba)
ax1.plot(fpr, tpr, label=f"AUC = {roc_auc:.3f}", linewidth=2, color="C0")
ax1.plot([0, 1], [0, 1], "k--", alpha=0.5)
ax1.set_xlabel("False Positive Rate")
ax1.set_ylabel("True Positive Rate")
ax1.set_title(f"ROC Curve ({prefix.upper()})")
ax1.legend()
ax1.grid(True, alpha=0.3)

precision_vals, recall_vals, _ = precision_recall_curve(y_test, y_proba)
pr_auc = average_precision_score(y_test, y_proba)
ax2.plot(recall_vals, precision_vals, label=f"AP = {pr_auc:.3f}",
↳ linewidth=2, color="C1")
ax2.set_xlabel("Recall")
ax2.set_ylabel("Precision")
ax2.set_title(f"Precision-Recall Curve ({prefix.upper()})")
ax2.legend()
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig(f"{output_dir}/{prefix}_roc_pr_curves.png", dpi=200,
↳ bbox_inches="tight")
plt.show()

# Confusion matrix heatmap
y_pred = (y_proba >= 0.5).astype(int)
cm = confusion_matrix(y_test, y_pred)

```

```

plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=["Pred 0", "Pred 1"],
            yticklabels=["True 0", "True 1"])
plt.title(f"Confusion Matrix ({prefix.upper()}) (XGBoost)")
plt.ylabel("True Label")
plt.xlabel("Predicted Label")
plt.tight_layout()
plt.savefig(f"{output_dir}/{prefix}_confusion_matrix.png", dpi=200,
↳bbox_inches="tight")
plt.show()

if cv_metrics:
    create_validation_comparison_plot(cv_metrics, output_dir, prefix)

return imp_df

# =====
# 8) SAVE RESULTS
# =====
def save_comprehensive_results(model, metrics, imp_df, df_clean, target_col,
↳feature_names, output_dir, prefix, cv_metrics=None):
    print(f"\n SAVING OUTPUTS to {output_dir}/ ...")
    os.makedirs(output_dir, exist_ok=True)

    with open(f"{output_dir}/{prefix}_comprehensive_metrics.txt", "w",
↳encoding="utf-8") as f:
        f.write(f"COMPREHENSIVE XGBOOST RESULTS ({prefix.upper()})\n")
        f.write("=" * 60 + "\n")
        f.write(f"Dataset shape: {df_clean.shape}\n")
        f.write(f"Target column: {target_col}\n")
        f.write(f"Features used: {len(feature_names)}\n\n")

        f.write("TEST SET METRICS:\n")
        for k, v in metrics.items():
            f.write(f"{k}: {v:.4f}\n")
        f.write("\n")

        if cv_metrics:
            f.write("10-FOLD CROSS-VALIDATION (Mean ± Std) WITH MULTIPLE
↳METRICS:\n")
            for metric, scores in cv_metrics.items():
                f.write(f"{metric}: {scores['test_mean']:.4f} ±
↳{scores['test_std']:.4f}\n")
            f.write("\n")

```

```

f.write("TOP 10 FEATURES:\n")
for _, row in imp_df.head(10).iterrows():
    f.write(f"{row['feature']}: {row['importance']:.6f}\n")

# Full-dataset predictions
X_full = df_clean.drop(columns=[target_col], errors="ignore")
X_full = X_full.reindex(columns=feature_names)

preds = model.predict_proba(X_full)[: , 1]
predictions_df = df_clean.copy()
predictions_df[f"{prefix}_Pred_Prob"] = preds
predictions_df[f"{prefix}_Pred"] = (preds >= 0.5).astype(int)
predictions_df[f"{prefix}_Correct"] = (predictions_df[f"{prefix}_Pred"] ==
↳ predictions_df[target_col]).astype(int)

predictions_df.to_excel(f"{output_dir}/{prefix}_predictions.xlsx",
↳ index=False)
joblib.dump(model, f"{output_dir}/{prefix}_xgb_model.joblib")

print(" Saved metrics, plots, predictions, and model.")

# =====
# 9) MAIN PIPELINE (UNIVERSAL TARGET)
# =====
def run_comprehensive_analysis(
    target="CI",
    use_cross_validation=True,
    use_validation_set=True,
    df=None
):
    """
    Run the full pipeline for ANY target column you type.
    """
    try:
        prefix = re.sub(r"[^a-zA-Z0-9]+", "_", str(target).strip().lower()).
↳ strip("_")
        output_dir = f"{prefix}_comprehensive_outputs_xgb"

        print(" COMPREHENSIVE MICROFINANCE ANALYSIS (XGBoost)")
        print("=" * 60)
        print(f" Target: {target}")
        print(f" Output folder: {output_dir}/")
        print("=" * 60)

        if df is None:

```

```

try:
    import google.colab # noqa: F401
    df = load_excel_from_colab()
except Exception:
    df = load_excel_locally()

print(f" Initial dataset shape: {df.shape}")

target_col = detect_target_column(df, target=target)

y, valid_mask = preprocess_target_binary(df[target_col],
↪target_label=target_col)
df_clean = df.loc[valid_mask].copy()
print(f" Clean dataset: {df_clean.shape} (after removing invalid,
↪target rows)")

X_final, feature_names = build_feature_matrix(df_clean, target_col)

cv_metrics = None
if use_cross_validation:
    cv_metrics, _ = perform_cross_validation(X_final, y, cv_folds=10)

model, X_train, X_test, X_val, y_train, y_test, y_val =
↪train_with_comprehensive_validation(
    X_final, y, use_validation_set=use_validation_set
)

_, y_proba, test_metrics = evaluate_model_comprehensive(model, X_test,
↪y_test, "Test")

if X_val is not None and y_val is not None:
    evaluate_model_comprehensive(model, X_val, y_val, "Validation")

imp_df = create_comprehensive_visualizations(
    y_test=y_test,
    y_proba=y_proba,
    model=model,
    feature_names=feature_names,
    output_dir=output_dir,
    prefix=prefix,
    cv_metrics=cv_metrics
)

save_comprehensive_results(
    model=model,
    metrics=test_metrics,
    imp_df=imp_df,

```

```

        df_clean=df_clean,
        target_col=target_col,
        feature_names=feature_names,
        output_dir=output_dir,
        prefix=prefix,
        cv_metrics=cv_metrics
    )

    print("\n DONE.")
    return model, test_metrics, imp_df, cv_metrics

except Exception as e:
    print(f"\n Analysis failed: {type(e).__name__}: {e}")
    import traceback
    traceback.print_exc(limit=2)
    return None, None, None, None

# ===== USAGE EXAMPLES =====
if __name__ == "__main__":
    run_comprehensive_analysis(target="BF1", use_cross_validation=True,
    ↪ use_validation_set=True)

    # Other examples:
    # run_comprehensive_analysis(target="CI", use_cross_validation=True,
    ↪ use_validation_set=True)
    # for t in ["CI", "BF1", "BF3", "BF4"]:
    #     run_comprehensive_analysis(target=t, use_cross_validation=True,
    ↪ use_validation_set=True)

```

COMPREHENSIVE MICROFINANCE ANALYSIS (XGBoost)

```

=====
Target: BF1
Output folder: bf1_comprehensive_outputs_xgb/
=====

Please select the Excel file with your encoded dataset...

<IPython.core.display.HTML object>

Saving rf_dataset_encoded_dataset.xlsx to rf_dataset_encoded_dataset.xlsx
Loaded dataset: (1002, 85) (rows, cols)
Initial dataset shape: (1002, 85)
Target column 'BF1' not found automatically.
Available columns:
['D', 'AY', 'AZ', 'BA', 'BB', 'BC', 'BD', 'BE_01', 'BE_02', 'BE_03', 'BE_04',
'BE_05', 'BE_06', 'BE_07', 'BE_08', 'BE_09', 'BE_10', 'BF', 'BG_01', 'BG_02',
'BG_03', 'BG_04', 'BG_05', 'BG_06', 'BH', 'BI', 'BN', 'BO', 'BP', 'BQ', 'BR',
'BS', 'BT', 'BU', 'BV', 'BW_01', 'BW_02', 'BW_03', 'BW_04', 'BW_05', 'BX', 'BY',

```

'CB', 'CD', 'CE', 'CF', 'CG', 'CH', 'CI', 'CK', 'CL', 'CM_01', 'CM_02', 'CM_03',
'CM_04', 'CM_05', 'CM_06', 'CN', 'CO_01', 'CO_02', 'CO_03', 'CO_04', 'CO_05',
'CO_06', 'CO_07', 'CO_08', 'CO_09', 'CP', 'CR_01', 'CR_02', 'CR_03', 'CR_04',
'CS', 'CT_01', 'CT_02', 'CT_03', 'CT_04', 'CT_05', 'CT_06', 'CU', 'Occupation',
'Financial Behavior', 'Family', 'Lifestyle', 'Digital Literacy']

Enter the exact target column name: BG_01

Using specified target column: BG_01

TARGET ANALYSIS: BG_01

Value distribution:

BG_01

0 533

1 469

Name: count, dtype: int64

Unique values: [np.int64(0), np.int64(1)]

Final class balance: {0: 0.532, 1: 0.468}

Clean dataset: (1002, 85) (after removing invalid target rows)

FEATURE ENGINEERING

Found 84 numeric features

Handling missing values with median imputation...

Final feature matrix: (1002, 84) (rows, cols)

PERFORMING 10-FOLD CROSS-VALIDATION

=====

CROSS-VALIDATION RESULTS (Mean ± Std):

ACCURACY	: 0.8842 ± 0.0251 (train: 0.9989)
PRECISION	: 0.8704 ± 0.0218 (train: 0.9981)
RECALL	: 0.8850 ± 0.0495 (train: 0.9995)
F1	: 0.8769 ± 0.0284 (train: 0.9988)
ROC_AUC	: 0.9364 ± 0.0293 (train: 1.0000)
AVERAGE_PRECISION	: 0.9337 ± 0.0252 (train: 1.0000)

Overfitting Analysis:

Train-Test Accuracy Gap: 0.1147

Potential overfitting detected

MODEL TRAINING: XGBoost

CREATING VALIDATION SET (20% of data)

=====

Training set: 600 samples

Test set: 201 samples

Validation set: 201 samples

Training model with regularization...

=====

TEST SET COMPREHENSIVE EVALUATION

=====

Accuracy: 0.8408
Precision: 0.8523
Recall: 0.7979
F1-Score: 0.8242
ROC-AUC: 0.8922
PR-AUC: 0.8841

Test Classification Report:

	precision	recall	f1-score	support
0	0.832	0.879	0.855	107
1	0.852	0.798	0.824	94
accuracy			0.841	201
macro avg	0.842	0.838	0.839	201
weighted avg	0.841	0.841	0.840	201

Test Confusion Matrix:

[[TN FP]
[FN TP]]
[[94 13]
[19 75]]

=====

VALIDATION SET COMPREHENSIVE EVALUATION

=====

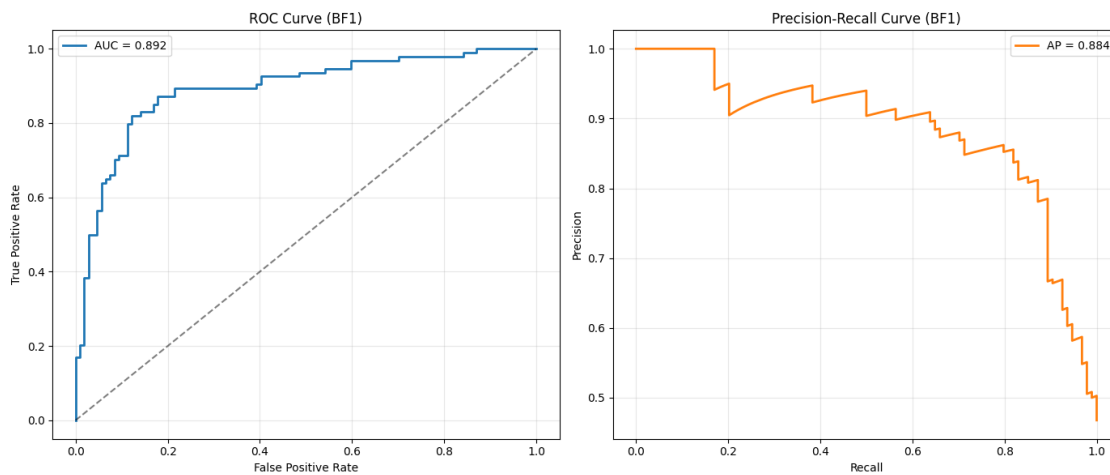
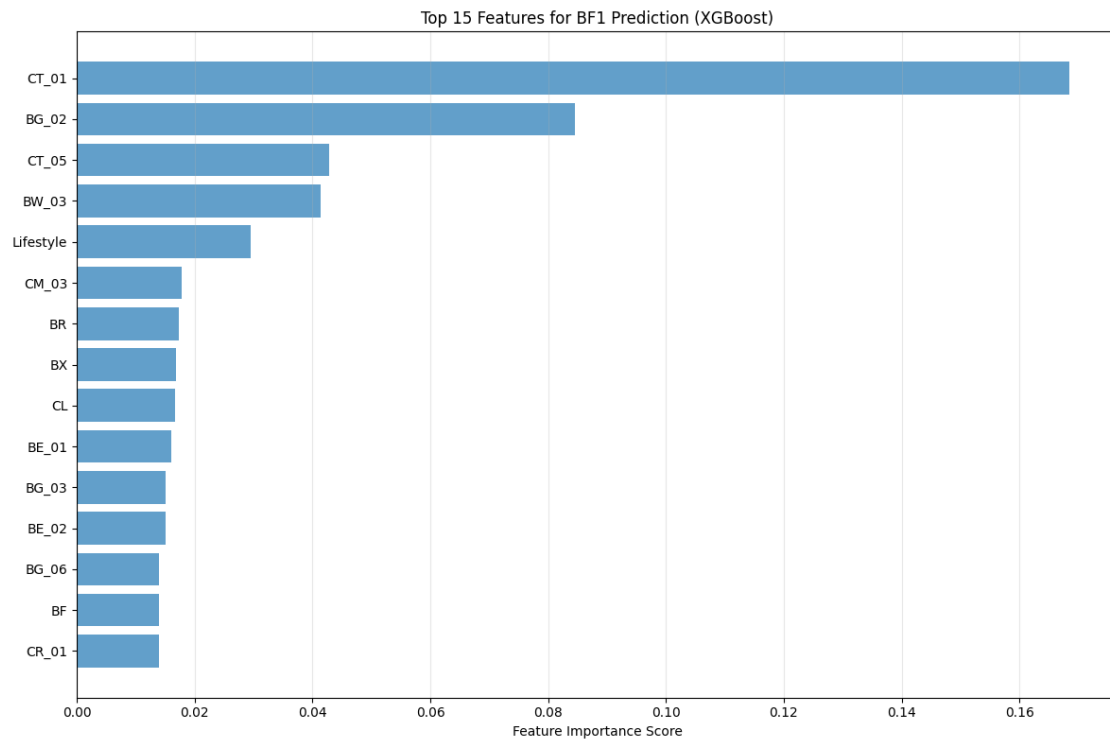
Accuracy: 0.8557
Precision: 0.8495
Recall: 0.8404
F1-Score: 0.8449
ROC-AUC: 0.9259
PR-AUC: 0.9280

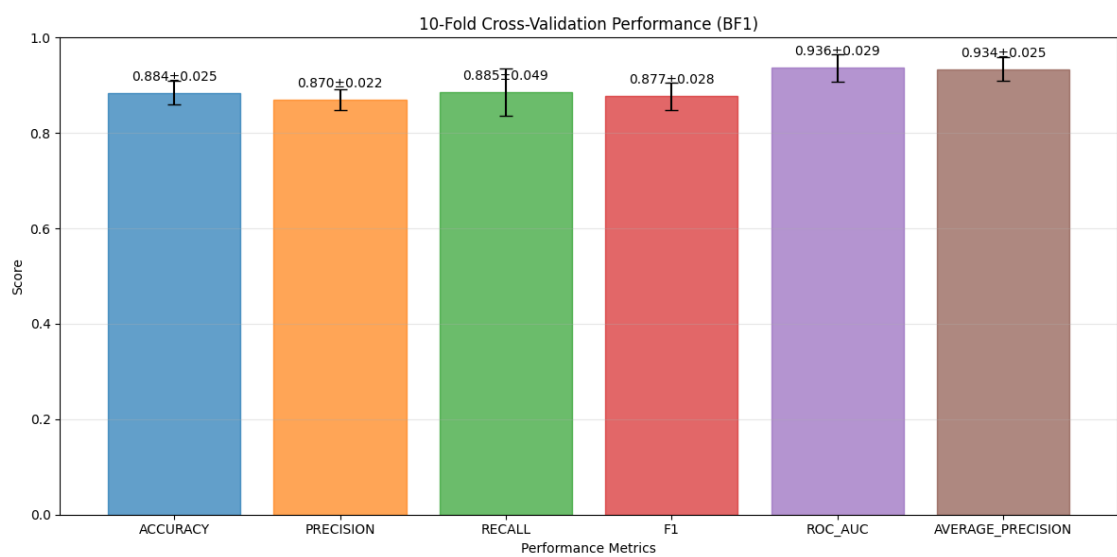
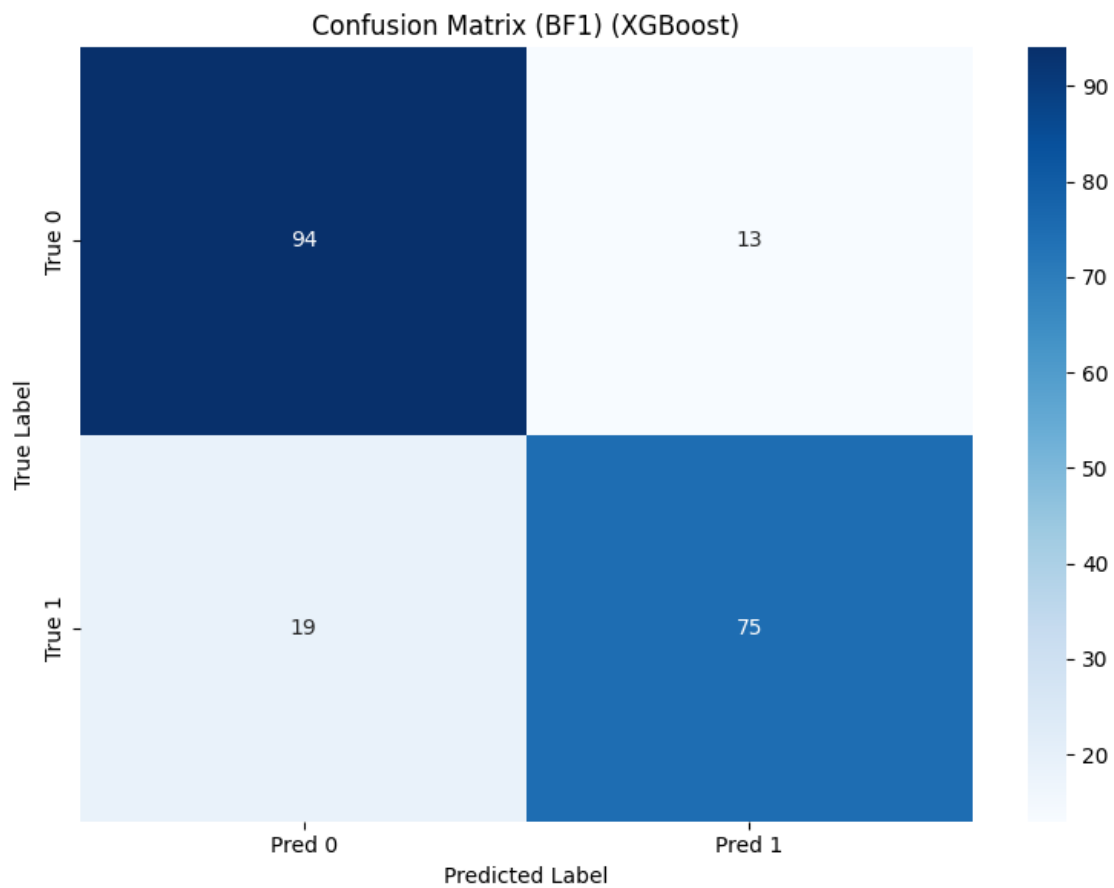
Validation Classification Report:

	precision	recall	f1-score	support
0	0.861	0.869	0.865	107
1	0.849	0.840	0.845	94
accuracy			0.856	201
macro avg	0.855	0.855	0.855	201
weighted avg	0.856	0.856	0.856	201

Validation Confusion Matrix:

[[TN FP
[FN TP]]
[[93 14]
[15 79]]





SAVING OUTPUTS to bf1_comprehensive_outputs_xgb/ ...
Saved metrics, plots, predictions, and model.

DONE.

```
[2]: """
Reusable XGBoost Pipeline for Microfinance Impact Analysis
-----

Matches your Random Forest pipeline structure:
- Universal target detection (CI, BF1, BF_1, BF 1, bf-1, etc.)
- Median imputation + StandardScaler in a Pipeline
- 10-fold Stratified CV with multiple metrics
- Train/Test/Validation split option
- Comprehensive evaluation + plots + saving outputs (metrics, preds, model,
  ↳feature importances)

NOTE:
- This is for binary classification (0/1 target).
"""

# =====
# 0) INSTALLS
# =====
# If you're in Colab, uncomment:
# !pip -q install xgboost openpyxl joblib

# =====
# 0) IMPORTS
# =====
import os, re, warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, StratifiedKFold,
  ↳cross_validate
from sklearn.metrics import (
    classification_report, confusion_matrix, roc_auc_score, roc_curve,
    precision_recall_curve, average_precision_score,
    accuracy_score, precision_score, recall_score, f1_score
)
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
```

```

import joblib

from xgboost import XGBClassifier

RANDOM_STATE = 42

# =====
# 1) DATA LOADING
# =====
def load_excel_from_colab():
    """Upload Excel in Google Colab."""
    try:
        from google.colab import files
        print(" Please select the Excel file with your encoded dataset...")
        uploaded = files.upload()
        path = list(uploaded.keys())[0]
        df = pd.read_excel(path)
        print(f" Loaded dataset: {df.shape} (rows, cols)")
        return df
    except Exception as e:
        raise RuntimeError("Colab upload failed.") from e

def load_excel_locally():
    """Upload Excel when running locally."""
    try:
        import tkinter as tk
        from tkinter import filedialog
        print(" Please select the Excel file...")
        root = tk.Tk()
        root.withdraw()
        file_path = filedialog.askopenfilename(
            title="Select Excel file",
            filetypes=[("Excel files", "*.xlsx *.xls")]
        )
        if not file_path:
            raise ValueError("No file selected")
        df = pd.read_excel(file_path)
        print(f" Loaded dataset: {df.shape} (rows, cols)")
        return df
    except Exception as e:
        print(f" File dialog failed: {e}")
        file_path = input(" Enter full path to your Excel file: ")
        df = pd.read_excel(file_path)
        print(f" Loaded dataset: {df.shape} (rows, cols)")
        return df

```

```

# =====
# 2) UNIVERSAL TARGET SELECTION
# =====
def detect_target_column(df: pd.DataFrame, target: str) -> str:
    """
    Universal target detector:
    1) If `target` matches an existing column exactly -> use it.
    2) If target looks like BF<number>, try common BF aliases (BF 1, BF_1, BF-1, etc.).
    3) Else try case-insensitive exact match.
    4) Else ask user to type exact column name.
    """
    if target is None or str(target).strip() == "":
        raise ValueError("target cannot be empty. Example: target='CI' or target='BF1'")

    target = str(target).strip()

    # (1) Exact match
    if target in df.columns:
        print(f" Found target column (exact): {target}")
        return target

    # (2) BF alias support if user typed BF-style
    m = re.match(r"^s*BF\s*(\d+)\s*$", target, flags=re.I)
    if m:
        k = m.group(1)
        aliases = [
            f"BF{k}", f"bf{k}",
            f"BF {k}", f"bf {k}",
            f"BF_{k}", f"bf_{k}",
            f"BF-{k}", f"bf-{k}",
        ]
        for a in aliases:
            if a in df.columns:
                print(f" Found target column (BF alias): {a}")
                return a

    # regex scan for bf variants
    pat = re.compile(rf"\bbf\s*[_-]?s*{k}\b", flags=re.I)
    for col in df.columns:
        if pat.search(str(col)):
            print(f" Found target column (BF regex): {col}")
            return col

```

```

# (3) Case-insensitive exact match
lower_map = {str(c).strip().lower(): c for c in df.columns}
if target.lower() in lower_map:
    real_col = lower_map[target.lower()]
    print(f" Found target column (case-insensitive): {real_col}")
    return real_col

# (4) Ask user
print(f" Target column '{target}' not found automatically.")
print(" Available columns:")
print(df.columns.tolist())
user_col = input(" Enter the exact target column name: ").strip()
if user_col in df.columns:
    print(f" Using specified target column: {user_col}")
    return user_col
raise ValueError(f"Column '{user_col}' not found in dataset")

def preprocess_target_binary(y_series: pd.Series, target_label: str):
    """
    Convert target responses into binary classification (expects 0/1 after
    ↪cleaning).
    """
    y_numeric = pd.to_numeric(y_series, errors="coerce")
    valid_mask = y_numeric.notna() & np.isfinite(y_numeric)
    y_clean = y_numeric[valid_mask]

    if len(y_clean) == 0:
        raise ValueError(f" No valid values for {target_label} after cleaning")

    print(f"\n TARGET ANALYSIS: {target_label}")
    print("-" * 40)
    print("Value distribution:")
    print(y_clean.value_counts().sort_index())

    unique_vals = sorted(y_clean.unique())
    print(f"Unique values: {unique_vals}")

    y_binary = y_clean.astype(int)

    class_balance = y_binary.value_counts(normalize=True).round(3)
    print(f" Final class balance: {class_balance.to_dict()}")

    return y_binary, valid_mask

# =====

```

```

# 3) FEATURE ENGINEERING
# =====
def build_feature_matrix(df_clean: pd.DataFrame, target_col: str):
    """Prepare features for ML (numeric-only)."""
    print("\n FEATURE ENGINEERING")
    print("-" * 40)

    X = df_clean.drop(columns=[target_col], errors="ignore")
    numeric_cols = X.select_dtypes(include=[np.number]).columns.tolist()
    print(f" Found {len(numeric_cols)} numeric features")

    if len(numeric_cols) == 0:
        raise ValueError(" No numeric features found for modeling")

    print(" Handling missing values with median imputation...")
    imputer = SimpleImputer(strategy="median")
    X_imputed = imputer.fit_transform(X[numeric_cols])
    X_imputed = pd.DataFrame(X_imputed, columns=numeric_cols, index=X.index)

    non_constant_cols = [c for c in X_imputed.columns if X_imputed[c].nunique()
↳ 1]
    dropped = len(numeric_cols) - len(non_constant_cols)
    if dropped > 0:
        print(f" Removed {dropped} constant columns")

    X_final = X_imputed[non_constant_cols]
    print(f" Final feature matrix: {X_final.shape} (rows, cols)")
    return X_final, non_constant_cols

# =====
# 4) CROSS-VALIDATION
# =====
def _compute_scale_pos_weight(y):
    # For XGBoost: scale_pos_weight = (#neg / #pos)
    pos = int((y == 1).sum())
    neg = int((y == 0).sum())
    if pos == 0:
        return 1.0
    return neg / pos

def perform_cross_validation(X, y, cv_folds=10):
    print(f"\n PERFORMING {cv_folds}-FOLD CROSS-VALIDATION")
    print("-" * 50)

    preprocessor = Pipeline([

```

```

        ("impute", SimpleImputer(strategy="median")),
        ("scale", StandardScaler())
    ])

    spw = _compute_scale_pos_weight(y)

    xgb = XGBClassifier(
        n_estimators=600,
        learning_rate=0.05,
        max_depth=4,
        min_child_weight=3,
        subsample=0.85,
        colsample_bytree=0.85,
        gamma=0.0,
        reg_lambda=1.0,
        reg_alpha=0.0,
        objective="binary:logistic",
        eval_metric="logloss",
        random_state=RANDOM_STATE,
        n_jobs=-1,
        scale_pos_weight=spw,
        tree_method="hist" # fast + stable in CPU
    )

    pipe = Pipeline([("pre", preprocessor), ("xgb", xgb)])

    scoring = {
        "accuracy": "accuracy",
        "precision": "precision",
        "recall": "recall",
        "f1": "f1",
        "roc_auc": "roc_auc",
        "average_precision": "average_precision"
    }

    cv_results = cross_validate(
        pipe, X, y,
        cv=StratifiedKFold(n_splits=cv_folds, shuffle=True,
        ↪random_state=RANDOM_STATE),
        scoring=scoring,
        return_train_score=True,
        n_jobs=-1
    )

    print(" CROSS-VALIDATION RESULTS (Mean ± Std):")
    print("-" * 40)

```

```

metrics_summary = {}
for metric in scoring.keys():
    test_scores = cv_results[f"test_{metric}"]
    train_scores = cv_results[f"train_{metric}"]

    test_mean = float(np.mean(test_scores))
    test_std = float(np.std(test_scores))
    train_mean = float(np.mean(train_scores))

    metrics_summary[metric] = {
        "test_mean": test_mean,
        "test_std": test_std,
        "train_mean": train_mean
    }

    print(f"{metric.upper():<15}: {test_mean:.4f} ± {test_std:.4f} (train:␣
↪{train_mean:.4f})")

    overfit_gap = metrics_summary["accuracy"]["train_mean"] -␣
↪metrics_summary["accuracy"]["test_mean"]
    print(f"\n Overfitting Analysis:")
    print(f"    Train-Test Accuracy Gap: {overfit_gap:.4f}")
    print("    Potential overfitting detected" if overfit_gap > 0.05 else "␣
↪ Model generalizes well")

    return metrics_summary, cv_results

# =====
# 5) TRAIN/TEST/VALIDATION SPLITS + TRAINING
# =====
def create_validation_set(X, y, validation_size=0.2):
    print(f"\n CREATING VALIDATION SET ({validation_size*100:.0f}% of data)")
    print("=" * 50)

    X_train_test, X_val, y_train_test, y_val = train_test_split(
        X, y, test_size=validation_size, stratify=y, random_state=RANDOM_STATE
    )

    X_train, X_test, y_train, y_test = train_test_split(
        X_train_test, y_train_test, test_size=0.25, stratify=y_train_test,␣
↪random_state=RANDOM_STATE
    )

    print(f" Training set: {X_train.shape[0]} samples")
    print(f" Test set: {X_test.shape[0]} samples")
    print(f" Validation set: {X_val.shape[0]} samples")

```

```

return X_train, X_test, X_val, y_train, y_test, y_val

def train_with_comprehensive_validation(X, y, use_validation_set=True):
    print("\n MODEL TRAINING: XGBoost")
    print("-" * 40)

    preprocessor = Pipeline([
        ("impute", SimpleImputer(strategy="median")),
        ("scale", StandardScaler())
    ])

    spw = _compute_scale_pos_weight(y)

    xgb = XGBClassifier(
        n_estimators=600,
        learning_rate=0.05,
        max_depth=4,
        min_child_weight=3,
        subsample=0.85,
        colsample_bytree=0.85,
        gamma=0.0,
        reg_lambda=1.0,
        reg_alpha=0.0,
        objective="binary:logistic",
        eval_metric="logloss",
        random_state=RANDOM_STATE,
        n_jobs=-1,
        scale_pos_weight=spw,
        tree_method="hist"
    )

    pipe = Pipeline([("pre", preprocessor), ("xgb", xgb)])

    if use_validation_set:
        X_train, X_test, X_val, y_train, y_test, y_val = \
        ↪ create_validation_set(X, y)

        # Early stopping uses a validation set, but sklearn.Pipeline
        ↪ complicates passing eval_set cleanly.
        # So we keep training stable via regularization + conservative params.
        print(" Training model with regularization...")
        pipe.fit(X_train, y_train)

    return pipe, X_train, X_test, X_val, y_train, y_test, y_val

```

```

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=RANDOM_STATE
)
print(f" Training set: {X_train.shape[0]} samples")
print(f" Test set: {X_test.shape[0]} samples")
print(" Training model with regularization...")
pipe.fit(X_train, y_train)
return pipe, X_train, X_test, None, y_train, y_test, None

# =====
# 6) EVALUATION
# =====
def evaluate_model_comprehensive(model, X_test, y_test, dataset_name="Test"):
    y_pred = model.predict(X_test)
    y_proba = model.predict_proba(X_test)[:, 1]

    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred, zero_division=0)
    rec = recall_score(y_test, y_pred, zero_division=0)
    f1 = f1_score(y_test, y_pred, zero_division=0)
    roc_auc = roc_auc_score(y_test, y_proba)
    pr_auc = average_precision_score(y_test, y_proba)

    print("\n" + "=" * 60)
    print(f" {dataset_name.upper()} SET COMPREHENSIVE EVALUATION")
    print("=" * 60)
    print(f" Accuracy: {acc:.4f}")
    print(f" Precision: {prec:.4f}")
    print(f" Recall: {rec:.4f}")
    print(f" F1-Score: {f1:.4f}")
    print(f" ROC-AUC: {roc_auc:.4f}")
    print(f" PR-AUC: {pr_auc:.4f}")

    print(f"\n {dataset_name} Classification Report:")
    print(classification_report(y_test, y_pred, digits=3))

    cm = confusion_matrix(y_test, y_pred)
    print(f" {dataset_name} Confusion Matrix:")
    print(" [[TN FP]\n [FN TP]]")
    print(cm)

    return y_pred, y_proba, {
        "accuracy": acc, "precision": prec, "recall": rec, "f1": f1,
        "roc_auc": roc_auc, "pr_auc": pr_auc
    }

```

```

# =====
# 7) VISUALIZATIONS
# =====
def create_validation_comparison_plot(cv_metrics, output_dir, prefix):
    metrics_names = list(cv_metrics.keys())
    test_means = [cv_metrics[m]["test_mean"] for m in metrics_names]
    test_stds = [cv_metrics[m]["test_std"] for m in metrics_names]

    plt.figure(figsize=(12, 6))
    x_pos = np.arange(len(metrics_names))
    bars = plt.bar(x_pos, test_means, yerr=test_stds, capsize=5, alpha=0.7)

    for i, b in enumerate(bars):
        b.set_color(f"C{i}")

    plt.xlabel("Performance Metrics")
    plt.ylabel("Score")
    plt.title(f"10-Fold Cross-Validation Performance ({prefix.upper()})")
    plt.xticks(x_pos, [m.upper() for m in metrics_names], rotation=0)
    plt.ylim(0, 1)
    plt.grid(True, alpha=0.3, axis="y")

    for i, (mean, std) in enumerate(zip(test_means, test_stds)):
        plt.text(i, mean + 0.02, f"{mean:.3f}±{std:.3f}", ha="center",
        ↪va="bottom", fontsize=10)

    plt.tight_layout()
    plt.savefig(f"{output_dir}/{prefix}_cross_validation_results.png", dpi=200,
    ↪bbox_inches="tight")
    plt.show()

def create_comprehensive_visualizations(y_test, y_proba, model, feature_names,
    ↪output_dir, prefix, cv_metrics=None):
    os.makedirs(output_dir, exist_ok=True)

    xgb_model = model.named_steps["xgb"]
    imp_df = pd.DataFrame({
        "feature": feature_names,
        "importance": xgb_model.feature_importances_
    }).sort_values("importance", ascending=False)

    imp_df.to_csv(f"{output_dir}/{prefix}_feature_importances.csv", index=False)

    # Feature importance plot
    plt.figure(figsize=(12, 8))

```

```

top_features = imp_df.head(15).iloc[:, :-1]
plt.barh(range(len(top_features)), top_features["importance"], alpha=0.7)
plt.yticks(range(len(top_features)), top_features["feature"])
plt.xlabel("Feature Importance Score")
plt.title(f"Top 15 Features for {prefix.upper()} Prediction (XGBoost)")
plt.grid(True, alpha=0.3, axis="x")
plt.tight_layout()
plt.savefig(f"{output_dir}/{prefix}_feature_importances.png", dpi=200,
↳bbox_inches="tight")
plt.show()

# ROC + PR curves
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))

fpr, tpr, _ = roc_curve(y_test, y_proba)
roc_auc = roc_auc_score(y_test, y_proba)
ax1.plot(fpr, tpr, label=f"AUC = {roc_auc:.3f}", linewidth=2, color="C0")
ax1.plot([0, 1], [0, 1], "k--", alpha=0.5)
ax1.set_xlabel("False Positive Rate")
ax1.set_ylabel("True Positive Rate")
ax1.set_title(f"ROC Curve ({prefix.upper()})")
ax1.legend()
ax1.grid(True, alpha=0.3)

precision_vals, recall_vals, _ = precision_recall_curve(y_test, y_proba)
pr_auc = average_precision_score(y_test, y_proba)
ax2.plot(recall_vals, precision_vals, label=f"AP = {pr_auc:.3f}",
↳linewidth=2, color="C1")
ax2.set_xlabel("Recall")
ax2.set_ylabel("Precision")
ax2.set_title(f"Precision-Recall Curve ({prefix.upper()})")
ax2.legend()
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig(f"{output_dir}/{prefix}_roc_pr_curves.png", dpi=200,
↳bbox_inches="tight")
plt.show()

# Confusion matrix heatmap
y_pred = (y_proba >= 0.5).astype(int)
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=["Pred 0", "Pred 1"],
            yticklabels=["True 0", "True 1"])

```

```

plt.title(f"Confusion Matrix ({prefix.upper()}) (XGBoost)")
plt.ylabel("True Label")
plt.xlabel("Predicted Label")
plt.tight_layout()
plt.savefig(f"{output_dir}/{prefix}_confusion_matrix.png", dpi=200,
↳ bbox_inches="tight")
plt.show()

if cv_metrics:
    create_validation_comparison_plot(cv_metrics, output_dir, prefix)

return imp_df

# =====
# 8) SAVE RESULTS
# =====
def save_comprehensive_results(model, metrics, imp_df, df_clean, target_col,
↳ feature_names, output_dir, prefix, cv_metrics=None):
    print(f"\n SAVING OUTPUTS to {output_dir}/ ...")
    os.makedirs(output_dir, exist_ok=True)

    with open(f"{output_dir}/{prefix}_comprehensive_metrics.txt", "w",
↳ encoding="utf-8") as f:
        f.write(f"COMPREHENSIVE XGBOOST RESULTS ({prefix.upper()})\n")
        f.write("=" * 60 + "\n")
        f.write(f"Dataset shape: {df_clean.shape}\n")
        f.write(f"Target column: {target_col}\n")
        f.write(f"Features used: {len(feature_names)}\n\n")

        f.write("TEST SET METRICS:\n")
        for k, v in metrics.items():
            f.write(f"{k}: {v:.4f}\n")
        f.write("\n")

        if cv_metrics:
            f.write("10-FOLD CROSS-VALIDATION (Mean ± Std) WITH MULTIPLE
↳ METRICS:\n")
            for metric, scores in cv_metrics.items():
                f.write(f"{metric}: {scores['test_mean']:.4f} ±
↳ {scores['test_std']:.4f}\n")
            f.write("\n")

        f.write("TOP 10 FEATURES:\n")
        for _, row in imp_df.head(10).iterrows():
            f.write(f"{row['feature']}: {row['importance']:.6f}\n")

```

```

# Full-dataset predictions
X_full = df_clean.drop(columns=[target_col], errors="ignore")
X_full = X_full.reindex(columns=feature_names)

preds = model.predict_proba(X_full)[: , 1]
predictions_df = df_clean.copy()
predictions_df[f"{prefix}_Pred_Prob"] = preds
predictions_df[f"{prefix}_Pred"] = (preds >= 0.5).astype(int)
predictions_df[f"{prefix}_Correct"] = (predictions_df[f"{prefix}_Pred"] ==
↳ predictions_df[target_col]).astype(int)

predictions_df.to_excel(f"{output_dir}/{prefix}_predictions.xlsx",
↳ index=False)
joblib.dump(model, f"{output_dir}/{prefix}_xgb_model.joblib")

print(" Saved metrics, plots, predictions, and model.")

# =====
# 9) MAIN PIPELINE (UNIVERSAL TARGET)
# =====
def run_comprehensive_analysis(
    target="CI",
    use_cross_validation=True,
    use_validation_set=True,
    df=None
):
    """
    Run the full pipeline for ANY target column you type.
    """
    try:
        prefix = re.sub(r"^[a-zA-Z0-9]+", "_", str(target).strip().lower()).
↳ strip("_")
        output_dir = f"{prefix}_comprehensive_outputs_xgb"

        print(" COMPREHENSIVE MICROFINANCE ANALYSIS (XGBoost)")
        print("=" * 60)
        print(f" Target: {target}")
        print(f" Output folder: {output_dir}/")
        print("=" * 60)

        if df is None:
            try:
                import google.colab # noqa: F401
                df = load_excel_from_colab()
            except Exception:
                df = load_excel_locally()

```

```

print(f" Initial dataset shape: {df.shape}")

target_col = detect_target_column(df, target=target)

y, valid_mask = preprocess_target_binary(df[target_col],
↪target_label=target_col)
df_clean = df.loc[valid_mask].copy()
print(f" Clean dataset: {df_clean.shape} (after removing invalid_
↪target rows)")

X_final, feature_names = build_feature_matrix(df_clean, target_col)

cv_metrics = None
if use_cross_validation:
    cv_metrics, _ = perform_cross_validation(X_final, y, cv_folds=10)

model, X_train, X_test, X_val, y_train, y_test, y_val =
↪train_with_comprehensive_validation(
    X_final, y, use_validation_set=use_validation_set
)

_, y_proba, test_metrics = evaluate_model_comprehensive(model, X_test,
↪y_test, "Test")

if X_val is not None and y_val is not None:
    evaluate_model_comprehensive(model, X_val, y_val, "Validation")

imp_df = create_comprehensive_visualizations(
    y_test=y_test,
    y_proba=y_proba,
    model=model,
    feature_names=feature_names,
    output_dir=output_dir,
    prefix=prefix,
    cv_metrics=cv_metrics
)

save_comprehensive_results(
    model=model,
    metrics=test_metrics,
    imp_df=imp_df,
    df_clean=df_clean,
    target_col=target_col,
    feature_names=feature_names,
    output_dir=output_dir,
    prefix=prefix,

```

```

        cv_metrics=cv_metrics
    )

    print("\n DONE.")
    return model, test_metrics, imp_df, cv_metrics

except Exception as e:
    print(f"\n Analysis failed: {type(e).__name__}: {e}")
    import traceback
    traceback.print_exc(limit=2)
    return None, None, None, None

# ===== USAGE EXAMPLES =====
if __name__ == "__main__":
    run_comprehensive_analysis(target="BF3", use_cross_validation=True,
    ↪use_validation_set=True)

    # Other examples:
    # run_comprehensive_analysis(target="CI", use_cross_validation=True,
    ↪use_validation_set=True)
    # for t in ["CI", "BF1", "BF3", "BF4"]:
    #     run_comprehensive_analysis(target=t, use_cross_validation=True,
    ↪use_validation_set=True)

```

COMPREHENSIVE MICROFINANCE ANALYSIS (XGBoost)

```

=====
Target: BF3
Output folder: bf3_comprehensive_outputs_xgb/
=====

Please select the Excel file with your encoded dataset...

<IPython.core.display.HTML object>

Saving rf_dataset_encoded_dataset.xlsx to rf_dataset_encoded_dataset (1).xlsx
Loaded dataset: (1002, 85) (rows, cols)
Initial dataset shape: (1002, 85)
Target column 'BF3' not found automatically.
Available columns:
['D', 'AY', 'AZ', 'BA', 'BB', 'BC', 'BD', 'BE_01', 'BE_02', 'BE_03', 'BE_04',
'BE_05', 'BE_06', 'BE_07', 'BE_08', 'BE_09', 'BE_10', 'BF', 'BG_01', 'BG_02',
'BG_03', 'BG_04', 'BG_05', 'BG_06', 'BH', 'BI', 'BN', 'BO', 'BP', 'BQ', 'BR',
'BS', 'BT', 'BU', 'BV', 'BW_01', 'BW_02', 'BW_03', 'BW_04', 'BW_05', 'BX', 'BY',
'CB', 'CD', 'CE', 'CF', 'CG', 'CH', 'CI', 'CK', 'CL', 'CM_01', 'CM_02', 'CM_03',
'CM_04', 'CM_05', 'CM_06', 'CN', 'CO_01', 'CO_02', 'CO_03', 'CO_04', 'CO_05',
'CO_06', 'CO_07', 'CO_08', 'CO_09', 'CP', 'CR_01', 'CR_02', 'CR_03', 'CR_04',
'CS', 'CT_01', 'CT_02', 'CT_03', 'CT_04', 'CT_05', 'CT_06', 'CU', 'Occupation',
'Financial Behavior', 'Family', 'Lifestyle', 'Digital Literacy']

```

Enter the exact target column name: BG_03

Using specified target column: BG_03

TARGET ANALYSIS: BG_03

Value distribution:

BG_03

0 590

1 412

Name: count, dtype: int64

Unique values: [np.int64(0), np.int64(1)]

Final class balance: {0: 0.589, 1: 0.411}

Clean dataset: (1002, 85) (after removing invalid target rows)

FEATURE ENGINEERING

Found 84 numeric features

Handling missing values with median imputation...

Final feature matrix: (1002, 84) (rows, cols)

PERFORMING 10-FOLD CROSS-VALIDATION

=====

CROSS-VALIDATION RESULTS (Mean ± Std):

ACCURACY	: 0.8753 ± 0.0312 (train: 0.9978)
PRECISION	: 0.8593 ± 0.0431 (train: 0.9968)
RECALL	: 0.8370 ± 0.0712 (train: 0.9978)
F1	: 0.8457 ± 0.0419 (train: 0.9973)
ROC_AUC	: 0.9363 ± 0.0293 (train: 0.9998)
AVERAGE_PRECISION	: 0.9218 ± 0.0311 (train: 0.9997)

Overfitting Analysis:

Train-Test Accuracy Gap: 0.1225

Potential overfitting detected

MODEL TRAINING: XGBoost

CREATING VALIDATION SET (20% of data)

=====

Training set: 600 samples

Test set: 201 samples

Validation set: 201 samples

Training model with regularization...

=====

TEST SET COMPREHENSIVE EVALUATION

=====

Accuracy: 0.8259
Precision: 0.7927
Recall: 0.7831
F1-Score: 0.7879
ROC-AUC: 0.8909
PR-AUC: 0.8713

Test Classification Report:

	precision	recall	f1-score	support
0	0.849	0.856	0.852	118
1	0.793	0.783	0.788	83
accuracy			0.826	201
macro avg	0.821	0.820	0.820	201
weighted avg	0.826	0.826	0.826	201

Test Confusion Matrix:

```
[[TN FP]
 [FN TP]]
[[101 17]
 [ 18 65]]
```

=====

VALIDATION SET COMPREHENSIVE EVALUATION

=====

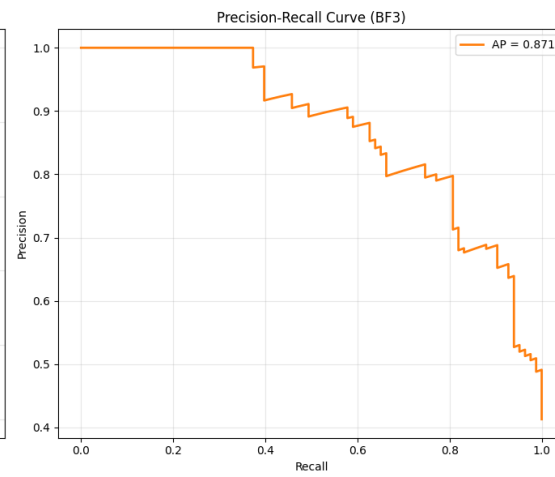
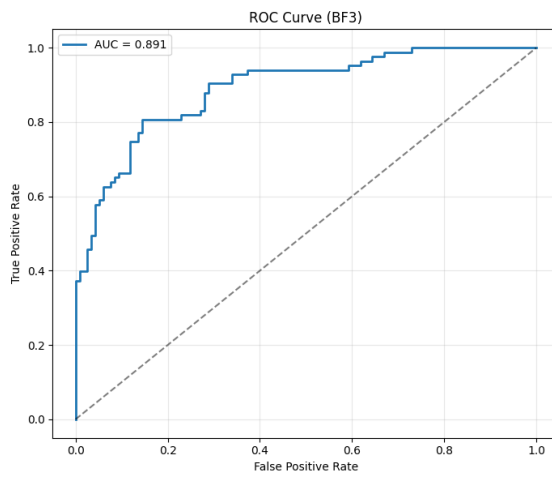
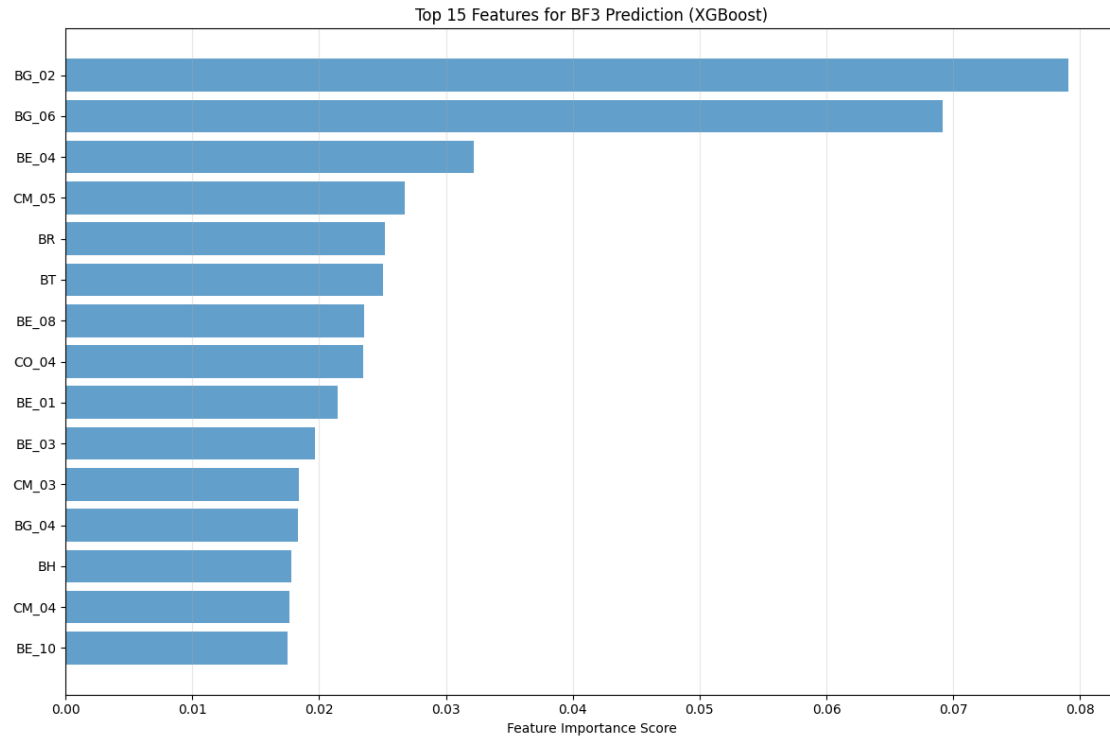
Accuracy: 0.8060
Precision: 0.7750
Recall: 0.7470
F1-Score: 0.7607
ROC-AUC: 0.8775
PR-AUC: 0.8495

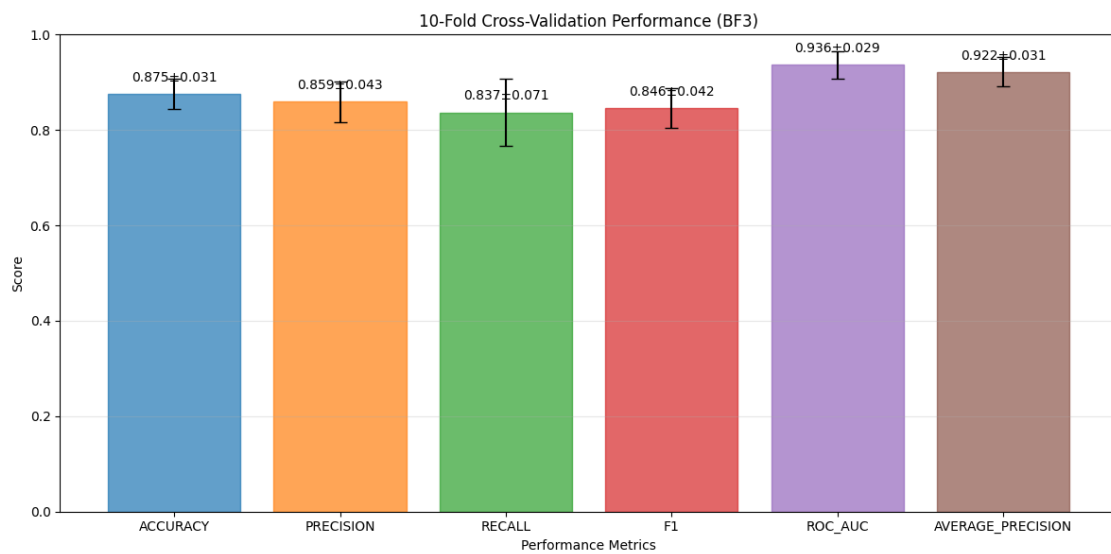
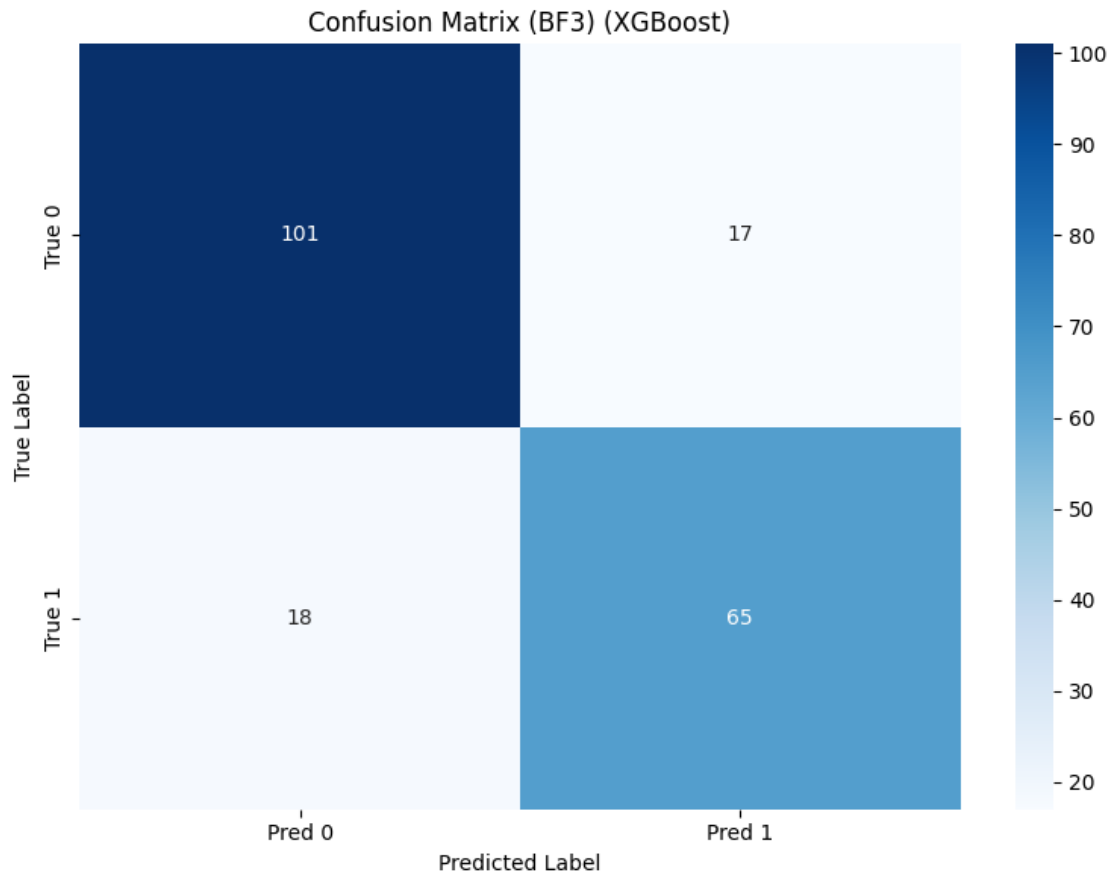
Validation Classification Report:

	precision	recall	f1-score	support
0	0.826	0.847	0.837	118
1	0.775	0.747	0.761	83
accuracy			0.806	201
macro avg	0.801	0.797	0.799	201
weighted avg	0.805	0.806	0.805	201

Validation Confusion Matrix:

```
[[TN FP]
 [FN TP]]
[[100 18]
 [ 21 62]]
```





SAVING OUTPUTS to bf3_comprehensive_outputs_xgb/ ...
Saved metrics, plots, predictions, and model.

DONE.

```
[3]: """
Reusable XGBoost Pipeline for Microfinance Impact Analysis
-----

Matches your Random Forest pipeline structure:
- Universal target detection (CI, BF1, BF_1, BF 1, bf-1, etc.)
- Median imputation + StandardScaler in a Pipeline
- 10-fold Stratified CV with multiple metrics
- Train/Test/Validation split option
- Comprehensive evaluation + plots + saving outputs (metrics, preds, model,
  ↳feature importances)

NOTE:
- This is for binary classification (0/1 target).
"""

# =====
# 0) INSTALLS
# =====
# If you're in Colab, uncomment:
# !pip -q install xgboost openpyxl joblib

# =====
# 0) IMPORTS
# =====
import os, re, warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, StratifiedKFold,
  ↳cross_validate
from sklearn.metrics import (
    classification_report, confusion_matrix, roc_auc_score, roc_curve,
    precision_recall_curve, average_precision_score,
    accuracy_score, precision_score, recall_score, f1_score
)
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
```

```

import joblib

from xgboost import XGBClassifier

RANDOM_STATE = 42

# =====
# 1) DATA LOADING
# =====
def load_excel_from_colab():
    """Upload Excel in Google Colab."""
    try:
        from google.colab import files
        print(" Please select the Excel file with your encoded dataset...")
        uploaded = files.upload()
        path = list(uploaded.keys())[0]
        df = pd.read_excel(path)
        print(f" Loaded dataset: {df.shape} (rows, cols)")
        return df
    except Exception as e:
        raise RuntimeError("Colab upload failed.") from e

def load_excel_locally():
    """Upload Excel when running locally."""
    try:
        import tkinter as tk
        from tkinter import filedialog
        print(" Please select the Excel file...")
        root = tk.Tk()
        root.withdraw()
        file_path = filedialog.askopenfilename(
            title="Select Excel file",
            filetypes=[("Excel files", "*.xlsx *.xls")]
        )
        if not file_path:
            raise ValueError("No file selected")
        df = pd.read_excel(file_path)
        print(f" Loaded dataset: {df.shape} (rows, cols)")
        return df
    except Exception as e:
        print(f" File dialog failed: {e}")
        file_path = input(" Enter full path to your Excel file: ")
        df = pd.read_excel(file_path)
        print(f" Loaded dataset: {df.shape} (rows, cols)")
        return df

```

```

# =====
# 2) UNIVERSAL TARGET SELECTION
# =====
def detect_target_column(df: pd.DataFrame, target: str) -> str:
    """
    Universal target detector:
    1) If `target` matches an existing column exactly -> use it.
    2) If target looks like BF<number>, try common BF aliases (BF 1, BF_1, BF-1, etc.).
    3) Else try case-insensitive exact match.
    4) Else ask user to type exact column name.
    """
    if target is None or str(target).strip() == "":
        raise ValueError("target cannot be empty. Example: target='CI' or target='BF1'")

    target = str(target).strip()

    # (1) Exact match
    if target in df.columns:
        print(f" Found target column (exact): {target}")
        return target

    # (2) BF alias support if user typed BF-style
    m = re.match(r"^s*BF\s*(\d+)\s*$", target, flags=re.I)
    if m:
        k = m.group(1)
        aliases = [
            f"BF{k}", f"bf{k}",
            f"BF {k}", f"bf {k}",
            f"BF_{k}", f"bf_{k}",
            f"BF-{k}", f"bf-{k}",
        ]
        for a in aliases:
            if a in df.columns:
                print(f" Found target column (BF alias): {a}")
                return a

    # regex scan for bf variants
    pat = re.compile(rf"\bbf\s*[_-]?s*{k}\b", flags=re.I)
    for col in df.columns:
        if pat.search(str(col)):
            print(f" Found target column (BF regex): {col}")
            return col

```

```

# (3) Case-insensitive exact match
lower_map = {str(c).strip().lower(): c for c in df.columns}
if target.lower() in lower_map:
    real_col = lower_map[target.lower()]
    print(f" Found target column (case-insensitive): {real_col}")
    return real_col

# (4) Ask user
print(f" Target column '{target}' not found automatically.")
print(" Available columns:")
print(df.columns.tolist())
user_col = input(" Enter the exact target column name: ").strip()
if user_col in df.columns:
    print(f" Using specified target column: {user_col}")
    return user_col
raise ValueError(f"Column '{user_col}' not found in dataset")

def preprocess_target_binary(y_series: pd.Series, target_label: str):
    """
    Convert target responses into binary classification (expects 0/1 after
    ↪cleaning).
    """
    y_numeric = pd.to_numeric(y_series, errors="coerce")
    valid_mask = y_numeric.notna() & np.isfinite(y_numeric)
    y_clean = y_numeric[valid_mask]

    if len(y_clean) == 0:
        raise ValueError(f" No valid values for {target_label} after cleaning")

    print(f"\n TARGET ANALYSIS: {target_label}")
    print("-" * 40)
    print("Value distribution:")
    print(y_clean.value_counts().sort_index())

    unique_vals = sorted(y_clean.unique())
    print(f"Unique values: {unique_vals}")

    y_binary = y_clean.astype(int)

    class_balance = y_binary.value_counts(normalize=True).round(3)
    print(f" Final class balance: {class_balance.to_dict()}")

    return y_binary, valid_mask

# =====

```

```

# 3) FEATURE ENGINEERING
# =====
def build_feature_matrix(df_clean: pd.DataFrame, target_col: str):
    """Prepare features for ML (numeric-only)."""
    print("\n FEATURE ENGINEERING")
    print("-" * 40)

    X = df_clean.drop(columns=[target_col], errors="ignore")
    numeric_cols = X.select_dtypes(include=[np.number]).columns.tolist()
    print(f" Found {len(numeric_cols)} numeric features")

    if len(numeric_cols) == 0:
        raise ValueError(" No numeric features found for modeling")

    print(" Handling missing values with median imputation...")
    imputer = SimpleImputer(strategy="median")
    X_imputed = imputer.fit_transform(X[numeric_cols])
    X_imputed = pd.DataFrame(X_imputed, columns=numeric_cols, index=X.index)

    non_constant_cols = [c for c in X_imputed.columns if X_imputed[c].nunique()
↳ 1]
    dropped = len(numeric_cols) - len(non_constant_cols)
    if dropped > 0:
        print(f" Removed {dropped} constant columns")

    X_final = X_imputed[non_constant_cols]
    print(f" Final feature matrix: {X_final.shape} (rows, cols)")
    return X_final, non_constant_cols

# =====
# 4) CROSS-VALIDATION
# =====
def _compute_scale_pos_weight(y):
    # For XGBoost: scale_pos_weight = (#neg / #pos)
    pos = int((y == 1).sum())
    neg = int((y == 0).sum())
    if pos == 0:
        return 1.0
    return neg / pos

def perform_cross_validation(X, y, cv_folds=10):
    print(f"\n PERFORMING {cv_folds}-FOLD CROSS-VALIDATION")
    print("=" * 50)

    preprocessor = Pipeline([

```

```

        ("impute", SimpleImputer(strategy="median")),
        ("scale", StandardScaler())
    ])

    spw = _compute_scale_pos_weight(y)

    xgb = XGBClassifier(
        n_estimators=600,
        learning_rate=0.05,
        max_depth=4,
        min_child_weight=3,
        subsample=0.85,
        colsample_bytree=0.85,
        gamma=0.0,
        reg_lambda=1.0,
        reg_alpha=0.0,
        objective="binary:logistic",
        eval_metric="logloss",
        random_state=RANDOM_STATE,
        n_jobs=-1,
        scale_pos_weight=spw,
        tree_method="hist" # fast + stable in CPU
    )

    pipe = Pipeline([("pre", preprocessor), ("xgb", xgb)])

    scoring = {
        "accuracy": "accuracy",
        "precision": "precision",
        "recall": "recall",
        "f1": "f1",
        "roc_auc": "roc_auc",
        "average_precision": "average_precision"
    }

    cv_results = cross_validate(
        pipe, X, y,
        cv=StratifiedKFold(n_splits=cv_folds, shuffle=True,
        ↪random_state=RANDOM_STATE),
        scoring=scoring,
        return_train_score=True,
        n_jobs=-1
    )

    print(" CROSS-VALIDATION RESULTS (Mean ± Std):")
    print("-" * 40)

```

```

metrics_summary = {}
for metric in scoring.keys():
    test_scores = cv_results[f"test_{metric}"]
    train_scores = cv_results[f"train_{metric}"]

    test_mean = float(np.mean(test_scores))
    test_std = float(np.std(test_scores))
    train_mean = float(np.mean(train_scores))

    metrics_summary[metric] = {
        "test_mean": test_mean,
        "test_std": test_std,
        "train_mean": train_mean
    }

    print(f"{metric.upper():<15}: {test_mean:.4f} ± {test_std:.4f} (train:␣
↪{train_mean:.4f})")

    overfit_gap = metrics_summary["accuracy"]["train_mean"] -␣
↪metrics_summary["accuracy"]["test_mean"]
    print(f"\n Overfitting Analysis:")
    print(f"    Train-Test Accuracy Gap: {overfit_gap:.4f}")
    print("    Potential overfitting detected" if overfit_gap > 0.05 else "␣
↪ Model generalizes well")

    return metrics_summary, cv_results

# =====
# 5) TRAIN/TEST/VALIDATION SPLITS + TRAINING
# =====
def create_validation_set(X, y, validation_size=0.2):
    print(f"\n CREATING VALIDATION SET ({validation_size*100:.0f}% of data)")
    print("=" * 50)

    X_train_test, X_val, y_train_test, y_val = train_test_split(
        X, y, test_size=validation_size, stratify=y, random_state=RANDOM_STATE
    )

    X_train, X_test, y_train, y_test = train_test_split(
        X_train_test, y_train_test, test_size=0.25, stratify=y_train_test,␣
↪random_state=RANDOM_STATE
    )

    print(f" Training set: {X_train.shape[0]} samples")
    print(f" Test set: {X_test.shape[0]} samples")
    print(f" Validation set: {X_val.shape[0]} samples")

```

```

    return X_train, X_test, X_val, y_train, y_test, y_val

def train_with_comprehensive_validation(X, y, use_validation_set=True):
    print("\n MODEL TRAINING: XGBoost")
    print("-" * 40)

    preprocessor = Pipeline([
        ("impute", SimpleImputer(strategy="median")),
        ("scale", StandardScaler())
    ])

    spw = _compute_scale_pos_weight(y)

    xgb = XGBClassifier(
        n_estimators=600,
        learning_rate=0.05,
        max_depth=4,
        min_child_weight=3,
        subsample=0.85,
        colsample_bytree=0.85,
        gamma=0.0,
        reg_lambda=1.0,
        reg_alpha=0.0,
        objective="binary:logistic",
        eval_metric="logloss",
        random_state=RANDOM_STATE,
        n_jobs=-1,
        scale_pos_weight=spw,
        tree_method="hist"
    )

    pipe = Pipeline([("pre", preprocessor), ("xgb", xgb)])

    if use_validation_set:
        X_train, X_test, X_val, y_train, y_test, y_val = \
        ↪ create_validation_set(X, y)

        # Early stopping uses a validation set, but sklearn.Pipeline
        ↪ complicates passing eval_set cleanly.
        # So we keep training stable via regularization + conservative params.
        print(" Training model with regularization...")
        pipe.fit(X_train, y_train)

    return pipe, X_train, X_test, X_val, y_train, y_test, y_val

```

```

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=RANDOM_STATE
)
print(f" Training set: {X_train.shape[0]} samples")
print(f" Test set: {X_test.shape[0]} samples")
print(" Training model with regularization...")
pipe.fit(X_train, y_train)
return pipe, X_train, X_test, None, y_train, y_test, None

# =====
# 6) EVALUATION
# =====
def evaluate_model_comprehensive(model, X_test, y_test, dataset_name="Test"):
    y_pred = model.predict(X_test)
    y_proba = model.predict_proba(X_test)[:, 1]

    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred, zero_division=0)
    rec = recall_score(y_test, y_pred, zero_division=0)
    f1 = f1_score(y_test, y_pred, zero_division=0)
    roc_auc = roc_auc_score(y_test, y_proba)
    pr_auc = average_precision_score(y_test, y_proba)

    print("\n" + "=" * 60)
    print(f" {dataset_name.upper()} SET COMPREHENSIVE EVALUATION")
    print("=" * 60)
    print(f" Accuracy: {acc:.4f}")
    print(f" Precision: {prec:.4f}")
    print(f" Recall: {rec:.4f}")
    print(f" F1-Score: {f1:.4f}")
    print(f" ROC-AUC: {roc_auc:.4f}")
    print(f" PR-AUC: {pr_auc:.4f}")

    print(f"\n {dataset_name} Classification Report:")
    print(classification_report(y_test, y_pred, digits=3))

    cm = confusion_matrix(y_test, y_pred)
    print(f" {dataset_name} Confusion Matrix:")
    print(" [[TN FP]\n [FN TP]]")
    print(cm)

    return y_pred, y_proba, {
        "accuracy": acc, "precision": prec, "recall": rec, "f1": f1,
        "roc_auc": roc_auc, "pr_auc": pr_auc
    }

```

```

# =====
# 7) VISUALIZATIONS
# =====
def create_validation_comparison_plot(cv_metrics, output_dir, prefix):
    metrics_names = list(cv_metrics.keys())
    test_means = [cv_metrics[m]["test_mean"] for m in metrics_names]
    test_stds = [cv_metrics[m]["test_std"] for m in metrics_names]

    plt.figure(figsize=(12, 6))
    x_pos = np.arange(len(metrics_names))
    bars = plt.bar(x_pos, test_means, yerr=test_stds, capsize=5, alpha=0.7)

    for i, b in enumerate(bars):
        b.set_color(f"C{i}")

    plt.xlabel("Performance Metrics")
    plt.ylabel("Score")
    plt.title(f"10-Fold Cross-Validation Performance ({prefix.upper()})")
    plt.xticks(x_pos, [m.upper() for m in metrics_names], rotation=0)
    plt.ylim(0, 1)
    plt.grid(True, alpha=0.3, axis="y")

    for i, (mean, std) in enumerate(zip(test_means, test_stds)):
        plt.text(i, mean + 0.02, f"{mean:.3f}±{std:.3f}", ha="center",
        ↪va="bottom", fontsize=10)

    plt.tight_layout()
    plt.savefig(f"{output_dir}/{prefix}_cross_validation_results.png", dpi=200,
    ↪bbox_inches="tight")
    plt.show()

def create_comprehensive_visualizations(y_test, y_proba, model, feature_names,
    ↪output_dir, prefix, cv_metrics=None):
    os.makedirs(output_dir, exist_ok=True)

    xgb_model = model.named_steps["xgb"]
    imp_df = pd.DataFrame({
        "feature": feature_names,
        "importance": xgb_model.feature_importances_
    }).sort_values("importance", ascending=False)

    imp_df.to_csv(f"{output_dir}/{prefix}_feature_importances.csv", index=False)

    # Feature importance plot
    plt.figure(figsize=(12, 8))

```

```

top_features = imp_df.head(15).iloc[:, :-1]
plt.barh(range(len(top_features)), top_features["importance"], alpha=0.7)
plt.yticks(range(len(top_features)), top_features["feature"])
plt.xlabel("Feature Importance Score")
plt.title(f"Top 15 Features for {prefix.upper()} Prediction (XGBoost)")
plt.grid(True, alpha=0.3, axis="x")
plt.tight_layout()
plt.savefig(f"{output_dir}/{prefix}_feature_importances.png", dpi=200,
↳bbox_inches="tight")
plt.show()

# ROC + PR curves
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))

fpr, tpr, _ = roc_curve(y_test, y_proba)
roc_auc = roc_auc_score(y_test, y_proba)
ax1.plot(fpr, tpr, label=f"AUC = {roc_auc:.3f}", linewidth=2, color="C0")
ax1.plot([0, 1], [0, 1], "k--", alpha=0.5)
ax1.set_xlabel("False Positive Rate")
ax1.set_ylabel("True Positive Rate")
ax1.set_title(f"ROC Curve ({prefix.upper()}")
ax1.legend()
ax1.grid(True, alpha=0.3)

precision_vals, recall_vals, _ = precision_recall_curve(y_test, y_proba)
pr_auc = average_precision_score(y_test, y_proba)
ax2.plot(recall_vals, precision_vals, label=f"AP = {pr_auc:.3f}",
↳linewidth=2, color="C1")
ax2.set_xlabel("Recall")
ax2.set_ylabel("Precision")
ax2.set_title(f"Precision-Recall Curve ({prefix.upper()}")
ax2.legend()
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig(f"{output_dir}/{prefix}_roc_pr_curves.png", dpi=200,
↳bbox_inches="tight")
plt.show()

# Confusion matrix heatmap
y_pred = (y_proba >= 0.5).astype(int)
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=["Pred 0", "Pred 1"],
            yticklabels=["True 0", "True 1"])

```

```

plt.title(f"Confusion Matrix ({prefix.upper()}) (XGBoost)")
plt.ylabel("True Label")
plt.xlabel("Predicted Label")
plt.tight_layout()
plt.savefig(f"{output_dir}/{prefix}_confusion_matrix.png", dpi=200,
↳ bbox_inches="tight")
plt.show()

if cv_metrics:
    create_validation_comparison_plot(cv_metrics, output_dir, prefix)

return imp_df

# =====
# 8) SAVE RESULTS
# =====
def save_comprehensive_results(model, metrics, imp_df, df_clean, target_col,
↳ feature_names, output_dir, prefix, cv_metrics=None):
    print(f"\n SAVING OUTPUTS to {output_dir}/ ...")
    os.makedirs(output_dir, exist_ok=True)

    with open(f"{output_dir}/{prefix}_comprehensive_metrics.txt", "w",
↳ encoding="utf-8") as f:
        f.write(f"COMPREHENSIVE XGBOOST RESULTS ({prefix.upper()})\n")
        f.write("=" * 60 + "\n")
        f.write(f"Dataset shape: {df_clean.shape}\n")
        f.write(f"Target column: {target_col}\n")
        f.write(f"Features used: {len(feature_names)}\n\n")

        f.write("TEST SET METRICS:\n")
        for k, v in metrics.items():
            f.write(f"{k}: {v:.4f}\n")
        f.write("\n")

        if cv_metrics:
            f.write("10-FOLD CROSS-VALIDATION (Mean ± Std) WITH MULTIPLE
↳ METRICS:\n")
            for metric, scores in cv_metrics.items():
                f.write(f"{metric}: {scores['test_mean']:.4f} ±
↳ {scores['test_std']:.4f}\n")
            f.write("\n")

        f.write("TOP 10 FEATURES:\n")
        for _, row in imp_df.head(10).iterrows():
            f.write(f"{row['feature']}: {row['importance']:.6f}\n")

```

```

# Full-dataset predictions
X_full = df_clean.drop(columns=[target_col], errors="ignore")
X_full = X_full.reindex(columns=feature_names)

preds = model.predict_proba(X_full)[: , 1]
predictions_df = df_clean.copy()
predictions_df[f"{prefix}_Pred_Prob"] = preds
predictions_df[f"{prefix}_Pred"] = (preds >= 0.5).astype(int)
predictions_df[f"{prefix}_Correct"] = (predictions_df[f"{prefix}_Pred"] ==
↳ predictions_df[target_col]).astype(int)

predictions_df.to_excel(f"{output_dir}/{prefix}_predictions.xlsx",
↳ index=False)
joblib.dump(model, f"{output_dir}/{prefix}_xgb_model.joblib")

print(" Saved metrics, plots, predictions, and model.")

# =====
# 9) MAIN PIPELINE (UNIVERSAL TARGET)
# =====
def run_comprehensive_analysis(
    target="CI",
    use_cross_validation=True,
    use_validation_set=True,
    df=None
):
    """
    Run the full pipeline for ANY target column you type.
    """
    try:
        prefix = re.sub(r"^[a-zA-Z0-9]+", "_", str(target).strip().lower()).
↳ strip("_")
        output_dir = f"{prefix}_comprehensive_outputs_xgb"

        print(" COMPREHENSIVE MICROFINANCE ANALYSIS (XGBoost)")
        print("=" * 60)
        print(f" Target: {target}")
        print(f" Output folder: {output_dir}/")
        print("=" * 60)

        if df is None:
            try:
                import google.colab # noqa: F401
                df = load_excel_from_colab()
            except Exception:
                df = load_excel_locally()

```

```

print(f" Initial dataset shape: {df.shape}")

target_col = detect_target_column(df, target=target)

y, valid_mask = preprocess_target_binary(df[target_col],
↪target_label=target_col)
df_clean = df.loc[valid_mask].copy()
print(f" Clean dataset: {df_clean.shape} (after removing invalid_
↪target rows)")

X_final, feature_names = build_feature_matrix(df_clean, target_col)

cv_metrics = None
if use_cross_validation:
    cv_metrics, _ = perform_cross_validation(X_final, y, cv_folds=10)

model, X_train, X_test, X_val, y_train, y_test, y_val =
↪train_with_comprehensive_validation(
    X_final, y, use_validation_set=use_validation_set
)

_, y_proba, test_metrics = evaluate_model_comprehensive(model, X_test,
↪y_test, "Test")

if X_val is not None and y_val is not None:
    evaluate_model_comprehensive(model, X_val, y_val, "Validation")

imp_df = create_comprehensive_visualizations(
    y_test=y_test,
    y_proba=y_proba,
    model=model,
    feature_names=feature_names,
    output_dir=output_dir,
    prefix=prefix,
    cv_metrics=cv_metrics
)

save_comprehensive_results(
    model=model,
    metrics=test_metrics,
    imp_df=imp_df,
    df_clean=df_clean,
    target_col=target_col,
    feature_names=feature_names,
    output_dir=output_dir,
    prefix=prefix,

```

```

        cv_metrics=cv_metrics
    )

    print("\n DONE.")
    return model, test_metrics, imp_df, cv_metrics

except Exception as e:
    print(f"\n Analysis failed: {type(e).__name__}: {e}")
    import traceback
    traceback.print_exc(limit=2)
    return None, None, None, None

# ===== USAGE EXAMPLES =====
if __name__ == "__main__":
    run_comprehensive_analysis(target="BF4", use_cross_validation=True,
    ↪use_validation_set=True)

    # Other examples:
    # run_comprehensive_analysis(target="CI", use_cross_validation=True,
    ↪use_validation_set=True)
    # for t in ["CI", "BF1", "BF3", "BF4"]:
    #     run_comprehensive_analysis(target=t, use_cross_validation=True,
    ↪use_validation_set=True)

```

COMPREHENSIVE MICROFINANCE ANALYSIS (XGBoost)

```

=====
Target: BF4
Output folder: bf4_comprehensive_outputs_xgb/
=====

Please select the Excel file with your encoded dataset...

<IPython.core.display.HTML object>

Saving rf_dataset_encoded_dataset.xlsx to rf_dataset_encoded_dataset (2).xlsx
Loaded dataset: (1002, 85) (rows, cols)
Initial dataset shape: (1002, 85)
Target column 'BF4' not found automatically.
Available columns:
['D', 'AY', 'AZ', 'BA', 'BB', 'BC', 'BD', 'BE_01', 'BE_02', 'BE_03', 'BE_04',
'BE_05', 'BE_06', 'BE_07', 'BE_08', 'BE_09', 'BE_10', 'BF', 'BG_01', 'BG_02',
'BG_03', 'BG_04', 'BG_05', 'BG_06', 'BH', 'BI', 'BN', 'BO', 'BP', 'BQ', 'BR',
'BS', 'BT', 'BU', 'BV', 'BW_01', 'BW_02', 'BW_03', 'BW_04', 'BW_05', 'BX', 'BY',
'CB', 'CD', 'CE', 'CF', 'CG', 'CH', 'CI', 'CK', 'CL', 'CM_01', 'CM_02', 'CM_03',
'CM_04', 'CM_05', 'CM_06', 'CN', 'CO_01', 'CO_02', 'CO_03', 'CO_04', 'CO_05',
'CO_06', 'CO_07', 'CO_08', 'CO_09', 'CP', 'CR_01', 'CR_02', 'CR_03', 'CR_04',
'CS', 'CT_01', 'CT_02', 'CT_03', 'CT_04', 'CT_05', 'CT_06', 'CU', 'Occupation',
'Financial Behavior', 'Family', 'Lifestyle', 'Digital Literacy']

```

Enter the exact target column name: BG_04
Using specified target column: BG_04

TARGET ANALYSIS: BG_04

Value distribution:

BG_04

0 605

1 397

Name: count, dtype: int64

Unique values: [np.int64(0), np.int64(1)]

Final class balance: {0: 0.604, 1: 0.396}

Clean dataset: (1002, 85) (after removing invalid target rows)

FEATURE ENGINEERING

Found 84 numeric features

Handling missing values with median imputation...

Final feature matrix: (1002, 84) (rows, cols)

PERFORMING 10-FOLD CROSS-VALIDATION

=====

CROSS-VALIDATION RESULTS (Mean ± Std):

ACCURACY	: 0.8952 ± 0.0250 (train: 0.9984)
PRECISION	: 0.8644 ± 0.0394 (train: 0.9961)
RECALL	: 0.8739 ± 0.0359 (train: 1.0000)
F1	: 0.8686 ± 0.0305 (train: 0.9980)
ROC_AUC	: 0.9536 ± 0.0252 (train: 1.0000)
AVERAGE_PRECISION	: 0.9275 ± 0.0433 (train: 1.0000)

Overfitting Analysis:

Train-Test Accuracy Gap: 0.1032

Potential overfitting detected

MODEL TRAINING: XGBoost

CREATING VALIDATION SET (20% of data)

=====

Training set: 600 samples

Test set: 201 samples

Validation set: 201 samples

Training model with regularization...

=====

TEST SET COMPREHENSIVE EVALUATION

=====

Accuracy: 0.8408
Precision: 0.7791
Recall: 0.8375
F1-Score: 0.8072
ROC-AUC: 0.9247
PR-AUC: 0.9006

Test Classification Report:

	precision	recall	f1-score	support
0	0.887	0.843	0.864	121
1	0.779	0.838	0.807	80
accuracy			0.841	201
macro avg	0.833	0.840	0.836	201
weighted avg	0.844	0.841	0.842	201

Test Confusion Matrix:

```
[[TN FP]
 [FN TP]]
[[102 19]
 [ 13 67]]
```

=====

VALIDATION SET COMPREHENSIVE EVALUATION

=====

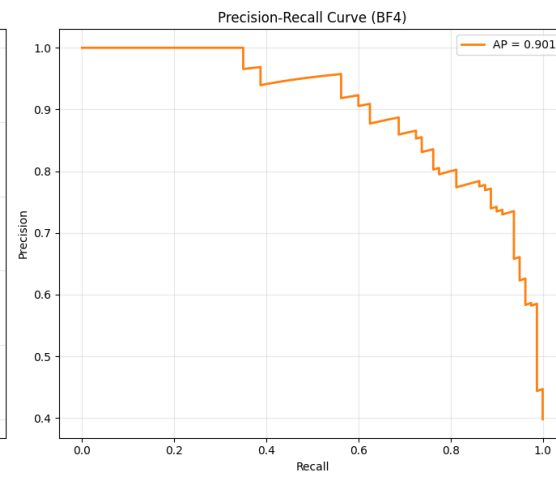
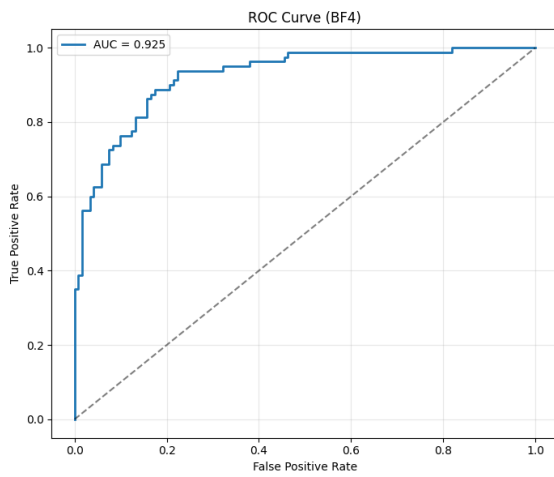
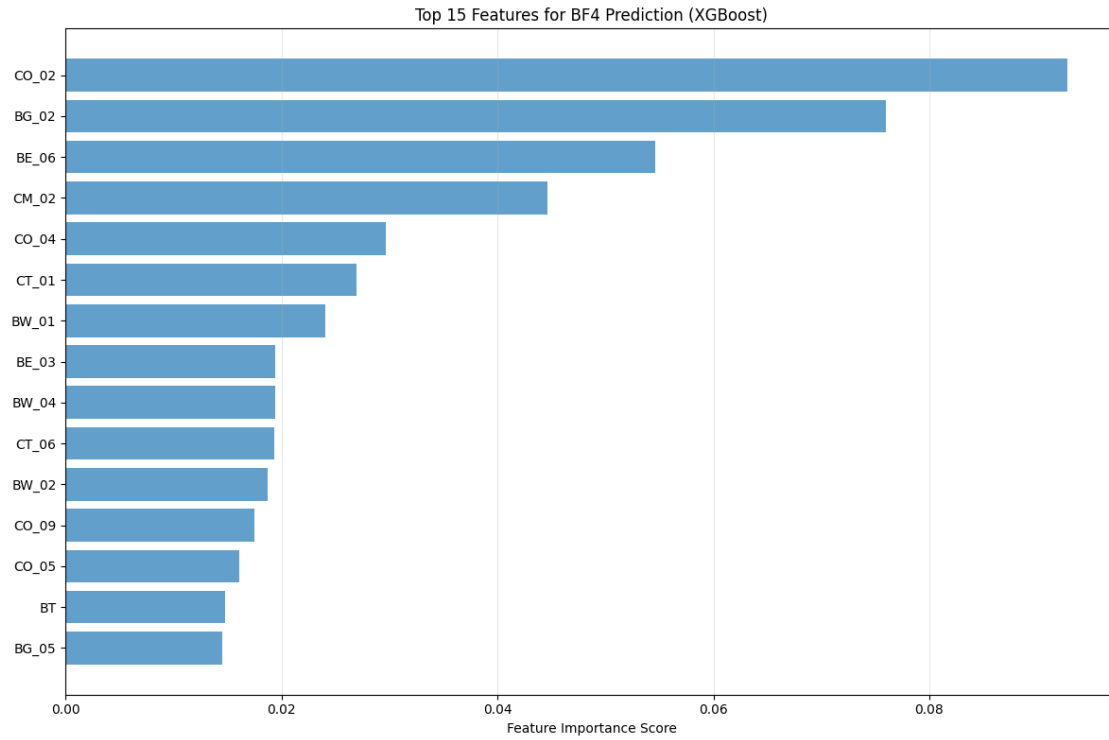
Accuracy: 0.8408
Precision: 0.8000
Recall: 0.8000
F1-Score: 0.8000
ROC-AUC: 0.9151
PR-AUC: 0.8714

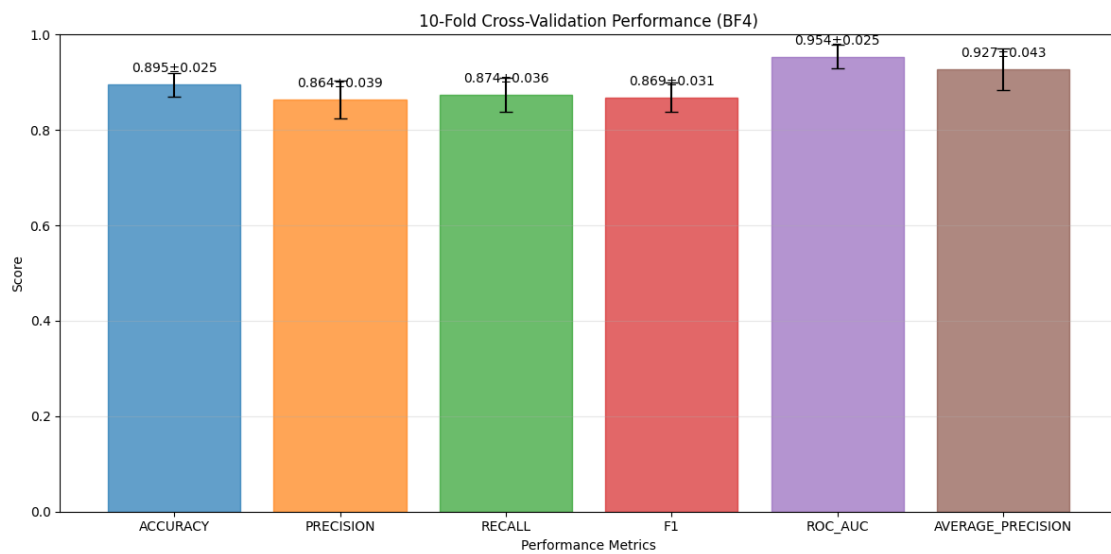
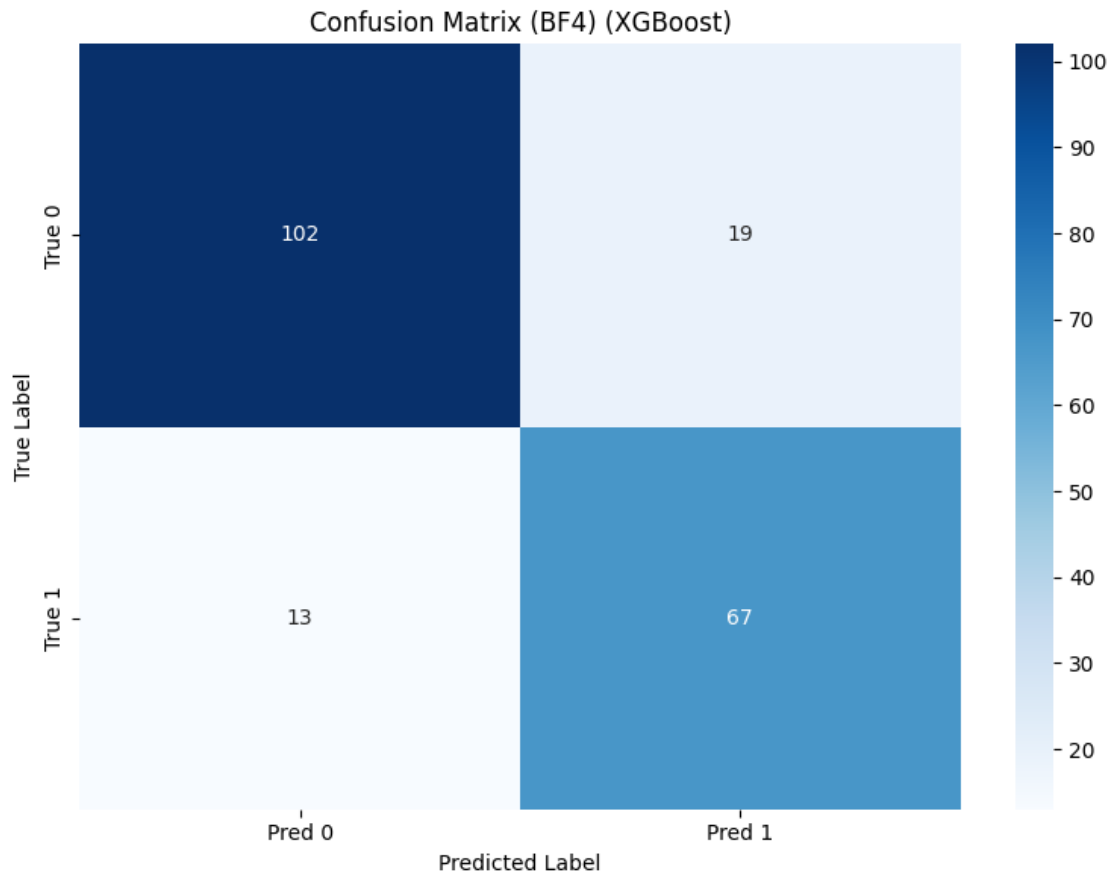
Validation Classification Report:

	precision	recall	f1-score	support
0	0.868	0.868	0.868	121
1	0.800	0.800	0.800	80
accuracy			0.841	201
macro avg	0.834	0.834	0.834	201
weighted avg	0.841	0.841	0.841	201

Validation Confusion Matrix:

```
[[TN FP]
 [FN TP]]
[[105 16]
 [ 16 64]]
```





SAVING OUTPUTS to bf4_comprehensive_outputs_xgb/ ...
Saved metrics, plots, predictions, and model.

DONE.

```
[4]: """
Reusable XGBoost Pipeline for Microfinance Impact Analysis
-----

Matches your Random Forest pipeline structure:
- Universal target detection (CI, BF1, BF_1, BF 1, bf-1, etc.)
- Median imputation + StandardScaler in a Pipeline
- 10-fold Stratified CV with multiple metrics
- Train/Test/Validation split option
- Comprehensive evaluation + plots + saving outputs (metrics, preds, model,
  ↳ feature importances)

NOTE:
- This is for binary classification (0/1 target).
"""

# =====
# 0) INSTALLS
# =====
# If you're in Colab, uncomment:
# !pip -q install xgboost openpyxl joblib

# =====
# 0) IMPORTS
# =====
import os, re, warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, StratifiedKFold,
  ↳ cross_validate
from sklearn.metrics import (
    classification_report, confusion_matrix, roc_auc_score, roc_curve,
    precision_recall_curve, average_precision_score,
    accuracy_score, precision_score, recall_score, f1_score
)
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
```

```

import joblib

from xgboost import XGBClassifier

RANDOM_STATE = 42

# =====
# 1) DATA LOADING
# =====
def load_excel_from_colab():
    """Upload Excel in Google Colab."""
    try:
        from google.colab import files
        print(" Please select the Excel file with your encoded dataset...")
        uploaded = files.upload()
        path = list(uploaded.keys())[0]
        df = pd.read_excel(path)
        print(f" Loaded dataset: {df.shape} (rows, cols)")
        return df
    except Exception as e:
        raise RuntimeError("Colab upload failed.") from e

def load_excel_locally():
    """Upload Excel when running locally."""
    try:
        import tkinter as tk
        from tkinter import filedialog
        print(" Please select the Excel file...")
        root = tk.Tk()
        root.withdraw()
        file_path = filedialog.askopenfilename(
            title="Select Excel file",
            filetypes=[("Excel files", "*.xlsx *.xls")]
        )
        if not file_path:
            raise ValueError("No file selected")
        df = pd.read_excel(file_path)
        print(f" Loaded dataset: {df.shape} (rows, cols)")
        return df
    except Exception as e:
        print(f" File dialog failed: {e}")
        file_path = input(" Enter full path to your Excel file: ")
        df = pd.read_excel(file_path)
        print(f" Loaded dataset: {df.shape} (rows, cols)")
        return df

```

```

# =====
# 2) UNIVERSAL TARGET SELECTION
# =====
def detect_target_column(df: pd.DataFrame, target: str) -> str:
    """
    Universal target detector:
    1) If `target` matches an existing column exactly -> use it.
    2) If target looks like BF<number>, try common BF aliases (BF 1, BF_1, BF-1, etc.).
    3) Else try case-insensitive exact match.
    4) Else ask user to type exact column name.
    """
    if target is None or str(target).strip() == "":
        raise ValueError("target cannot be empty. Example: target='CI' or target='BF1'")

    target = str(target).strip()

    # (1) Exact match
    if target in df.columns:
        print(f" Found target column (exact): {target}")
        return target

    # (2) BF alias support if user typed BF-style
    m = re.match(r"^s*BF\s*(\d+)\s*$", target, flags=re.I)
    if m:
        k = m.group(1)
        aliases = [
            f"BF{k}", f"bf{k}",
            f"BF {k}", f"bf {k}",
            f"BF_{k}", f"bf_{k}",
            f"BF-{k}", f"bf-{k}",
        ]
        for a in aliases:
            if a in df.columns:
                print(f" Found target column (BF alias): {a}")
                return a

    # regex scan for bf variants
    pat = re.compile(rf"\bbf\s*[_-]?s*{k}\b", flags=re.I)
    for col in df.columns:
        if pat.search(str(col)):
            print(f" Found target column (BF regex): {col}")
            return col

```

```

# (3) Case-insensitive exact match
lower_map = {str(c).strip().lower(): c for c in df.columns}
if target.lower() in lower_map:
    real_col = lower_map[target.lower()]
    print(f" Found target column (case-insensitive): {real_col}")
    return real_col

# (4) Ask user
print(f" Target column '{target}' not found automatically.")
print(" Available columns:")
print(df.columns.tolist())
user_col = input(" Enter the exact target column name: ").strip()
if user_col in df.columns:
    print(f" Using specified target column: {user_col}")
    return user_col
raise ValueError(f"Column '{user_col}' not found in dataset")

def preprocess_target_binary(y_series: pd.Series, target_label: str):
    """
    Convert target responses into binary classification (expects 0/1 after
    ↪cleaning).
    """
    y_numeric = pd.to_numeric(y_series, errors="coerce")
    valid_mask = y_numeric.notna() & np.isfinite(y_numeric)
    y_clean = y_numeric[valid_mask]

    if len(y_clean) == 0:
        raise ValueError(f" No valid values for {target_label} after cleaning")

    print(f"\n TARGET ANALYSIS: {target_label}")
    print("-" * 40)
    print("Value distribution:")
    print(y_clean.value_counts().sort_index())

    unique_vals = sorted(y_clean.unique())
    print(f"Unique values: {unique_vals}")

    y_binary = y_clean.astype(int)

    class_balance = y_binary.value_counts(normalize=True).round(3)
    print(f" Final class balance: {class_balance.to_dict()}")

    return y_binary, valid_mask

# =====

```

```

# 3) FEATURE ENGINEERING
# =====
def build_feature_matrix(df_clean: pd.DataFrame, target_col: str):
    """Prepare features for ML (numeric-only)."""
    print("\n FEATURE ENGINEERING")
    print("-" * 40)

    X = df_clean.drop(columns=[target_col], errors="ignore")
    numeric_cols = X.select_dtypes(include=[np.number]).columns.tolist()
    print(f" Found {len(numeric_cols)} numeric features")

    if len(numeric_cols) == 0:
        raise ValueError(" No numeric features found for modeling")

    print(" Handling missing values with median imputation...")
    imputer = SimpleImputer(strategy="median")
    X_imputed = imputer.fit_transform(X[numeric_cols])
    X_imputed = pd.DataFrame(X_imputed, columns=numeric_cols, index=X.index)

    non_constant_cols = [c for c in X_imputed.columns if X_imputed[c].nunique()
↳ 1]
    dropped = len(numeric_cols) - len(non_constant_cols)
    if dropped > 0:
        print(f" Removed {dropped} constant columns")

    X_final = X_imputed[non_constant_cols]
    print(f" Final feature matrix: {X_final.shape} (rows, cols)")
    return X_final, non_constant_cols

# =====
# 4) CROSS-VALIDATION
# =====
def _compute_scale_pos_weight(y):
    # For XGBoost: scale_pos_weight = (#neg / #pos)
    pos = int((y == 1).sum())
    neg = int((y == 0).sum())
    if pos == 0:
        return 1.0
    return neg / pos

def perform_cross_validation(X, y, cv_folds=10):
    print(f"\n PERFORMING {cv_folds}-FOLD CROSS-VALIDATION")
    print("=" * 50)

    preprocessor = Pipeline([

```

```

        ("impute", SimpleImputer(strategy="median")),
        ("scale", StandardScaler())
    ])

    spw = _compute_scale_pos_weight(y)

    xgb = XGBClassifier(
        n_estimators=600,
        learning_rate=0.05,
        max_depth=4,
        min_child_weight=3,
        subsample=0.85,
        colsample_bytree=0.85,
        gamma=0.0,
        reg_lambda=1.0,
        reg_alpha=0.0,
        objective="binary:logistic",
        eval_metric="logloss",
        random_state=RANDOM_STATE,
        n_jobs=-1,
        scale_pos_weight=spw,
        tree_method="hist" # fast + stable in CPU
    )

    pipe = Pipeline([("pre", preprocessor), ("xgb", xgb)])

    scoring = {
        "accuracy": "accuracy",
        "precision": "precision",
        "recall": "recall",
        "f1": "f1",
        "roc_auc": "roc_auc",
        "average_precision": "average_precision"
    }

    cv_results = cross_validate(
        pipe, X, y,
        cv=StratifiedKFold(n_splits=cv_folds, shuffle=True,
        ↪random_state=RANDOM_STATE),
        scoring=scoring,
        return_train_score=True,
        n_jobs=-1
    )

    print(" CROSS-VALIDATION RESULTS (Mean ± Std):")
    print("-" * 40)

```

```

metrics_summary = {}
for metric in scoring.keys():
    test_scores = cv_results[f"test_{metric}"]
    train_scores = cv_results[f"train_{metric}"]

    test_mean = float(np.mean(test_scores))
    test_std = float(np.std(test_scores))
    train_mean = float(np.mean(train_scores))

    metrics_summary[metric] = {
        "test_mean": test_mean,
        "test_std": test_std,
        "train_mean": train_mean
    }

    print(f"{metric.upper():<15}: {test_mean:.4f} ± {test_std:.4f} (train:␣
↪{train_mean:.4f})")

    overfit_gap = metrics_summary["accuracy"]["train_mean"] -␣
↪metrics_summary["accuracy"]["test_mean"]
    print(f"\n Overfitting Analysis:")
    print(f"    Train-Test Accuracy Gap: {overfit_gap:.4f}")
    print("    Potential overfitting detected" if overfit_gap > 0.05 else "␣
↪ Model generalizes well")

    return metrics_summary, cv_results

# =====
# 5) TRAIN/TEST/VALIDATION SPLITS + TRAINING
# =====
def create_validation_set(X, y, validation_size=0.2):
    print(f"\n CREATING VALIDATION SET ({validation_size*100:.0f}% of data)")
    print("=" * 50)

    X_train_test, X_val, y_train_test, y_val = train_test_split(
        X, y, test_size=validation_size, stratify=y, random_state=RANDOM_STATE
    )

    X_train, X_test, y_train, y_test = train_test_split(
        X_train_test, y_train_test, test_size=0.25, stratify=y_train_test,␣
↪random_state=RANDOM_STATE
    )

    print(f" Training set: {X_train.shape[0]} samples")
    print(f" Test set: {X_test.shape[0]} samples")
    print(f" Validation set: {X_val.shape[0]} samples")

```

```

return X_train, X_test, X_val, y_train, y_test, y_val

def train_with_comprehensive_validation(X, y, use_validation_set=True):
    print("\n MODEL TRAINING: XGBoost")
    print("-" * 40)

    preprocessor = Pipeline([
        ("impute", SimpleImputer(strategy="median")),
        ("scale", StandardScaler())
    ])

    spw = _compute_scale_pos_weight(y)

    xgb = XGBClassifier(
        n_estimators=600,
        learning_rate=0.05,
        max_depth=4,
        min_child_weight=3,
        subsample=0.85,
        colsample_bytree=0.85,
        gamma=0.0,
        reg_lambda=1.0,
        reg_alpha=0.0,
        objective="binary:logistic",
        eval_metric="logloss",
        random_state=RANDOM_STATE,
        n_jobs=-1,
        scale_pos_weight=spw,
        tree_method="hist"
    )

    pipe = Pipeline([("pre", preprocessor), ("xgb", xgb)])

    if use_validation_set:
        X_train, X_test, X_val, y_train, y_test, y_val = \
        ↪ create_validation_set(X, y)

        # Early stopping uses a validation set, but sklearn.Pipeline
        ↪ complicates passing eval_set cleanly.
        # So we keep training stable via regularization + conservative params.
        print(" Training model with regularization...")
        pipe.fit(X_train, y_train)

    return pipe, X_train, X_test, X_val, y_train, y_test, y_val

```

```

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=RANDOM_STATE
)
print(f" Training set: {X_train.shape[0]} samples")
print(f" Test set: {X_test.shape[0]} samples")
print(" Training model with regularization...")
pipe.fit(X_train, y_train)
return pipe, X_train, X_test, None, y_train, y_test, None

# =====
# 6) EVALUATION
# =====
def evaluate_model_comprehensive(model, X_test, y_test, dataset_name="Test"):
    y_pred = model.predict(X_test)
    y_proba = model.predict_proba(X_test)[:, 1]

    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred, zero_division=0)
    rec = recall_score(y_test, y_pred, zero_division=0)
    f1 = f1_score(y_test, y_pred, zero_division=0)
    roc_auc = roc_auc_score(y_test, y_proba)
    pr_auc = average_precision_score(y_test, y_proba)

    print("\n" + "=" * 60)
    print(f" {dataset_name.upper()} SET COMPREHENSIVE EVALUATION")
    print("=" * 60)
    print(f" Accuracy: {acc:.4f}")
    print(f" Precision: {prec:.4f}")
    print(f" Recall: {rec:.4f}")
    print(f" F1-Score: {f1:.4f}")
    print(f" ROC-AUC: {roc_auc:.4f}")
    print(f" PR-AUC: {pr_auc:.4f}")

    print(f"\n {dataset_name} Classification Report:")
    print(classification_report(y_test, y_pred, digits=3))

    cm = confusion_matrix(y_test, y_pred)
    print(f" {dataset_name} Confusion Matrix:")
    print(" [[TN FP]\n [FN TP]]")
    print(cm)

    return y_pred, y_proba, {
        "accuracy": acc, "precision": prec, "recall": rec, "f1": f1,
        "roc_auc": roc_auc, "pr_auc": pr_auc
    }

```

```

# =====
# 7) VISUALIZATIONS
# =====
def create_validation_comparison_plot(cv_metrics, output_dir, prefix):
    metrics_names = list(cv_metrics.keys())
    test_means = [cv_metrics[m]["test_mean"] for m in metrics_names]
    test_stds = [cv_metrics[m]["test_std"] for m in metrics_names]

    plt.figure(figsize=(12, 6))
    x_pos = np.arange(len(metrics_names))
    bars = plt.bar(x_pos, test_means, yerr=test_stds, capsize=5, alpha=0.7)

    for i, b in enumerate(bars):
        b.set_color(f"C{i}")

    plt.xlabel("Performance Metrics")
    plt.ylabel("Score")
    plt.title(f"10-Fold Cross-Validation Performance ({prefix.upper()})")
    plt.xticks(x_pos, [m.upper() for m in metrics_names], rotation=0)
    plt.ylim(0, 1)
    plt.grid(True, alpha=0.3, axis="y")

    for i, (mean, std) in enumerate(zip(test_means, test_stds)):
        plt.text(i, mean + 0.02, f"{mean:.3f}±{std:.3f}", ha="center",
        ↪va="bottom", fontsize=10)

    plt.tight_layout()
    plt.savefig(f"{output_dir}/{prefix}_cross_validation_results.png", dpi=200,
    ↪bbox_inches="tight")
    plt.show()

def create_comprehensive_visualizations(y_test, y_proba, model, feature_names,
    ↪output_dir, prefix, cv_metrics=None):
    os.makedirs(output_dir, exist_ok=True)

    xgb_model = model.named_steps["xgb"]
    imp_df = pd.DataFrame({
        "feature": feature_names,
        "importance": xgb_model.feature_importances_
    }).sort_values("importance", ascending=False)

    imp_df.to_csv(f"{output_dir}/{prefix}_feature_importances.csv", index=False)

    # Feature importance plot
    plt.figure(figsize=(12, 8))

```

```

top_features = imp_df.head(15).iloc[:, :-1]
plt.barh(range(len(top_features)), top_features["importance"], alpha=0.7)
plt.yticks(range(len(top_features)), top_features["feature"])
plt.xlabel("Feature Importance Score")
plt.title(f"Top 15 Features for {prefix.upper()} Prediction (XGBoost)")
plt.grid(True, alpha=0.3, axis="x")
plt.tight_layout()
plt.savefig(f"{output_dir}/{prefix}_feature_importances.png", dpi=200,
↳bbox_inches="tight")
plt.show()

# ROC + PR curves
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))

fpr, tpr, _ = roc_curve(y_test, y_proba)
roc_auc = roc_auc_score(y_test, y_proba)
ax1.plot(fpr, tpr, label=f"AUC = {roc_auc:.3f}", linewidth=2, color="C0")
ax1.plot([0, 1], [0, 1], "k--", alpha=0.5)
ax1.set_xlabel("False Positive Rate")
ax1.set_ylabel("True Positive Rate")
ax1.set_title(f"ROC Curve ({prefix.upper()}")
ax1.legend()
ax1.grid(True, alpha=0.3)

precision_vals, recall_vals, _ = precision_recall_curve(y_test, y_proba)
pr_auc = average_precision_score(y_test, y_proba)
ax2.plot(recall_vals, precision_vals, label=f"AP = {pr_auc:.3f}",
↳linewidth=2, color="C1")
ax2.set_xlabel("Recall")
ax2.set_ylabel("Precision")
ax2.set_title(f"Precision-Recall Curve ({prefix.upper()}")
ax2.legend()
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig(f"{output_dir}/{prefix}_roc_pr_curves.png", dpi=200,
↳bbox_inches="tight")
plt.show()

# Confusion matrix heatmap
y_pred = (y_proba >= 0.5).astype(int)
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=["Pred 0", "Pred 1"],
            yticklabels=["True 0", "True 1"])

```

```

plt.title(f"Confusion Matrix ({prefix.upper()}) (XGBoost)")
plt.ylabel("True Label")
plt.xlabel("Predicted Label")
plt.tight_layout()
plt.savefig(f"{output_dir}/{prefix}_confusion_matrix.png", dpi=200,
↳ bbox_inches="tight")
plt.show()

if cv_metrics:
    create_validation_comparison_plot(cv_metrics, output_dir, prefix)

return imp_df

# =====
# 8) SAVE RESULTS
# =====
def save_comprehensive_results(model, metrics, imp_df, df_clean, target_col,
↳ feature_names, output_dir, prefix, cv_metrics=None):
    print(f"\n SAVING OUTPUTS to {output_dir}/ ...")
    os.makedirs(output_dir, exist_ok=True)

    with open(f"{output_dir}/{prefix}_comprehensive_metrics.txt", "w",
↳ encoding="utf-8") as f:
        f.write(f"COMPREHENSIVE XGBOOST RESULTS ({prefix.upper()})\n")
        f.write("=" * 60 + "\n")
        f.write(f"Dataset shape: {df_clean.shape}\n")
        f.write(f"Target column: {target_col}\n")
        f.write(f"Features used: {len(feature_names)}\n\n")

        f.write("TEST SET METRICS:\n")
        for k, v in metrics.items():
            f.write(f"{k}: {v:.4f}\n")
        f.write("\n")

        if cv_metrics:
            f.write("10-FOLD CROSS-VALIDATION (Mean ± Std) WITH MULTIPLE
↳ METRICS:\n")
            for metric, scores in cv_metrics.items():
                f.write(f"{metric}: {scores['test_mean']:.4f} ±
↳ {scores['test_std']:.4f}\n")
            f.write("\n")

        f.write("TOP 10 FEATURES:\n")
        for _, row in imp_df.head(10).iterrows():
            f.write(f"{row['feature']}: {row['importance']:.6f}\n")

```

```

# Full-dataset predictions
X_full = df_clean.drop(columns=[target_col], errors="ignore")
X_full = X_full.reindex(columns=feature_names)

preds = model.predict_proba(X_full)[: , 1]
predictions_df = df_clean.copy()
predictions_df[f"{prefix}_Pred_Prob"] = preds
predictions_df[f"{prefix}_Pred"] = (preds >= 0.5).astype(int)
predictions_df[f"{prefix}_Correct"] = (predictions_df[f"{prefix}_Pred"] ==
↳ predictions_df[target_col]).astype(int)

predictions_df.to_excel(f"{output_dir}/{prefix}_predictions.xlsx",
↳ index=False)
joblib.dump(model, f"{output_dir}/{prefix}_xgb_model.joblib")

print(" Saved metrics, plots, predictions, and model.")

# =====
# 9) MAIN PIPELINE (UNIVERSAL TARGET)
# =====
def run_comprehensive_analysis(
    target="CI",
    use_cross_validation=True,
    use_validation_set=True,
    df=None
):
    """
    Run the full pipeline for ANY target column you type.
    """
    try:
        prefix = re.sub(r"^[a-zA-Z0-9]+", "_", str(target).strip().lower()).
↳ strip("_")
        output_dir = f"{prefix}_comprehensive_outputs_xgb"

        print(" COMPREHENSIVE MICROFINANCE ANALYSIS (XGBoost)")
        print("=" * 60)
        print(f" Target: {target}")
        print(f" Output folder: {output_dir}/")
        print("=" * 60)

        if df is None:
            try:
                import google.colab # noqa: F401
                df = load_excel_from_colab()
            except Exception:
                df = load_excel_locally()

```

```

print(f" Initial dataset shape: {df.shape}")

target_col = detect_target_column(df, target=target)

y, valid_mask = preprocess_target_binary(df[target_col],
↳target_label=target_col)
df_clean = df.loc[valid_mask].copy()
print(f" Clean dataset: {df_clean.shape} (after removing invalid
↳target rows)")

X_final, feature_names = build_feature_matrix(df_clean, target_col)

cv_metrics = None
if use_cross_validation:
    cv_metrics, _ = perform_cross_validation(X_final, y, cv_folds=10)

model, X_train, X_test, X_val, y_train, y_test, y_val =
↳train_with_comprehensive_validation(
    X_final, y, use_validation_set=use_validation_set
)

_, y_proba, test_metrics = evaluate_model_comprehensive(model, X_test,
↳y_test, "Test")

if X_val is not None and y_val is not None:
    evaluate_model_comprehensive(model, X_val, y_val, "Validation")

imp_df = create_comprehensive_visualizations(
    y_test=y_test,
    y_proba=y_proba,
    model=model,
    feature_names=feature_names,
    output_dir=output_dir,
    prefix=prefix,
    cv_metrics=cv_metrics
)

save_comprehensive_results(
    model=model,
    metrics=test_metrics,
    imp_df=imp_df,
    df_clean=df_clean,
    target_col=target_col,
    feature_names=feature_names,
    output_dir=output_dir,
    prefix=prefix,

```

```

        cv_metrics=cv_metrics
    )

    print("\n DONE.")
    return model, test_metrics, imp_df, cv_metrics

except Exception as e:
    print(f"\n Analysis failed: {type(e).__name__}: {e}")
    import traceback
    traceback.print_exc(limit=2)
    return None, None, None, None

# ===== USAGE EXAMPLES =====
if __name__ == "__main__":
    run_comprehensive_analysis(target="BF5", use_cross_validation=True,
    ↪use_validation_set=True)

    # Other examples:
    # run_comprehensive_analysis(target="CI", use_cross_validation=True,
    ↪use_validation_set=True)
    # for t in ["CI", "BF1", "BF3", "BF4"]:
    #     run_comprehensive_analysis(target=t, use_cross_validation=True,
    ↪use_validation_set=True)

```

COMPREHENSIVE MICROFINANCE ANALYSIS (XGBoost)

```

=====
Target: BF5
Output folder: bf5_comprehensive_outputs_xgb/
=====

Please select the Excel file with your encoded dataset...

<IPython.core.display.HTML object>

Saving rf_dataset_encoded_dataset.xlsx to rf_dataset_encoded_dataset (3).xlsx
Loaded dataset: (1002, 85) (rows, cols)
Initial dataset shape: (1002, 85)
Target column 'BF5' not found automatically.
Available columns:
['D', 'AY', 'AZ', 'BA', 'BB', 'BC', 'BD', 'BE_01', 'BE_02', 'BE_03', 'BE_04',
'BE_05', 'BE_06', 'BE_07', 'BE_08', 'BE_09', 'BE_10', 'BF', 'BG_01', 'BG_02',
'BG_03', 'BG_04', 'BG_05', 'BG_06', 'BH', 'BI', 'BN', 'BO', 'BP', 'BQ', 'BR',
'BS', 'BT', 'BU', 'BV', 'BW_01', 'BW_02', 'BW_03', 'BW_04', 'BW_05', 'BX', 'BY',
'CB', 'CD', 'CE', 'CF', 'CG', 'CH', 'CI', 'CK', 'CL', 'CM_01', 'CM_02', 'CM_03',
'CM_04', 'CM_05', 'CM_06', 'CN', 'CO_01', 'CO_02', 'CO_03', 'CO_04', 'CO_05',
'CO_06', 'CO_07', 'CO_08', 'CO_09', 'CP', 'CR_01', 'CR_02', 'CR_03', 'CR_04',
'CS', 'CT_01', 'CT_02', 'CT_03', 'CT_04', 'CT_05', 'CT_06', 'CU', 'Occupation',
'Financial Behavior', 'Family', 'Lifestyle', 'Digital Literacy']

```

Enter the exact target column name: BG_05
Using specified target column: BG_05

TARGET ANALYSIS: BG_05

Value distribution:

BG_05

0 657

1 345

Name: count, dtype: int64

Unique values: [np.int64(0), np.int64(1)]

Final class balance: {0: 0.656, 1: 0.344}

Clean dataset: (1002, 85) (after removing invalid target rows)

FEATURE ENGINEERING

Found 84 numeric features

Handling missing values with median imputation...

Final feature matrix: (1002, 84) (rows, cols)

PERFORMING 10-FOLD CROSS-VALIDATION

=====

CROSS-VALIDATION RESULTS (Mean ± Std):

ACCURACY	: 0.8773 ± 0.0331 (train: 0.9997)
PRECISION	: 0.8214 ± 0.0594 (train: 0.9990)
RECALL	: 0.8261 ± 0.0564 (train: 1.0000)
F1	: 0.8224 ± 0.0475 (train: 0.9995)
ROC_AUC	: 0.9386 ± 0.0272 (train: 1.0000)
AVERAGE_PRECISION	: 0.8943 ± 0.0504 (train: 1.0000)

Overfitting Analysis:

Train-Test Accuracy Gap: 0.1224

Potential overfitting detected

MODEL TRAINING: XGBoost

CREATING VALIDATION SET (20% of data)

=====

Training set: 600 samples

Test set: 201 samples

Validation set: 201 samples

Training model with regularization...

=====

TEST SET COMPREHENSIVE EVALUATION

=====

Accuracy: 0.8408
Precision: 0.7761
Recall: 0.7536
F1-Score: 0.7647
ROC-AUC: 0.9093
PR-AUC: 0.8722

Test Classification Report:

	precision	recall	f1-score	support
0	0.873	0.886	0.880	132
1	0.776	0.754	0.765	69
accuracy			0.841	201
macro avg	0.825	0.820	0.822	201
weighted avg	0.840	0.841	0.840	201

Test Confusion Matrix:

```
[[TN FP]
 [FN TP]]
[[117 15]
 [ 17 52]]
```

=====

VALIDATION SET COMPREHENSIVE EVALUATION

=====

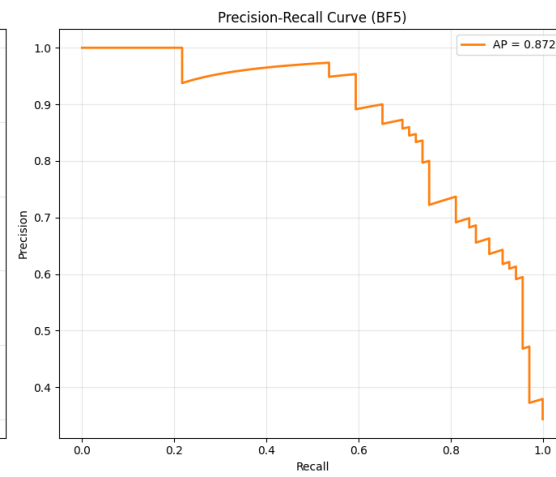
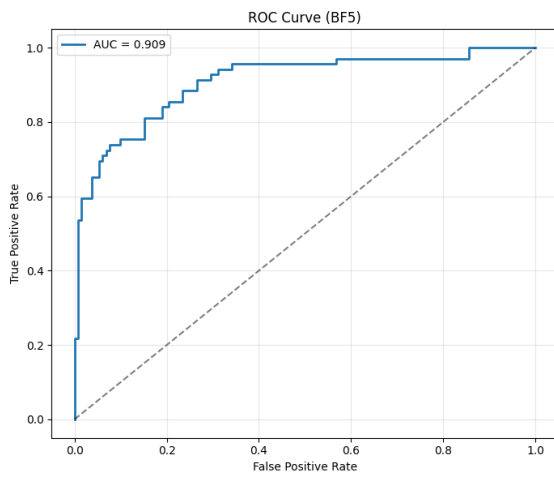
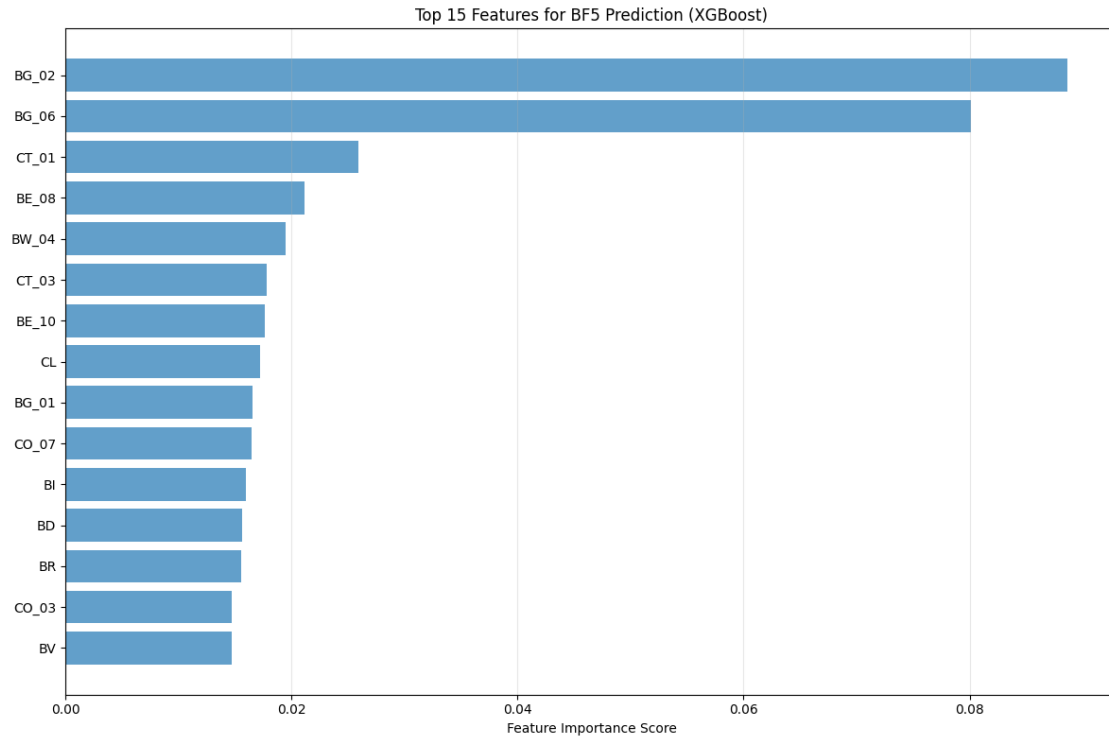
Accuracy: 0.8557
Precision: 0.8125
Recall: 0.7536
F1-Score: 0.7820
ROC-AUC: 0.9069
PR-AUC: 0.8456

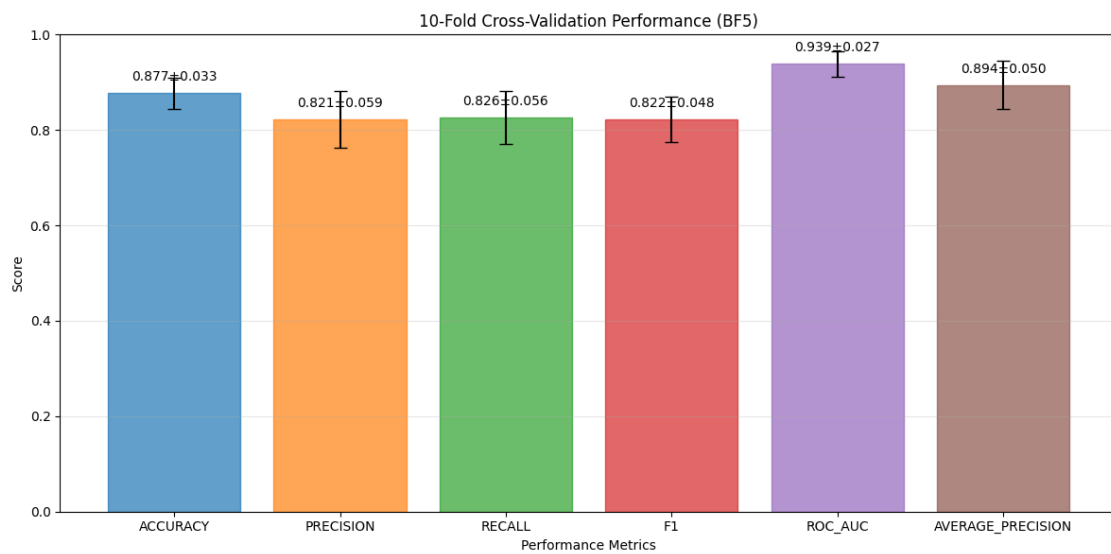
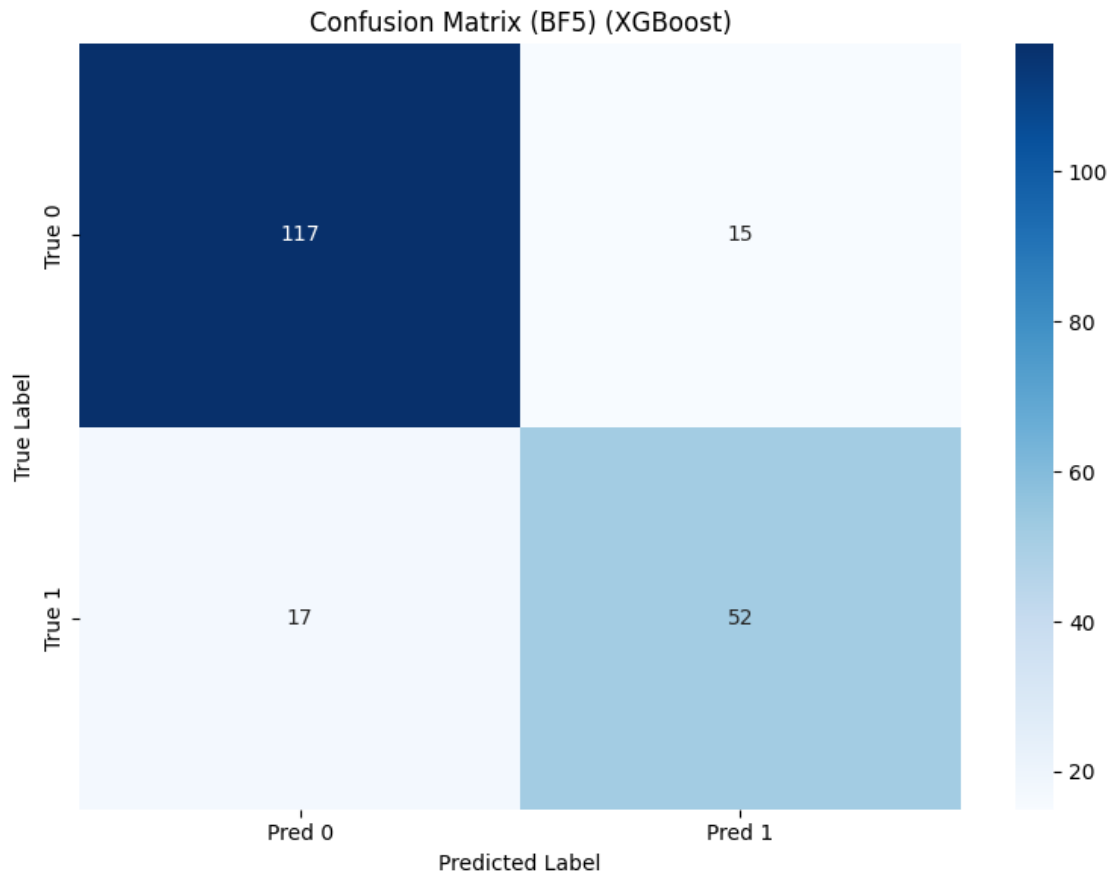
Validation Classification Report:

	precision	recall	f1-score	support
0	0.876	0.909	0.892	132
1	0.812	0.754	0.782	69
accuracy			0.856	201
macro avg	0.844	0.831	0.837	201
weighted avg	0.854	0.856	0.854	201

Validation Confusion Matrix:

```
[[TN FP]
 [FN TP]]
[[120 12]
 [ 17 52]]
```





SAVING OUTPUTS to bf5_comprehensive_outputs_xgb/ ...
Saved metrics, plots, predictions, and model.

DONE.

```
[5]: """
Reusable XGBoost Pipeline for Microfinance Impact Analysis
-----

Matches your Random Forest pipeline structure:
- Universal target detection (CI, BF1, BF_1, BF 1, bf-1, etc.)
- Median imputation + StandardScaler in a Pipeline
- 10-fold Stratified CV with multiple metrics
- Train/Test/Validation split option
- Comprehensive evaluation + plots + saving outputs (metrics, preds, model,
  ↳feature importances)

NOTE:
- This is for binary classification (0/1 target).
"""

# =====
# 0) INSTALLS
# =====
# If you're in Colab, uncomment:
# !pip -q install xgboost openpyxl joblib

# =====
# 0) IMPORTS
# =====
import os, re, warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, StratifiedKFold,
  ↳cross_validate
from sklearn.metrics import (
    classification_report, confusion_matrix, roc_auc_score, roc_curve,
    precision_recall_curve, average_precision_score,
    accuracy_score, precision_score, recall_score, f1_score
)
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
```

```

import joblib

from xgboost import XGBClassifier

RANDOM_STATE = 42

# =====
# 1) DATA LOADING
# =====
def load_excel_from_colab():
    """Upload Excel in Google Colab."""
    try:
        from google.colab import files
        print(" Please select the Excel file with your encoded dataset...")
        uploaded = files.upload()
        path = list(uploaded.keys())[0]
        df = pd.read_excel(path)
        print(f" Loaded dataset: {df.shape} (rows, cols)")
        return df
    except Exception as e:
        raise RuntimeError("Colab upload failed.") from e

def load_excel_locally():
    """Upload Excel when running locally."""
    try:
        import tkinter as tk
        from tkinter import filedialog
        print(" Please select the Excel file...")
        root = tk.Tk()
        root.withdraw()
        file_path = filedialog.askopenfilename(
            title="Select Excel file",
            filetypes=[("Excel files", "*.xlsx *.xls")]
        )
        if not file_path:
            raise ValueError("No file selected")
        df = pd.read_excel(file_path)
        print(f" Loaded dataset: {df.shape} (rows, cols)")
        return df
    except Exception as e:
        print(f" File dialog failed: {e}")
        file_path = input(" Enter full path to your Excel file: ")
        df = pd.read_excel(file_path)
        print(f" Loaded dataset: {df.shape} (rows, cols)")
        return df

```

```

# =====
# 2) UNIVERSAL TARGET SELECTION
# =====
def detect_target_column(df: pd.DataFrame, target: str) -> str:
    """
    Universal target detector:
    1) If `target` matches an existing column exactly -> use it.
    2) If target looks like BF<number>, try common BF aliases (BF 1, BF_1, BF-1, etc.).
    3) Else try case-insensitive exact match.
    4) Else ask user to type exact column name.
    """
    if target is None or str(target).strip() == "":
        raise ValueError("target cannot be empty. Example: target='CI' or target='BF1'")

    target = str(target).strip()

    # (1) Exact match
    if target in df.columns:
        print(f" Found target column (exact): {target}")
        return target

    # (2) BF alias support if user typed BF-style
    m = re.match(r"^s*BF\s*(\d+)\s*$", target, flags=re.I)
    if m:
        k = m.group(1)
        aliases = [
            f"BF{k}", f"bf{k}",
            f"BF {k}", f"bf {k}",
            f"BF_{k}", f"bf_{k}",
            f"BF-{k}", f"bf-{k}",
        ]
        for a in aliases:
            if a in df.columns:
                print(f" Found target column (BF alias): {a}")
                return a

    # regex scan for bf variants
    pat = re.compile(rf"\bbf\s*[_-]?s*{k}\b", flags=re.I)
    for col in df.columns:
        if pat.search(str(col)):
            print(f" Found target column (BF regex): {col}")
            return col

```

```

# (3) Case-insensitive exact match
lower_map = {str(c).strip().lower(): c for c in df.columns}
if target.lower() in lower_map:
    real_col = lower_map[target.lower()]
    print(f" Found target column (case-insensitive): {real_col}")
    return real_col

# (4) Ask user
print(f" Target column '{target}' not found automatically.")
print(" Available columns:")
print(df.columns.tolist())
user_col = input(" Enter the exact target column name: ").strip()
if user_col in df.columns:
    print(f" Using specified target column: {user_col}")
    return user_col
raise ValueError(f"Column '{user_col}' not found in dataset")

def preprocess_target_binary(y_series: pd.Series, target_label: str):
    """
    Convert target responses into binary classification (expects 0/1 after
    ↪cleaning).
    """
    y_numeric = pd.to_numeric(y_series, errors="coerce")
    valid_mask = y_numeric.notna() & np.isfinite(y_numeric)
    y_clean = y_numeric[valid_mask]

    if len(y_clean) == 0:
        raise ValueError(f" No valid values for {target_label} after cleaning")

    print(f"\n TARGET ANALYSIS: {target_label}")
    print("-" * 40)
    print("Value distribution:")
    print(y_clean.value_counts().sort_index())

    unique_vals = sorted(y_clean.unique())
    print(f"Unique values: {unique_vals}")

    y_binary = y_clean.astype(int)

    class_balance = y_binary.value_counts(normalize=True).round(3)
    print(f" Final class balance: {class_balance.to_dict()}")

    return y_binary, valid_mask

# =====

```

```

# 3) FEATURE ENGINEERING
# =====
def build_feature_matrix(df_clean: pd.DataFrame, target_col: str):
    """Prepare features for ML (numeric-only)."""
    print("\n FEATURE ENGINEERING")
    print("-" * 40)

    X = df_clean.drop(columns=[target_col], errors="ignore")
    numeric_cols = X.select_dtypes(include=[np.number]).columns.tolist()
    print(f" Found {len(numeric_cols)} numeric features")

    if len(numeric_cols) == 0:
        raise ValueError(" No numeric features found for modeling")

    print(" Handling missing values with median imputation...")
    imputer = SimpleImputer(strategy="median")
    X_imputed = imputer.fit_transform(X[numeric_cols])
    X_imputed = pd.DataFrame(X_imputed, columns=numeric_cols, index=X.index)

    non_constant_cols = [c for c in X_imputed.columns if X_imputed[c].nunique()
↳ 1]
    dropped = len(numeric_cols) - len(non_constant_cols)
    if dropped > 0:
        print(f" Removed {dropped} constant columns")

    X_final = X_imputed[non_constant_cols]
    print(f" Final feature matrix: {X_final.shape} (rows, cols)")
    return X_final, non_constant_cols

# =====
# 4) CROSS-VALIDATION
# =====
def _compute_scale_pos_weight(y):
    # For XGBoost: scale_pos_weight = (#neg / #pos)
    pos = int((y == 1).sum())
    neg = int((y == 0).sum())
    if pos == 0:
        return 1.0
    return neg / pos

def perform_cross_validation(X, y, cv_folds=10):
    print(f"\n PERFORMING {cv_folds}-FOLD CROSS-VALIDATION")
    print("=" * 50)

    preprocessor = Pipeline([

```

```

        ("impute", SimpleImputer(strategy="median")),
        ("scale", StandardScaler())
    ])

    spw = _compute_scale_pos_weight(y)

    xgb = XGBClassifier(
        n_estimators=600,
        learning_rate=0.05,
        max_depth=4,
        min_child_weight=3,
        subsample=0.85,
        colsample_bytree=0.85,
        gamma=0.0,
        reg_lambda=1.0,
        reg_alpha=0.0,
        objective="binary:logistic",
        eval_metric="logloss",
        random_state=RANDOM_STATE,
        n_jobs=-1,
        scale_pos_weight=spw,
        tree_method="hist" # fast + stable in CPU
    )

    pipe = Pipeline([("pre", preprocessor), ("xgb", xgb)])

    scoring = {
        "accuracy": "accuracy",
        "precision": "precision",
        "recall": "recall",
        "f1": "f1",
        "roc_auc": "roc_auc",
        "average_precision": "average_precision"
    }

    cv_results = cross_validate(
        pipe, X, y,
        cv=StratifiedKfold(n_splits=cv_folds, shuffle=True,
        ↪random_state=RANDOM_STATE),
        scoring=scoring,
        return_train_score=True,
        n_jobs=-1
    )

    print(" CROSS-VALIDATION RESULTS (Mean ± Std):")
    print("-" * 40)

```

```

metrics_summary = {}
for metric in scoring.keys():
    test_scores = cv_results[f"test_{metric}"]
    train_scores = cv_results[f"train_{metric}"]

    test_mean = float(np.mean(test_scores))
    test_std = float(np.std(test_scores))
    train_mean = float(np.mean(train_scores))

    metrics_summary[metric] = {
        "test_mean": test_mean,
        "test_std": test_std,
        "train_mean": train_mean
    }

    print(f"{metric.upper():<15}: {test_mean:.4f} ± {test_std:.4f} (train:␣
↪{train_mean:.4f})")

    overfit_gap = metrics_summary["accuracy"]["train_mean"] -␣
↪metrics_summary["accuracy"]["test_mean"]
    print(f"\n Overfitting Analysis:")
    print(f"    Train-Test Accuracy Gap: {overfit_gap:.4f}")
    print("    Potential overfitting detected" if overfit_gap > 0.05 else "␣
↪ Model generalizes well")

    return metrics_summary, cv_results

# =====
# 5) TRAIN/TEST/VALIDATION SPLITS + TRAINING
# =====
def create_validation_set(X, y, validation_size=0.2):
    print(f"\n CREATING VALIDATION SET ({validation_size*100:.0f}% of data)")
    print("=" * 50)

    X_train_test, X_val, y_train_test, y_val = train_test_split(
        X, y, test_size=validation_size, stratify=y, random_state=RANDOM_STATE
    )

    X_train, X_test, y_train, y_test = train_test_split(
        X_train_test, y_train_test, test_size=0.25, stratify=y_train_test,␣
↪random_state=RANDOM_STATE
    )

    print(f" Training set: {X_train.shape[0]} samples")
    print(f" Test set: {X_test.shape[0]} samples")
    print(f" Validation set: {X_val.shape[0]} samples")

```

```

return X_train, X_test, X_val, y_train, y_test, y_val

def train_with_comprehensive_validation(X, y, use_validation_set=True):
    print("\n MODEL TRAINING: XGBoost")
    print("-" * 40)

    preprocessor = Pipeline([
        ("impute", SimpleImputer(strategy="median")),
        ("scale", StandardScaler())
    ])

    spw = _compute_scale_pos_weight(y)

    xgb = XGBClassifier(
        n_estimators=600,
        learning_rate=0.05,
        max_depth=4,
        min_child_weight=3,
        subsample=0.85,
        colsample_bytree=0.85,
        gamma=0.0,
        reg_lambda=1.0,
        reg_alpha=0.0,
        objective="binary:logistic",
        eval_metric="logloss",
        random_state=RANDOM_STATE,
        n_jobs=-1,
        scale_pos_weight=spw,
        tree_method="hist"
    )

    pipe = Pipeline([("pre", preprocessor), ("xgb", xgb)])

    if use_validation_set:
        X_train, X_test, X_val, y_train, y_test, y_val = \
        ↪ create_validation_set(X, y)

        # Early stopping uses a validation set, but sklearn.Pipeline
        ↪ complicates passing eval_set cleanly.
        # So we keep training stable via regularization + conservative params.
        print(" Training model with regularization...")
        pipe.fit(X_train, y_train)

    return pipe, X_train, X_test, X_val, y_train, y_test, y_val

```

```

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=RANDOM_STATE
)
print(f" Training set: {X_train.shape[0]} samples")
print(f" Test set: {X_test.shape[0]} samples")
print(" Training model with regularization...")
pipe.fit(X_train, y_train)
return pipe, X_train, X_test, None, y_train, y_test, None

# =====
# 6) EVALUATION
# =====
def evaluate_model_comprehensive(model, X_test, y_test, dataset_name="Test"):
    y_pred = model.predict(X_test)
    y_proba = model.predict_proba(X_test)[:, 1]

    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred, zero_division=0)
    rec = recall_score(y_test, y_pred, zero_division=0)
    f1 = f1_score(y_test, y_pred, zero_division=0)
    roc_auc = roc_auc_score(y_test, y_proba)
    pr_auc = average_precision_score(y_test, y_proba)

    print("\n" + "=" * 60)
    print(f" {dataset_name.upper()} SET COMPREHENSIVE EVALUATION")
    print("=" * 60)
    print(f" Accuracy: {acc:.4f}")
    print(f" Precision: {prec:.4f}")
    print(f" Recall: {rec:.4f}")
    print(f" F1-Score: {f1:.4f}")
    print(f" ROC-AUC: {roc_auc:.4f}")
    print(f" PR-AUC: {pr_auc:.4f}")

    print(f"\n {dataset_name} Classification Report:")
    print(classification_report(y_test, y_pred, digits=3))

    cm = confusion_matrix(y_test, y_pred)
    print(f" {dataset_name} Confusion Matrix:")
    print(" [[TN FP]\n [FN TP]]")
    print(cm)

    return y_pred, y_proba, {
        "accuracy": acc, "precision": prec, "recall": rec, "f1": f1,
        "roc_auc": roc_auc, "pr_auc": pr_auc
    }

```

```

# =====
# 7) VISUALIZATIONS
# =====
def create_validation_comparison_plot(cv_metrics, output_dir, prefix):
    metrics_names = list(cv_metrics.keys())
    test_means = [cv_metrics[m]["test_mean"] for m in metrics_names]
    test_stds = [cv_metrics[m]["test_std"] for m in metrics_names]

    plt.figure(figsize=(12, 6))
    x_pos = np.arange(len(metrics_names))
    bars = plt.bar(x_pos, test_means, yerr=test_stds, capsize=5, alpha=0.7)

    for i, b in enumerate(bars):
        b.set_color(f"C{i}")

    plt.xlabel("Performance Metrics")
    plt.ylabel("Score")
    plt.title(f"10-Fold Cross-Validation Performance ({prefix.upper()})")
    plt.xticks(x_pos, [m.upper() for m in metrics_names], rotation=0)
    plt.ylim(0, 1)
    plt.grid(True, alpha=0.3, axis="y")

    for i, (mean, std) in enumerate(zip(test_means, test_stds)):
        plt.text(i, mean + 0.02, f"{mean:.3f}±{std:.3f}", ha="center",
        ↪va="bottom", fontsize=10)

    plt.tight_layout()
    plt.savefig(f"{output_dir}/{prefix}_cross_validation_results.png", dpi=200,
    ↪bbox_inches="tight")
    plt.show()

def create_comprehensive_visualizations(y_test, y_proba, model, feature_names,
    ↪output_dir, prefix, cv_metrics=None):
    os.makedirs(output_dir, exist_ok=True)

    xgb_model = model.named_steps["xgb"]
    imp_df = pd.DataFrame({
        "feature": feature_names,
        "importance": xgb_model.feature_importances_
    }).sort_values("importance", ascending=False)

    imp_df.to_csv(f"{output_dir}/{prefix}_feature_importances.csv", index=False)

    # Feature importance plot
    plt.figure(figsize=(12, 8))

```

```

top_features = imp_df.head(15).iloc[:, :-1]
plt.barh(range(len(top_features)), top_features["importance"], alpha=0.7)
plt.yticks(range(len(top_features)), top_features["feature"])
plt.xlabel("Feature Importance Score")
plt.title(f"Top 15 Features for {prefix.upper()} Prediction (XGBoost)")
plt.grid(True, alpha=0.3, axis="x")
plt.tight_layout()
plt.savefig(f"{output_dir}/{prefix}_feature_importances.png", dpi=200,
↳bbox_inches="tight")
plt.show()

# ROC + PR curves
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))

fpr, tpr, _ = roc_curve(y_test, y_proba)
roc_auc = roc_auc_score(y_test, y_proba)
ax1.plot(fpr, tpr, label=f"AUC = {roc_auc:.3f}", linewidth=2, color="C0")
ax1.plot([0, 1], [0, 1], "k--", alpha=0.5)
ax1.set_xlabel("False Positive Rate")
ax1.set_ylabel("True Positive Rate")
ax1.set_title(f"ROC Curve ({prefix.upper()}")
ax1.legend()
ax1.grid(True, alpha=0.3)

precision_vals, recall_vals, _ = precision_recall_curve(y_test, y_proba)
pr_auc = average_precision_score(y_test, y_proba)
ax2.plot(recall_vals, precision_vals, label=f"AP = {pr_auc:.3f}",
↳linewidth=2, color="C1")
ax2.set_xlabel("Recall")
ax2.set_ylabel("Precision")
ax2.set_title(f"Precision-Recall Curve ({prefix.upper()}")
ax2.legend()
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig(f"{output_dir}/{prefix}_roc_pr_curves.png", dpi=200,
↳bbox_inches="tight")
plt.show()

# Confusion matrix heatmap
y_pred = (y_proba >= 0.5).astype(int)
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=["Pred 0", "Pred 1"],
            yticklabels=["True 0", "True 1"])

```

```

plt.title(f"Confusion Matrix ({prefix.upper()}) (XGBoost)")
plt.ylabel("True Label")
plt.xlabel("Predicted Label")
plt.tight_layout()
plt.savefig(f"{output_dir}/{prefix}_confusion_matrix.png", dpi=200,
↳ bbox_inches="tight")
plt.show()

if cv_metrics:
    create_validation_comparison_plot(cv_metrics, output_dir, prefix)

return imp_df

# =====
# 8) SAVE RESULTS
# =====
def save_comprehensive_results(model, metrics, imp_df, df_clean, target_col,
↳ feature_names, output_dir, prefix, cv_metrics=None):
    print(f"\n SAVING OUTPUTS to {output_dir}/ ...")
    os.makedirs(output_dir, exist_ok=True)

    with open(f"{output_dir}/{prefix}_comprehensive_metrics.txt", "w",
↳ encoding="utf-8") as f:
        f.write(f"COMPREHENSIVE XGBOOST RESULTS ({prefix.upper()})\n")
        f.write("=" * 60 + "\n")
        f.write(f"Dataset shape: {df_clean.shape}\n")
        f.write(f"Target column: {target_col}\n")
        f.write(f"Features used: {len(feature_names)}\n\n")

        f.write("TEST SET METRICS:\n")
        for k, v in metrics.items():
            f.write(f"{k}: {v:.4f}\n")
        f.write("\n")

        if cv_metrics:
            f.write("10-FOLD CROSS-VALIDATION (Mean ± Std) WITH MULTIPLE
↳ METRICS:\n")
            for metric, scores in cv_metrics.items():
                f.write(f"{metric}: {scores['test_mean']:.4f} ±
↳ {scores['test_std']:.4f}\n")
            f.write("\n")

        f.write("TOP 10 FEATURES:\n")
        for _, row in imp_df.head(10).iterrows():
            f.write(f"{row['feature']}: {row['importance']:.6f}\n")

```

```

# Full-dataset predictions
X_full = df_clean.drop(columns=[target_col], errors="ignore")
X_full = X_full.reindex(columns=feature_names)

preds = model.predict_proba(X_full)[: , 1]
predictions_df = df_clean.copy()
predictions_df[f"{prefix}_Pred_Prob"] = preds
predictions_df[f"{prefix}_Pred"] = (preds >= 0.5).astype(int)
predictions_df[f"{prefix}_Correct"] = (predictions_df[f"{prefix}_Pred"] ==
↳ predictions_df[target_col]).astype(int)

predictions_df.to_excel(f"{output_dir}/{prefix}_predictions.xlsx",
↳ index=False)
joblib.dump(model, f"{output_dir}/{prefix}_xgb_model.joblib")

print(" Saved metrics, plots, predictions, and model.")

# =====
# 9) MAIN PIPELINE (UNIVERSAL TARGET)
# =====
def run_comprehensive_analysis(
    target="CI",
    use_cross_validation=True,
    use_validation_set=True,
    df=None
):
    """
    Run the full pipeline for ANY target column you type.
    """
    try:
        prefix = re.sub(r"^[a-zA-Z0-9]+", "_", str(target).strip().lower()).
↳ strip("_")
        output_dir = f"{prefix}_comprehensive_outputs_xgb"

        print(" COMPREHENSIVE MICROFINANCE ANALYSIS (XGBoost)")
        print("=" * 60)
        print(f" Target: {target}")
        print(f" Output folder: {output_dir}/")
        print("=" * 60)

        if df is None:
            try:
                import google.colab # noqa: F401
                df = load_excel_from_colab()
            except Exception:
                df = load_excel_locally()

```

```

print(f" Initial dataset shape: {df.shape}")

target_col = detect_target_column(df, target=target)

y, valid_mask = preprocess_target_binary(df[target_col],
↳target_label=target_col)
df_clean = df.loc[valid_mask].copy()
print(f" Clean dataset: {df_clean.shape} (after removing invalid
↳target rows)")

X_final, feature_names = build_feature_matrix(df_clean, target_col)

cv_metrics = None
if use_cross_validation:
    cv_metrics, _ = perform_cross_validation(X_final, y, cv_folds=10)

model, X_train, X_test, X_val, y_train, y_test, y_val =
↳train_with_comprehensive_validation(
    X_final, y, use_validation_set=use_validation_set
)

_, y_proba, test_metrics = evaluate_model_comprehensive(model, X_test,
↳y_test, "Test")

if X_val is not None and y_val is not None:
    evaluate_model_comprehensive(model, X_val, y_val, "Validation")

imp_df = create_comprehensive_visualizations(
    y_test=y_test,
    y_proba=y_proba,
    model=model,
    feature_names=feature_names,
    output_dir=output_dir,
    prefix=prefix,
    cv_metrics=cv_metrics
)

save_comprehensive_results(
    model=model,
    metrics=test_metrics,
    imp_df=imp_df,
    df_clean=df_clean,
    target_col=target_col,
    feature_names=feature_names,
    output_dir=output_dir,
    prefix=prefix,

```

```

        cv_metrics=cv_metrics
    )

    print("\n DONE.")
    return model, test_metrics, imp_df, cv_metrics

except Exception as e:
    print(f"\n Analysis failed: {type(e).__name__}: {e}")
    import traceback
    traceback.print_exc(limit=2)
    return None, None, None, None

# ===== USAGE EXAMPLES =====
if __name__ == "__main__":
    run_comprehensive_analysis(target="BF6", use_cross_validation=True,
    ↪use_validation_set=True)

    # Other examples:
    # run_comprehensive_analysis(target="CI", use_cross_validation=True,
    ↪use_validation_set=True)
    # for t in ["CI", "BF1", "BF3", "BF4"]:
    #     run_comprehensive_analysis(target=t, use_cross_validation=True,
    ↪use_validation_set=True)

```

COMPREHENSIVE MICROFINANCE ANALYSIS (XGBoost)

```

=====
Target: BF6
Output folder: bf6_comprehensive_outputs_xgb/
=====

Please select the Excel file with your encoded dataset...

<IPython.core.display.HTML object>

Saving rf_dataset_encoded_dataset.xlsx to rf_dataset_encoded_dataset (4).xlsx
Loaded dataset: (1002, 85) (rows, cols)
Initial dataset shape: (1002, 85)
Target column 'BF6' not found automatically.
Available columns:
['D', 'AY', 'AZ', 'BA', 'BB', 'BC', 'BD', 'BE_01', 'BE_02', 'BE_03', 'BE_04',
'BE_05', 'BE_06', 'BE_07', 'BE_08', 'BE_09', 'BE_10', 'BF', 'BG_01', 'BG_02',
'BG_03', 'BG_04', 'BG_05', 'BG_06', 'BH', 'BI', 'BN', 'BO', 'BP', 'BQ', 'BR',
'BS', 'BT', 'BU', 'BV', 'BW_01', 'BW_02', 'BW_03', 'BW_04', 'BW_05', 'BX', 'BY',
'CB', 'CD', 'CE', 'CF', 'CG', 'CH', 'CI', 'CK', 'CL', 'CM_01', 'CM_02', 'CM_03',
'CM_04', 'CM_05', 'CM_06', 'CN', 'CO_01', 'CO_02', 'CO_03', 'CO_04', 'CO_05',
'CO_06', 'CO_07', 'CO_08', 'CO_09', 'CP', 'CR_01', 'CR_02', 'CR_03', 'CR_04',
'CS', 'CT_01', 'CT_02', 'CT_03', 'CT_04', 'CT_05', 'CT_06', 'CU', 'Occupation',
'Financial Behavior', 'Family', 'Lifestyle', 'Digital Literacy']

```

Enter the exact target column name: BG_06
Using specified target column: BG_06

TARGET ANALYSIS: BG_06

Value distribution:

BG_06

0 783

1 219

Name: count, dtype: int64

Unique values: [np.int64(0), np.int64(1)]

Final class balance: {0: 0.781, 1: 0.219}

Clean dataset: (1002, 85) (after removing invalid target rows)

FEATURE ENGINEERING

Found 84 numeric features

Handling missing values with median imputation...

Final feature matrix: (1002, 84) (rows, cols)

PERFORMING 10-FOLD CROSS-VALIDATION

=====

CROSS-VALIDATION RESULTS (Mean ± Std):

ACCURACY	: 0.9021 ± 0.0400 (train: 0.9989)
PRECISION	: 0.7947 ± 0.1002 (train: 0.9950)
RECALL	: 0.7431 ± 0.1256 (train: 1.0000)
F1	: 0.7651 ± 0.1072 (train: 0.9975)
ROC_AUC	: 0.9305 ± 0.0487 (train: 0.9995)
AVERAGE_PRECISION	: 0.8598 ± 0.1011 (train: 0.9975)

Overfitting Analysis:

Train-Test Accuracy Gap: 0.0968

Potential overfitting detected

MODEL TRAINING: XGBoost

CREATING VALIDATION SET (20% of data)

=====

Training set: 600 samples

Test set: 201 samples

Validation set: 201 samples

Training model with regularization...

=====

TEST SET COMPREHENSIVE EVALUATION

=====

Accuracy: 0.8955
Precision: 0.8286
Recall: 0.6591
F1-Score: 0.7342
ROC-AUC: 0.9053
PR-AUC: 0.8369

Test Classification Report:

	precision	recall	f1-score	support
0	0.910	0.962	0.935	157
1	0.829	0.659	0.734	44
accuracy			0.896	201
macro avg	0.869	0.810	0.835	201
weighted avg	0.892	0.896	0.891	201

Test Confusion Matrix:

```
[[TN FP]
 [FN TP]]
[[151  6]
 [ 15 29]]
```

VALIDATION SET COMPREHENSIVE EVALUATION

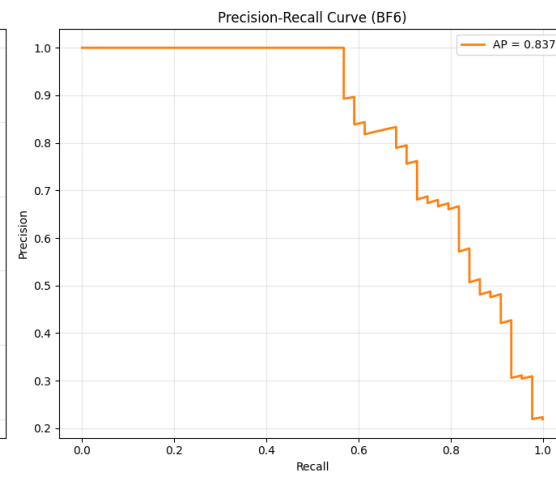
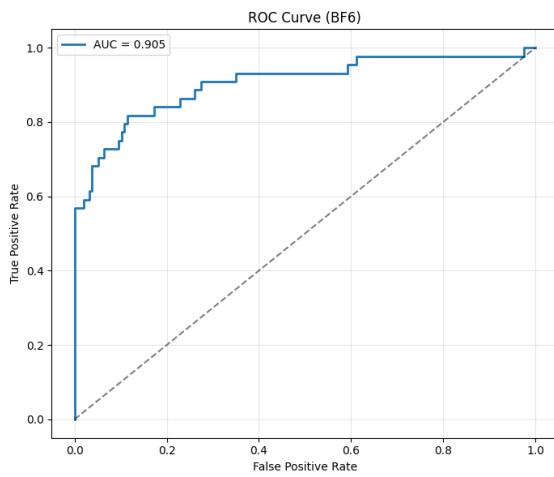
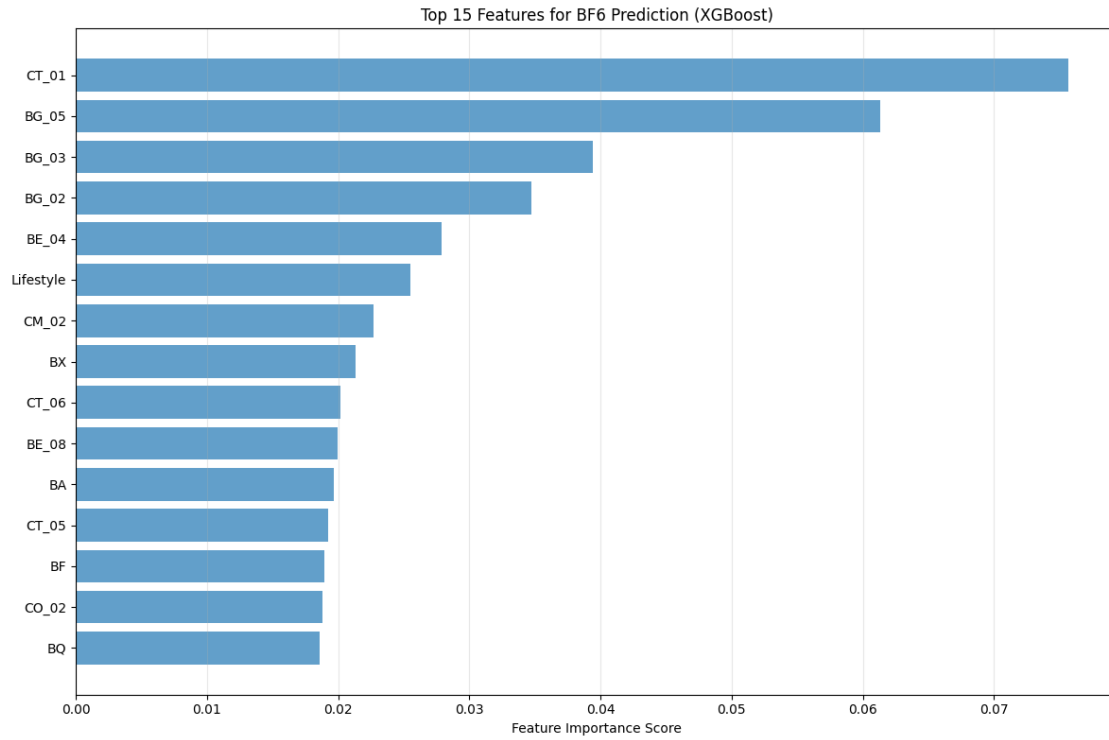
Accuracy: 0.8408
Precision: 0.6579
Recall: 0.5682
F1-Score: 0.6098
ROC-AUC: 0.8510
PR-AUC: 0.7335

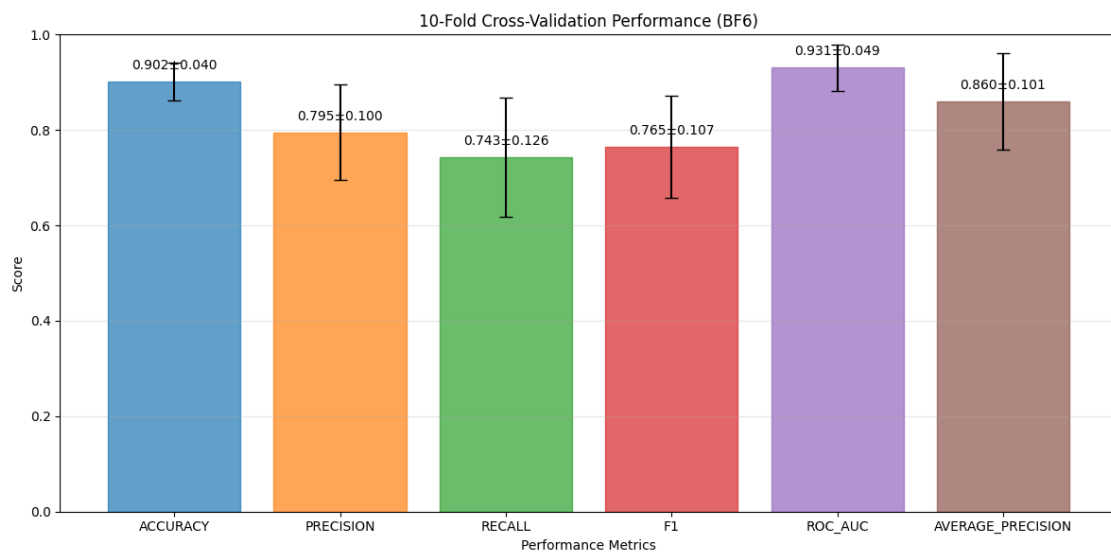
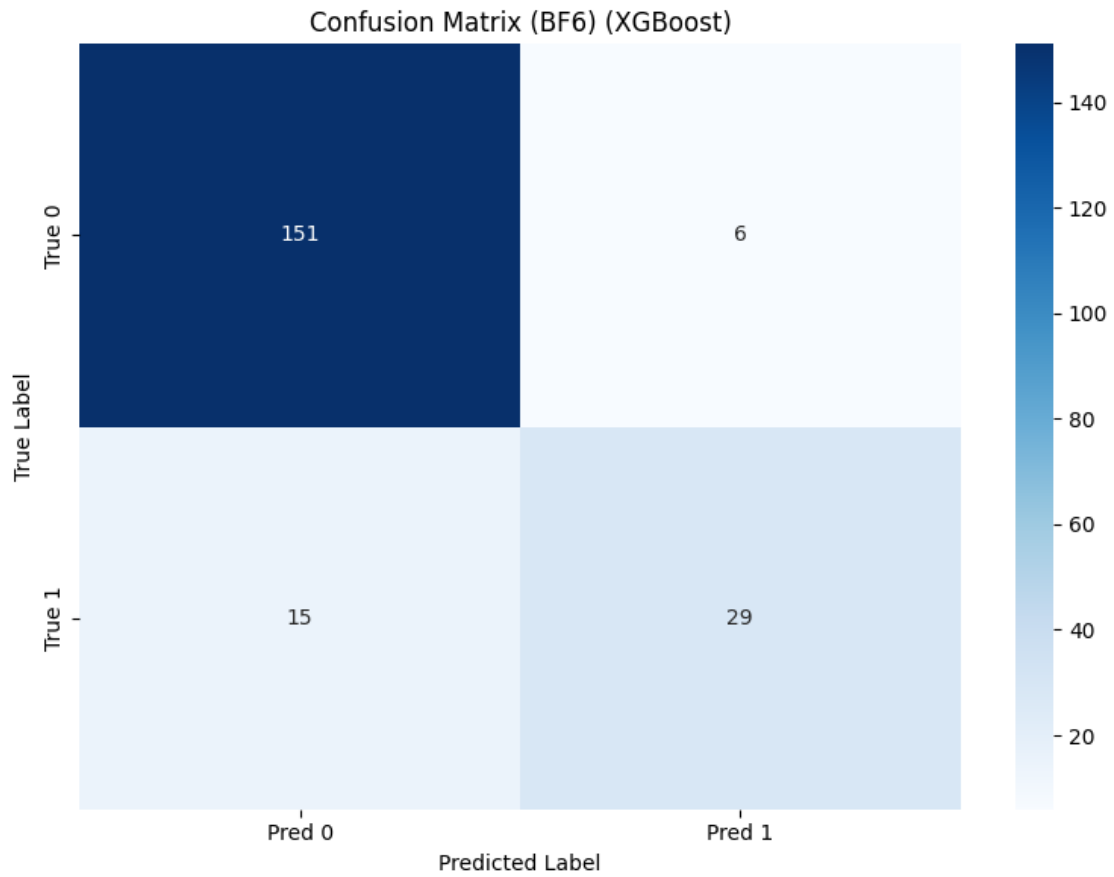
Validation Classification Report:

	precision	recall	f1-score	support
0	0.883	0.917	0.900	157
1	0.658	0.568	0.610	44
accuracy			0.841	201
macro avg	0.771	0.743	0.755	201
weighted avg	0.834	0.841	0.836	201

Validation Confusion Matrix:

```
[[TN FP]
 [FN TP]]
[[144 13]
 [ 19 25]]
```





SAVING OUTPUTS to bf6_comprehensive_outputs_xgb/ ...
Saved metrics, plots, predictions, and model.

DONE.

```
[6]: from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
[ ]: # Install the package for Tex and then convert to PDF directly as LaTeX
!sudo apt-get install texlive-xetex texlive-fonts-recommended
    ↳ texlive-plain-generic pandoc
# Provide the file path of the notebook file inside the quotations
!jupyter nbconvert --to pdf "/content/drive/MyDrive/Colab Notebooks/CSE thesis
    ↳ XGBoost P3.ipynb"
```

Reading package lists... Done

Building dependency tree... Done

Reading state information... Done

pandoc is already the newest version (2.9.2.1-3ubuntu2).

pandoc set to manually installed.

The following additional packages will be installed:

dvisvgm fonts-droid-fallback fonts-lato fonts-lmodern fonts-noto-mono
fonts-texgyre fonts-urw-base35 libapache-pom-java libcommons-logging-java
libcommons-parent-java libfontbox-java libgs9 libgs9-common libidn12
libijs-0.35 libjbig2dec0 libkpathsea6 libpdfbox-java libptexenc1 libruby3.0
libsyntax2 libteckit0 libtexlua53 libtexluajit2 libwoff1 libzip-0-13
lmodern poppler-data preview-latex-style rake ruby ruby-net-telnet
ruby-rubygems ruby-webrick ruby-xmlrpc ruby3.0 rubygems-integration t1utils
teckit tex-common tex-gyre texlive-base texlive-binaries texlive-latex-base
texlive-latex-extra texlive-latex-recommended texlive-pictures tipa
xfonts-encodings xfonts-utils

Suggested packages:

fonts-noto fonts-freefont-otf | fonts-freefont-ttf libavalon-framework-java
libcommons-logging-java-doc libexcalibur-logkit-java liblog4j1.2-java
poppler-utils ghostscript fonts-japanese-mincho | fonts-ipafont-mincho
fonts-japanese-gothic | fonts-ipafont-gothic fonts-arphic-ukai
fonts-arphic-uming fonts-nanum ri ruby-dev bundler debhelper gv
| postscript-viewer perl-tk xpdf | pdf-viewer xzdec
texlive-fonts-recommended-doc texlive-latex-base-doc python3-pygments
icc-profiles libfile-which-perl libspreadsheet-parseexcel-perl
texlive-latex-extra-doc texlive-latex-recommended-doc texlive-luatex
texlive-pstricks dot2tex prerex texlive-pictures-doc vprerex
default-jre-headless tipa-doc

The following NEW packages will be installed:

dvisvgm fonts-droid-fallback fonts-lato fonts-lmodern fonts-noto-mono
fonts-texgyre fonts-urw-base35 libapache-pom-java libcommons-logging-java

```

libcommons-parent-java libfontbox-java libgs9 libgs9-common libidn12
libijs-0.35 libjbig2dec0 libkpathsea6 libpdfbox-java libptexenc1 libruby3.0
libsyntax2 libteckit0 libtexlua53 libtexlua53 libwoff1 libzip-0-13
lmodern poppler-data preview-latex-style rake ruby ruby-net-telnet
ruby-rubygems ruby-webrick ruby-xmlrpc ruby3.0 rubygems-integration tiutils
teckit tex-common tex-gyre texlive-base texlive-binaries
texlive-fonts-recommended texlive-latex-base texlive-latex-extra
texlive-latex-recommended texlive-pictures texlive-plain-generic
texlive-xetex tipa xfonts-encodings xfonts-utils
0 upgraded, 53 newly installed, 0 to remove and 2 not upgraded.
Need to get 182 MB of archives.
After this operation, 571 MB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu jammy/main amd64 fonts-droid-fallback all
1:6.0.1r16-1.1build1 [1,805 kB]
Get:2 http://archive.ubuntu.com/ubuntu jammy/main amd64 fonts-lato all 2.0-2.1
[2,696 kB]
Get:3 http://archive.ubuntu.com/ubuntu jammy/main amd64 poppler-data all
0.4.11-1 [2,171 kB]
Get:4 http://archive.ubuntu.com/ubuntu jammy/universe amd64 tex-common all 6.17
[33.7 kB]
Get:5 http://archive.ubuntu.com/ubuntu jammy/main amd64 fonts-urw-base35 all
20200910-1 [6,367 kB]
Get:6 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libgs9-common
all 9.55.0~dfsg1-0ubuntu5.13 [753 kB]
Get:7 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libidn12 amd64
1.38-4ubuntu1 [60.0 kB]
Get:8 http://archive.ubuntu.com/ubuntu jammy/main amd64 libijs-0.35 amd64
0.35-15build2 [16.5 kB]
Get:9 http://archive.ubuntu.com/ubuntu jammy/main amd64 libjbig2dec0 amd64
0.19-3build2 [64.7 kB]
Get:10 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libgs9 amd64
9.55.0~dfsg1-0ubuntu5.13 [5,032 kB]
Get:11 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libkpathsea6
amd64 2021.20210626.59705-1ubuntu0.2 [60.4 kB]
Get:12 http://archive.ubuntu.com/ubuntu jammy/main amd64 libwoff1 amd64
1.0.2-1build4 [45.2 kB]
Get:13 http://archive.ubuntu.com/ubuntu jammy/universe amd64 dvisvgm amd64
2.13.1-1 [1,221 kB]
Get:14 http://archive.ubuntu.com/ubuntu jammy/universe amd64 fonts-lmodern all
2.004.5-6.1 [4,532 kB]
Get:15 http://archive.ubuntu.com/ubuntu jammy/main amd64 fonts-noto-mono all
20201225-1build1 [397 kB]
Get:16 http://archive.ubuntu.com/ubuntu jammy/universe amd64 fonts-texgyre all
20180621-3.1 [10.2 MB]
Get:17 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libapache-pom-java
all 18-1 [4,720 B]
Get:18 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libcommons-parent-
java all 43-1 [10.8 kB]

```

Get:19 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 libcommons-logging-java all 1.2-2 [60.3 kB]

Get:20 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libptexenc1 amd64 2021.20210626.59705-1ubuntu0.2 [39.1 kB]

Get:21 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 rubygems-integration all 1.18 [5,336 B]

Get:22 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 ruby3.0 amd64 3.0.2-7ubuntu2.11 [50.1 kB]

Get:23 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 ruby-rubygems all 3.3.5-2ubuntu1.2 [228 kB]

Get:24 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 ruby amd64 1:3.0~exp1 [5,100 B]

Get:25 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 rake all 13.0.6-2 [61.7 kB]

Get:26 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 ruby-net-telnet all 0.1.1-2 [12.6 kB]

Get:27 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 ruby-webrick all 1.7.0-3ubuntu0.2 [52.5 kB]

Get:28 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 ruby-xmlrpc all 0.3.2-1ubuntu0.1 [24.9 kB]

Get:29 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libruby3.0 amd64 3.0.2-7ubuntu2.11 [5,114 kB]

Get:30 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libsynchronet2 amd64 2021.20210626.59705-1ubuntu0.2 [55.6 kB]

Get:31 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 libteckit0 amd64 2.5.11+ds1-1 [421 kB]

Get:32 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libtexlua53 amd64 2021.20210626.59705-1ubuntu0.2 [120 kB]

Get:33 <http://archive.ubuntu.com/ubuntu> jammy-updates/main amd64 libtexluajit2 amd64 2021.20210626.59705-1ubuntu0.2 [267 kB]

Get:34 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 libzip-0-13 amd64 0.13.72+dfsg.1-1.1 [27.0 kB]

Get:35 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 xfonts-encodings all 1:1.0.5-0ubuntu2 [578 kB]

Get:36 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 xfonts-utils amd64 1:7.7+6build2 [94.6 kB]

Get:37 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 lmodern all 2.004.5-6.1 [9,471 kB]

Get:38 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 preview-latex-style all 12.2-1ubuntu1 [185 kB]

Get:39 <http://archive.ubuntu.com/ubuntu> jammy/main amd64 t1utils amd64 1.41-4build2 [61.3 kB]

Get:40 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 teckit amd64 2.5.11+ds1-1 [699 kB]

Get:41 <http://archive.ubuntu.com/ubuntu> jammy/universe amd64 tex-gyre all 20180621-3.1 [6,209 kB]

Get:42 <http://archive.ubuntu.com/ubuntu> jammy-updates/universe amd64 texlive-binaries amd64 2021.20210626.59705-1ubuntu0.2 [9,860 kB]

```

Get:43 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-base all
2021.20220204-1 [21.0 MB]
Get:44 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-fonts-
recommended all 2021.20220204-1 [4,972 kB]
Get:45 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-latex-base
all 2021.20220204-1 [1,128 kB]
Get:46 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libfontbox-java all
1:1.8.16-2 [207 kB]
Get:47 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libpdfbox-java all
1:1.8.16-2 [5,199 kB]
Get:48 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-latex-
recommended all 2021.20220204-1 [14.4 MB]
Get:49 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-pictures
all 2021.20220204-1 [8,720 kB]
Get:50 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-latex-extra
all 2021.20220204-1 [13.9 MB]
Get:51 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-plain-
generic all 2021.20220204-1 [27.5 MB]
Get:52 http://archive.ubuntu.com/ubuntu jammy/universe amd64 tipa all 2:1.3-21
[2,967 kB]
Get:53 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-xetex all
2021.20220204-1 [12.4 MB]
Fetched 182 MB in 5s (33.3 MB/s)
debconf: unable to initialize frontend: Dialog
debconf: (No usable dialog-like program is installed, so the dialog based
frontend cannot be used. at /usr/share/perl5/Debconf/FrontEnd/Dialog.pm line 78,
<> line 53.)
debconf: falling back to frontend: Readline
debconf: unable to initialize frontend: Readline
debconf: (This frontend requires a controlling tty.)
debconf: falling back to frontend: Teletype
dpkg-preconfigure: unable to re-open stdin:
Selecting previously unselected package fonts-droid-fallback.
(Reading database ... 117540 files and directories currently installed.)
Preparing to unpack .../00-fonts-droid-fallback_1%3a6.0.1r16-1.1build1_all.deb
...
Unpacking fonts-droid-fallback (1:6.0.1r16-1.1build1) ...
Selecting previously unselected package fonts-lato.
Preparing to unpack .../01-fonts-lato_2.0-2.1_all.deb ...
Unpacking fonts-lato (2.0-2.1) ...
Selecting previously unselected package poppler-data.
Preparing to unpack .../02-poppler-data_0.4.11-1_all.deb ...
Unpacking poppler-data (0.4.11-1) ...
Selecting previously unselected package tex-common.
Preparing to unpack .../03-tex-common_6.17_all.deb ...
Unpacking tex-common (6.17) ...
Selecting previously unselected package fonts-urw-base35.
Preparing to unpack .../04-fonts-urw-base35_20200910-1_all.deb ...

```

```

Unpacking fonts-urw-base35 (20200910-1) ...
Selecting previously unselected package libgs9-common.
Preparing to unpack .../05-libgs9-common_9.55.0~dfsg1-0ubuntu5.13_all.deb ...
Unpacking libgs9-common (9.55.0~dfsg1-0ubuntu5.13) ...
Selecting previously unselected package libidn12:amd64.
Preparing to unpack .../06-libidn12_1.38-4ubuntu1_amd64.deb ...
Unpacking libidn12:amd64 (1.38-4ubuntu1) ...
Selecting previously unselected package libijs-0.35:amd64.
Preparing to unpack .../07-libijs-0.35_0.35-15build2_amd64.deb ...
Unpacking libijs-0.35:amd64 (0.35-15build2) ...
Selecting previously unselected package libjbig2dec0:amd64.
Preparing to unpack .../08-libjbig2dec0_0.19-3build2_amd64.deb ...
Unpacking libjbig2dec0:amd64 (0.19-3build2) ...
Selecting previously unselected package libgs9:amd64.
Preparing to unpack .../09-libgs9_9.55.0~dfsg1-0ubuntu5.13_amd64.deb ...
Unpacking libgs9:amd64 (9.55.0~dfsg1-0ubuntu5.13) ...
Selecting previously unselected package libkpathsea6:amd64.
Preparing to unpack .../10-libkpathsea6_2021.20210626.59705-1ubuntu0.2_amd64.deb
...
Unpacking libkpathsea6:amd64 (2021.20210626.59705-1ubuntu0.2) ...
Selecting previously unselected package libwoff1:amd64.
Preparing to unpack .../11-libwoff1_1.0.2-1build4_amd64.deb ...
Unpacking libwoff1:amd64 (1.0.2-1build4) ...
Selecting previously unselected package dvisvgm.
Preparing to unpack .../12-dvisvgm_2.13.1-1_amd64.deb ...
Unpacking dvisvgm (2.13.1-1) ...
Selecting previously unselected package fonts-lmodern.
Preparing to unpack .../13-fonts-lmodern_2.004.5-6.1_all.deb ...
Unpacking fonts-lmodern (2.004.5-6.1) ...
Selecting previously unselected package fonts-noto-mono.
Preparing to unpack .../14-fonts-noto-mono_20201225-1build1_all.deb ...
Unpacking fonts-noto-mono (20201225-1build1) ...
Selecting previously unselected package fonts-texgyre.
Preparing to unpack .../15-fonts-texgyre_20180621-3.1_all.deb ...
Unpacking fonts-texgyre (20180621-3.1) ...
Selecting previously unselected package libapache-pom-java.
Preparing to unpack .../16-libapache-pom-java_18-1_all.deb ...
Unpacking libapache-pom-java (18-1) ...
Selecting previously unselected package libcommons-parent-java.
Preparing to unpack .../17-libcommons-parent-java_43-1_all.deb ...
Unpacking libcommons-parent-java (43-1) ...
Selecting previously unselected package libcommons-logging-java.
Preparing to unpack .../18-libcommons-logging-java_1.2-2_all.deb ...
Unpacking libcommons-logging-java (1.2-2) ...
Selecting previously unselected package libptexenc1:amd64.
Preparing to unpack .../19-libptexenc1_2021.20210626.59705-1ubuntu0.2_amd64.deb
...
Unpacking libptexenc1:amd64 (2021.20210626.59705-1ubuntu0.2) ...

```

```

Selecting previously unselected package rubygems-integration.
Preparing to unpack .../20-rubygems-integration_1.18_all.deb ...
Unpacking rubygems-integration (1.18) ...
Selecting previously unselected package ruby3.0.
Preparing to unpack .../21-ruby3.0_3.0.2-7ubuntu2.11_amd64.deb ...
Unpacking ruby3.0 (3.0.2-7ubuntu2.11) ...
Selecting previously unselected package ruby-rubygems.
Preparing to unpack .../22-ruby-rubygems_3.3.5-2ubuntu1.2_all.deb ...
Unpacking ruby-rubygems (3.3.5-2ubuntu1.2) ...
Selecting previously unselected package ruby.
Preparing to unpack .../23-ruby_1%3a3.0~exp1_amd64.deb ...
Unpacking ruby (1:3.0~exp1) ...
Selecting previously unselected package rake.
Preparing to unpack .../24-rake_13.0.6-2_all.deb ...
Unpacking rake (13.0.6-2) ...
Selecting previously unselected package ruby-net-telnet.
Preparing to unpack .../25-ruby-net-telnet_0.1.1-2_all.deb ...
Unpacking ruby-net-telnet (0.1.1-2) ...
Selecting previously unselected package ruby-webrick.
Preparing to unpack .../26-ruby-webrick_1.7.0-3ubuntu0.2_all.deb ...
Unpacking ruby-webrick (1.7.0-3ubuntu0.2) ...
Selecting previously unselected package ruby-xmlrpc.
Preparing to unpack .../27-ruby-xmlrpc_0.3.2-1ubuntu0.1_all.deb ...
Unpacking ruby-xmlrpc (0.3.2-1ubuntu0.1) ...
Selecting previously unselected package libruby3.0:amd64.
Preparing to unpack .../28-libruby3.0_3.0.2-7ubuntu2.11_amd64.deb ...
Unpacking libruby3.0:amd64 (3.0.2-7ubuntu2.11) ...
Selecting previously unselected package libsyntax2:amd64.
Preparing to unpack .../29-libsyntax2_2021.20210626.59705-1ubuntu0.2_amd64.deb
...
Unpacking libsyntax2:amd64 (2021.20210626.59705-1ubuntu0.2) ...
Selecting previously unselected package libteckit0:amd64.
Preparing to unpack .../30-libteckit0_2.5.11+ds1-1_amd64.deb ...
Unpacking libteckit0:amd64 (2.5.11+ds1-1) ...
Selecting previously unselected package libtexlua53:amd64.
Preparing to unpack .../31-libtexlua53_2021.20210626.59705-1ubuntu0.2_amd64.deb
...
Unpacking libtexlua53:amd64 (2021.20210626.59705-1ubuntu0.2) ...
Selecting previously unselected package libtexluajit2:amd64.
Preparing to unpack
.../32-libtexluajit2_2021.20210626.59705-1ubuntu0.2_amd64.deb ...
Unpacking libtexluajit2:amd64 (2021.20210626.59705-1ubuntu0.2) ...
Selecting previously unselected package libzip-0-13:amd64.
Preparing to unpack .../33-libzip-0-13_0.13.72+dfsg.1-1.1_amd64.deb ...
Unpacking libzip-0-13:amd64 (0.13.72+dfsg.1-1.1) ...
Selecting previously unselected package xfonts-encodings.
Preparing to unpack .../34-xfonts-encodings_1%3a1.0.5-0ubuntu2_all.deb ...
Unpacking xfonts-encodings (1:1.0.5-0ubuntu2) ...

```

Selecting previously unselected package xfonts-utils.
 Preparing to unpack .../35-xfonts-utils_1%3a7.7+6build2_amd64.deb ...
 Unpacking xfonts-utils (1:7.7+6build2) ...
 Selecting previously unselected package lmodern.
 Preparing to unpack .../36-lmodern_2.004.5-6.1_all.deb ...
 Unpacking lmodern (2.004.5-6.1) ...
 Selecting previously unselected package preview-latex-style.
 Preparing to unpack .../37-preview-latex-style_12.2-1ubuntu1_all.deb ...
 Unpacking preview-latex-style (12.2-1ubuntu1) ...
 Selecting previously unselected package t1utils.
 Preparing to unpack .../38-t1utils_1.41-4build2_amd64.deb ...
 Unpacking t1utils (1.41-4build2) ...
 Selecting previously unselected package teckit.
 Preparing to unpack .../39-teckit_2.5.11+ds1-1_amd64.deb ...
 Unpacking teckit (2.5.11+ds1-1) ...
 Selecting previously unselected package tex-gyre.
 Preparing to unpack .../40-tex-gyre_20180621-3.1_all.deb ...
 Unpacking tex-gyre (20180621-3.1) ...
 Selecting previously unselected package texlive-binaries.
 Preparing to unpack .../41-texlive-binaries_2021.20210626.59705-1ubuntu0.2_amd64.deb ...
 Unpacking texlive-binaries (2021.20210626.59705-1ubuntu0.2) ...
 Selecting previously unselected package texlive-base.
 Preparing to unpack .../42-texlive-base_2021.20220204-1_all.deb ...
 Unpacking texlive-base (2021.20220204-1) ...
 Selecting previously unselected package texlive-fonts-recommended.
 Preparing to unpack .../43-texlive-fonts-recommended_2021.20220204-1_all.deb ...
 Unpacking texlive-fonts-recommended (2021.20220204-1) ...
 Selecting previously unselected package texlive-latex-base.
 Preparing to unpack .../44-texlive-latex-base_2021.20220204-1_all.deb ...
 Unpacking texlive-latex-base (2021.20220204-1) ...
 Selecting previously unselected package libfontbox-java.
 Preparing to unpack .../45-libfontbox-java_1%3a1.8.16-2_all.deb ...
 Unpacking libfontbox-java (1:1.8.16-2) ...
 Selecting previously unselected package libpdfbox-java.
 Preparing to unpack .../46-libpdfbox-java_1%3a1.8.16-2_all.deb ...
 Unpacking libpdfbox-java (1:1.8.16-2) ...
 Selecting previously unselected package texlive-latex-recommended.
 Preparing to unpack .../47-texlive-latex-recommended_2021.20220204-1_all.deb ...
 Unpacking texlive-latex-recommended (2021.20220204-1) ...
 Selecting previously unselected package texlive-pictures.
 Preparing to unpack .../48-texlive-pictures_2021.20220204-1_all.deb ...
 Unpacking texlive-pictures (2021.20220204-1) ...
 Selecting previously unselected package texlive-latex-extra.
 Preparing to unpack .../49-texlive-latex-extra_2021.20220204-1_all.deb ...
 Unpacking texlive-latex-extra (2021.20220204-1) ...
 Selecting previously unselected package texlive-plain-generic.
 Preparing to unpack .../50-texlive-plain-generic_2021.20220204-1_all.deb ...

```

Unpacking texlive-plain-generic (2021.20220204-1) ...
Selecting previously unselected package tipa.
Preparing to unpack .../51-tipa_2%3a1.3-21_all.deb ...
Unpacking tipa (2:1.3-21) ...
Selecting previously unselected package texlive-xetex.
Preparing to unpack .../52-texlive-xetex_2021.20220204-1_all.deb ...
Unpacking texlive-xetex (2021.20220204-1) ...
Setting up fonts-lato (2.0-2.1) ...
Setting up fonts-noto-mono (20201225-1build1) ...
Setting up libwoff1:amd64 (1.0.2-1build4) ...
Setting up libtexlua53:amd64 (2021.20210626.59705-1ubuntu0.2) ...
Setting up libijs-0.35:amd64 (0.35-15build2) ...
Setting up libtexluajit2:amd64 (2021.20210626.59705-1ubuntu0.2) ...
Setting up libfontbox-java (1:1.8.16-2) ...
Setting up rubygems-integration (1.18) ...
Setting up libzip-0-13:amd64 (0.13.72+dfsg.1-1.1) ...
Setting up fonts-urw-base35 (20200910-1) ...
Setting up poppler-data (0.4.11-1) ...
Setting up tex-common (6.17) ...
debconf: unable to initialize frontend: Dialog
debconf: (No usable dialog-like program is installed, so the dialog based
frontend cannot be used. at /usr/share/perl5/Debconf/FrontEnd/Dialog.pm line
78.)
debconf: falling back to frontend: Readline
update-language: texlive-base not installed and configured, doing nothing!
Setting up libjbig2dec0:amd64 (0.19-3build2) ...
Setting up libteckit0:amd64 (2.5.11+ds1-1) ...
Setting up libapache-pom-java (18-1) ...
Setting up ruby-net-telnet (0.1.1-2) ...
Setting up xfonts-encodings (1:1.0.5-0ubuntu2) ...
Setting up t1utils (1.41-4build2) ...
Setting up libidn12:amd64 (1.38-4ubuntu1) ...
Setting up fonts-texgyre (20180621-3.1) ...
Setting up libkpathsea6:amd64 (2021.20210626.59705-1ubuntu0.2) ...
Setting up ruby-webrick (1.7.0-3ubuntu0.2) ...
Setting up fonts-lmodern (2.004.5-6.1) ...
Setting up fonts-droid-fallback (1:6.0.1r16-1.1build1) ...
Setting up ruby-xmlrpc (0.3.2-1ubuntu0.1) ...
Setting up libsynchronet2:amd64 (2021.20210626.59705-1ubuntu0.2) ...
Setting up libgs9-common (9.55.0~dfsg1-0ubuntu5.13) ...
Setting up teckit (2.5.11+ds1-1) ...
Setting up libpdfbox-java (1:1.8.16-2) ...
Setting up libgs9:amd64 (9.55.0~dfsg1-0ubuntu5.13) ...
Setting up preview-latex-style (12.2-1ubuntu1) ...
Setting up libcommons-parent-java (43-1) ...
Setting up dvisvgm (2.13.1-1) ...
Setting up libcommons-logging-java (1.2-2) ...
Setting up xfonts-utils (1:7.7+6build2) ...

```

```

Setting up libptexenc1:amd64 (2021.20210626.59705-1ubuntu0.2) ...
Setting up texlive-binaries (2021.20210626.59705-1ubuntu0.2) ...
update-alternatives: using /usr/bin/xdvi-xaw to provide /usr/bin/xdvi.bin
(xdvi.bin) in auto mode
update-alternatives: using /usr/bin/bibtex.original to provide /usr/bin/bibtex
(bibtex) in auto mode
Setting up lmodern (2.004.5-6.1) ...
Setting up texlive-base (2021.20220204-1) ...
/usr/bin/ucfr
/usr/bin/ucfr
/usr/bin/ucfr
/usr/bin/ucfr
mktexlsr: Updating /var/lib/texmf/ls-R-TEXLIVEDIST...
mktexlsr: Updating /var/lib/texmf/ls-R-TEXMFMAIN...
mktexlsr: Updating /var/lib/texmf/ls-R...
mktexlsr: Done.
tl-paper: setting paper size for dvips to a4:
/var/lib/texmf/dvips/config/config-paper.ps
tl-paper: setting paper size for dvipdfmx to a4:
/var/lib/texmf/dvipdfmx/dvipdfmx-paper.cfg
tl-paper: setting paper size for xdvi to a4: /var/lib/texmf/xdvi/XDvi-paper
tl-paper: setting paper size for pdftex to a4: /var/lib/texmf/tex/generic/tex-
ini-files/pdftexconfig.tex
debconf: unable to initialize frontend: Dialog
debconf: (No usable dialog-like program is installed, so the dialog based
frontend cannot be used. at /usr/share/perl5/Debconf/FrontEnd/Dialog.pm line
78.)
debconf: falling back to frontend: Readline
Setting up tex-gyre (20180621-3.1) ...
Setting up texlive-plain-generic (2021.20220204-1) ...
Setting up texlive-latex-base (2021.20220204-1) ...
Setting up texlive-latex-recommended (2021.20220204-1) ...
Setting up texlive-pictures (2021.20220204-1) ...
Setting up texlive-fonts-recommended (2021.20220204-1) ...
Setting up tipa (2:1.3-21) ...
Setting up texlive-latex-extra (2021.20220204-1) ...
Setting up texlive-xetex (2021.20220204-1) ...
Setting up rake (13.0.6-2) ...
Setting up libruby3.0:amd64 (3.0.2-7ubuntu2.11) ...
Setting up ruby3.0 (3.0.2-7ubuntu2.11) ...
Setting up ruby (1:3.0~exp1) ...
Setting up ruby-rubygems (3.3.5-2ubuntu1.2) ...
Processing triggers for man-db (2.10.2-1) ...
Processing triggers for mailcap (3.70+nmu1ubuntu1) ...
Processing triggers for fontconfig (2.13.1-4.2ubuntu5) ...
Processing triggers for libc-bin (2.35-0ubuntu3.11) ...
/sbin/ldconfig.real: /usr/local/lib/libtcm.so.1 is not a symbolic link

```

```
/sbin/ldconfig.real: /usr/local/lib/libtbb.so.12 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbbind_2_5.so.3 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbbind.so.3 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbbind_2_0.so.3 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libur_adapter_level_zero_v2.so.0 is not a
symbolic link

/sbin/ldconfig.real: /usr/local/lib/libur_adapter_opencl.so.0 is not a symbolic
link

/sbin/ldconfig.real: /usr/local/lib/libtcm_debug.so.1 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libumf.so.1 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libur_adapter_level_zero.so.0 is not a
symbolic link

/sbin/ldconfig.real: /usr/local/lib/libhwloc.so.15 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libur_loader.so.0 is not a symbolic link

/sbin/ldconfig.real: /usr/local/lib/libtbbmalloc_proxy.so.2 is not a symbolic
link

/sbin/ldconfig.real: /usr/local/lib/libtbbmalloc.so.2 is not a symbolic link
```

```
Processing triggers for tex-common (6.17) ...
debconf: unable to initialize frontend: Dialog
debconf: (No usable dialog-like program is installed, so the dialog based
frontend cannot be used. at /usr/share/perl5/Debconf/FrontEnd/Dialog.pm line
78.)
debconf: falling back to frontend: Readline
Running upmap-sys. This may take some time... done.
Running mktexlsr /var/lib/texmf ... done.
Building format(s) --all.
    This may take some time...
```

[]: